

# Machine Learning



- ◇ Chapter IX:
- ◇ Machine Learning Modeling

◇ Prof. Dr. Aderval S. Luna

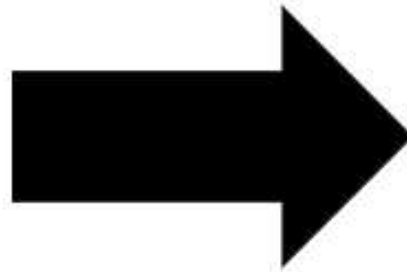
# Machine Learning: Modeling





## Training

Extract patterns from data



## Evaluating

Use patterns to predict results





# How to Use Data to Train Your Algorithm

## **Preparing**

the data by collecting, processing, annotating, and QA



## **Splitting**

the training data into training, validation, and testing sets



## **Training**

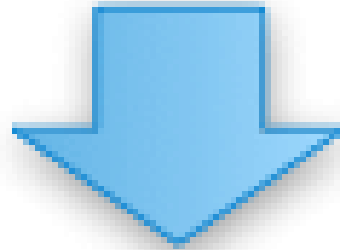
the algorithm by feeding the data into it



## **Refining**

the algorithm and re-training

Data



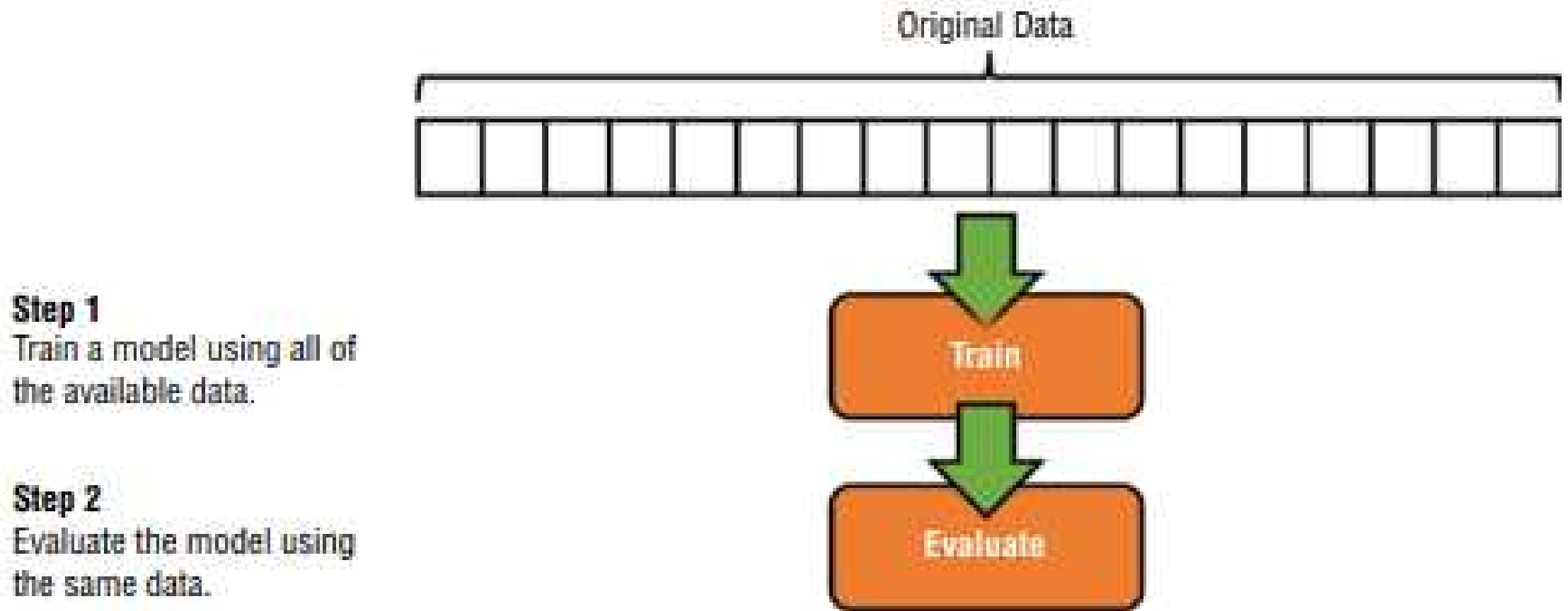
Training

Test

# Splitting the Dataset

- ◊ Without Splitting Method
- ◊ Hold-out Method
- ◊ Internal Validation Method
- ◊ k-Fold Cross-Validation Method
- ◊ Leave-one-out Cross-Validation Method
- ◊ Random Cross-Validation Method
- ◊ Bootstrap Method

# Without Splitting Method



**Figure 9.1** Model build and evaluation process using all of the observed data

# The Holdout Method

## Step 1

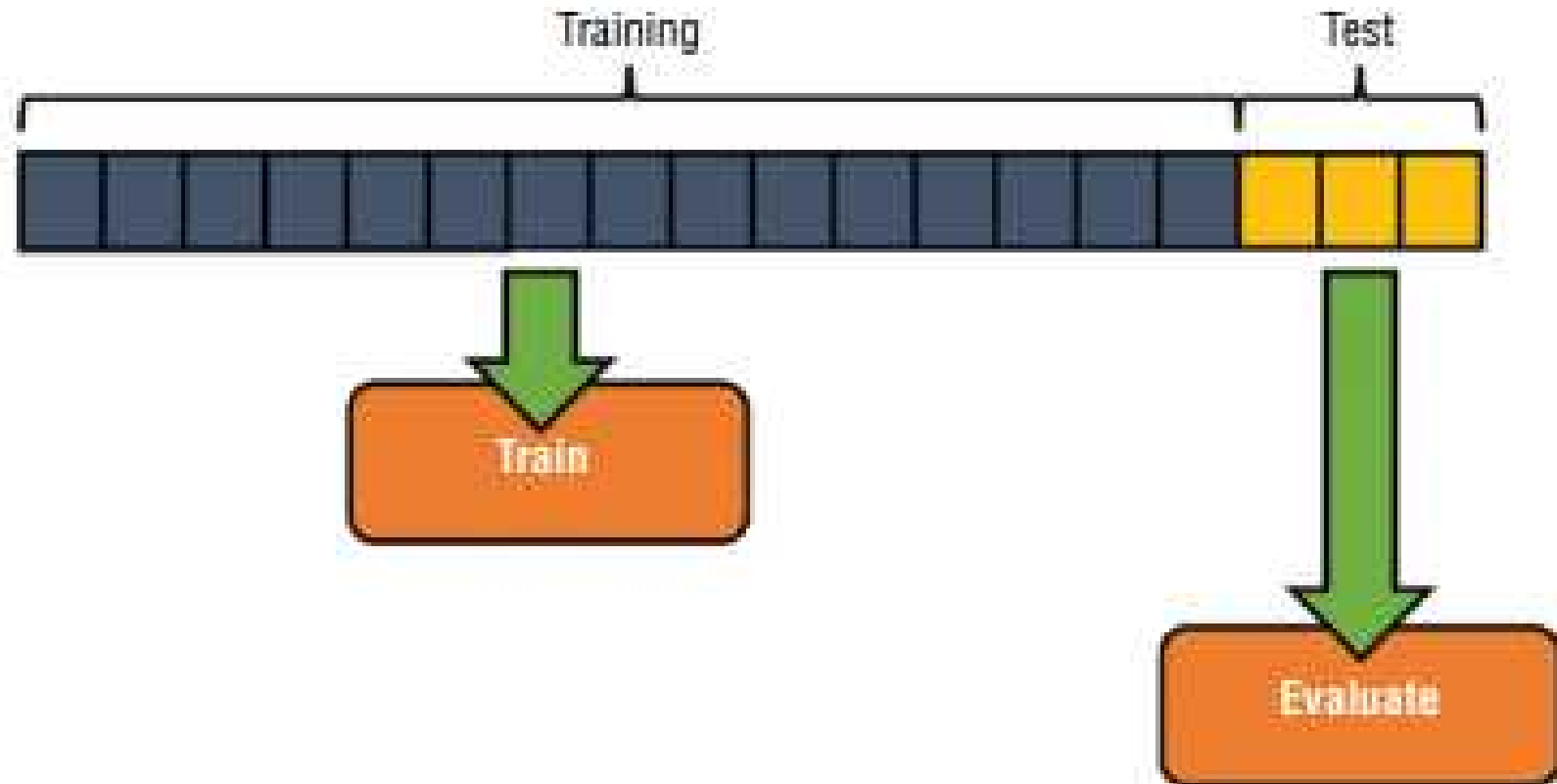
Split the data into training and test partitions.

## Step 2

Train a model using the training data.

## Step 3

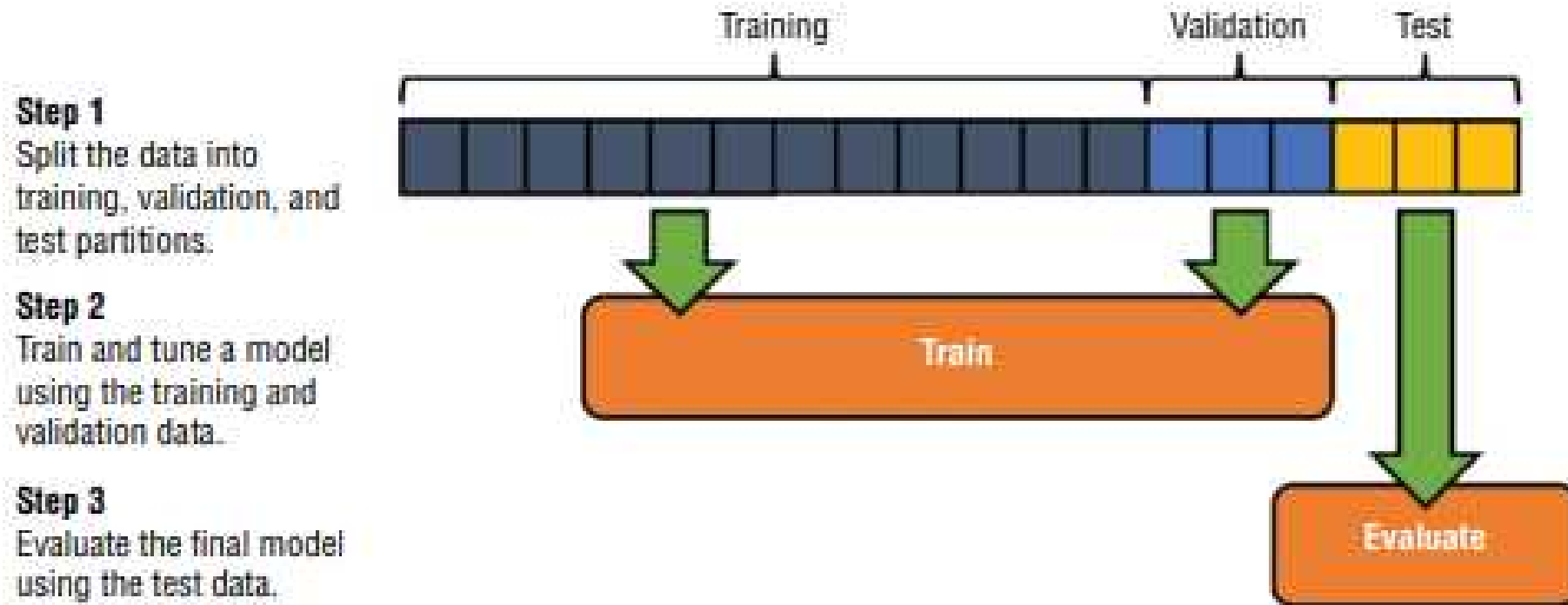
Evaluate the model using the test data.



**Figure 9.2** Model build and evaluation process using subsets of the observed data for training and for test (the holdout method)



# Internal Validation Method



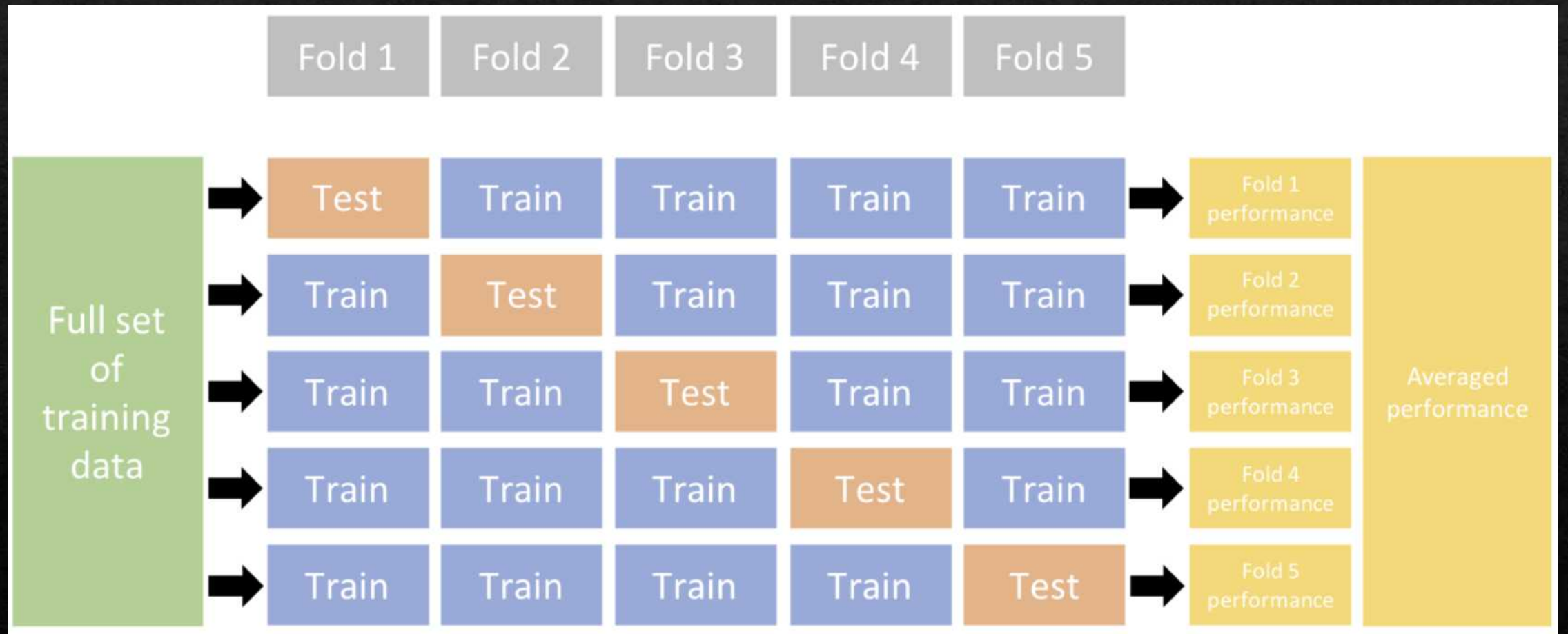
**Figure 9.3** Model build and evaluation process using the training and validation data to optimize and choose a model. The test data is used to estimate the future performance of the final model.

# The $k$ -Fold Cross-Validation



**Figure 9.4** The  $k$ -fold cross-validation approach with  $k=5$  (5-fold cross validation). A set of  $n$  examples is split into five independent folds.

# Illustration of the k-fold cross validation



# The Leave-One-Out Cross-Validation



**Figure 9.5** The leave-one-out cross-validation approach (LOOCV). A set of  $n$  examples with only one instance is used for validation in each iteration.



# Leave-One-Out Cross-Validation

Leave-One-Out Cross-Validation - the number of folds is the same as the number of training instances

Pros:

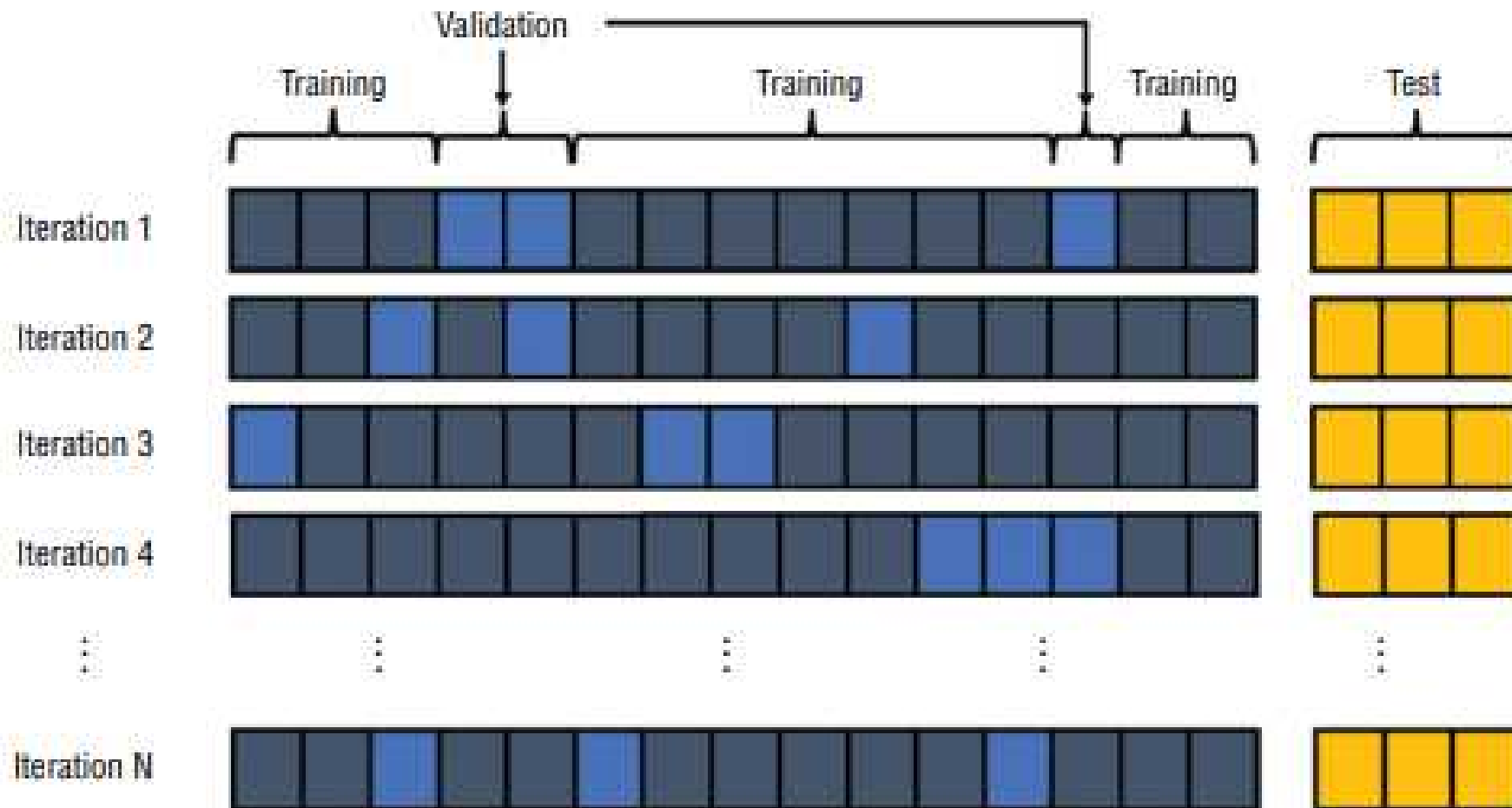
- Makes the best use of the data since the greatest possible amount of data is used for training
- Involves no random sampling

Cons:

- Computationally expensive (increases directly as there are more instances)
- None of the samples will be stratified



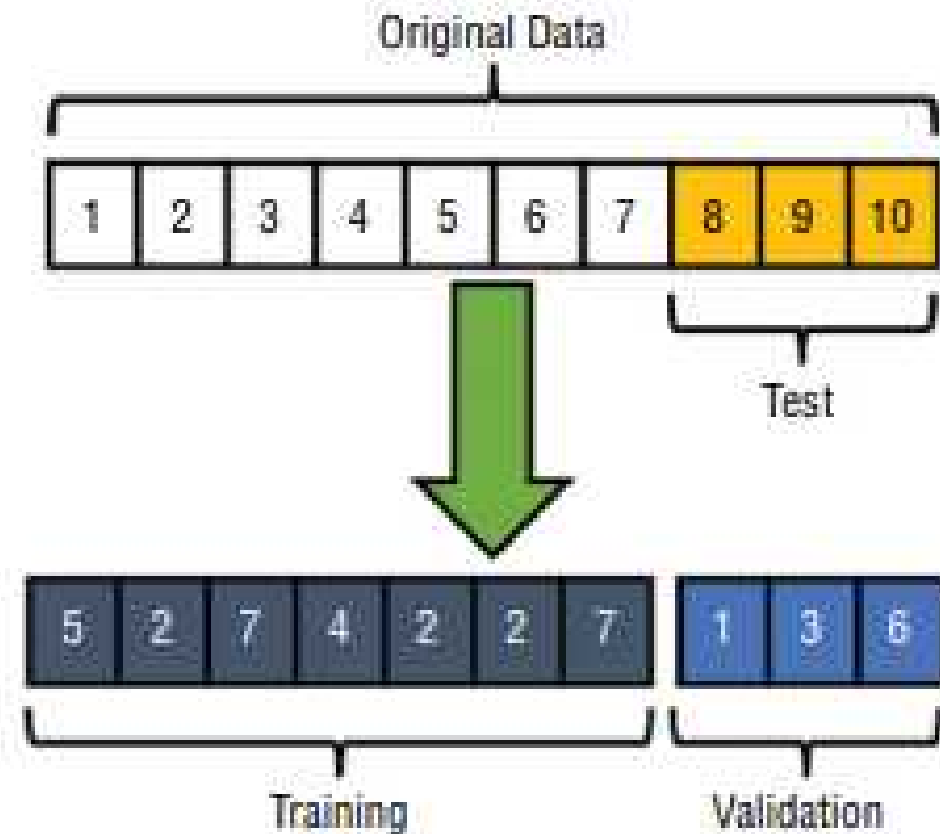
# The Random Cross-Validation Method



**Figure 9.6** The random cross-validation approach. The training and validation sets are created independently in each iteration.

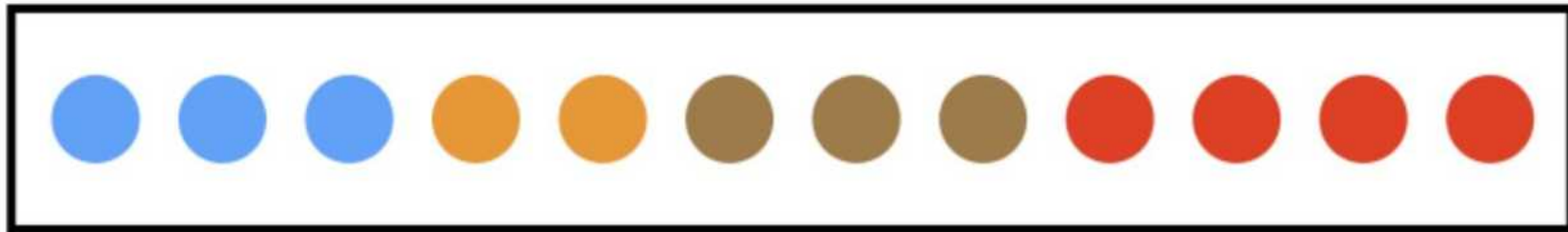
# The Bootstrap Sampling Method

**Figure 9.7** The bootstrap sampling approach. The training set is created by random sampling with replacement. Examples not selected as part of the training set are used for validation.

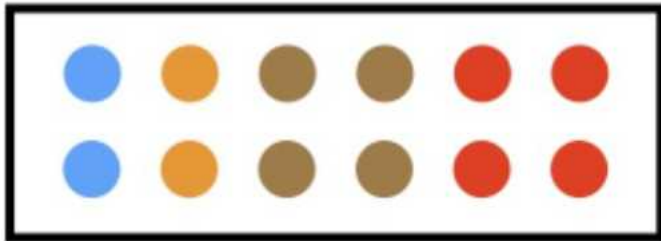


# Bootstrapping

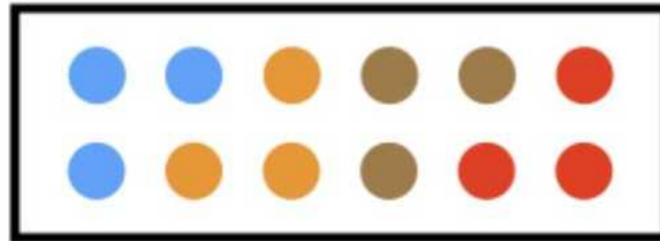
Original Data



Bootstrap sample 1



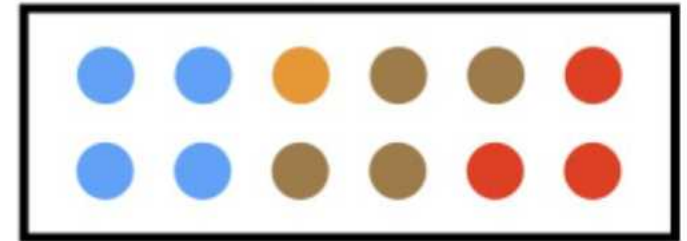
Bootstrap sample 2



...



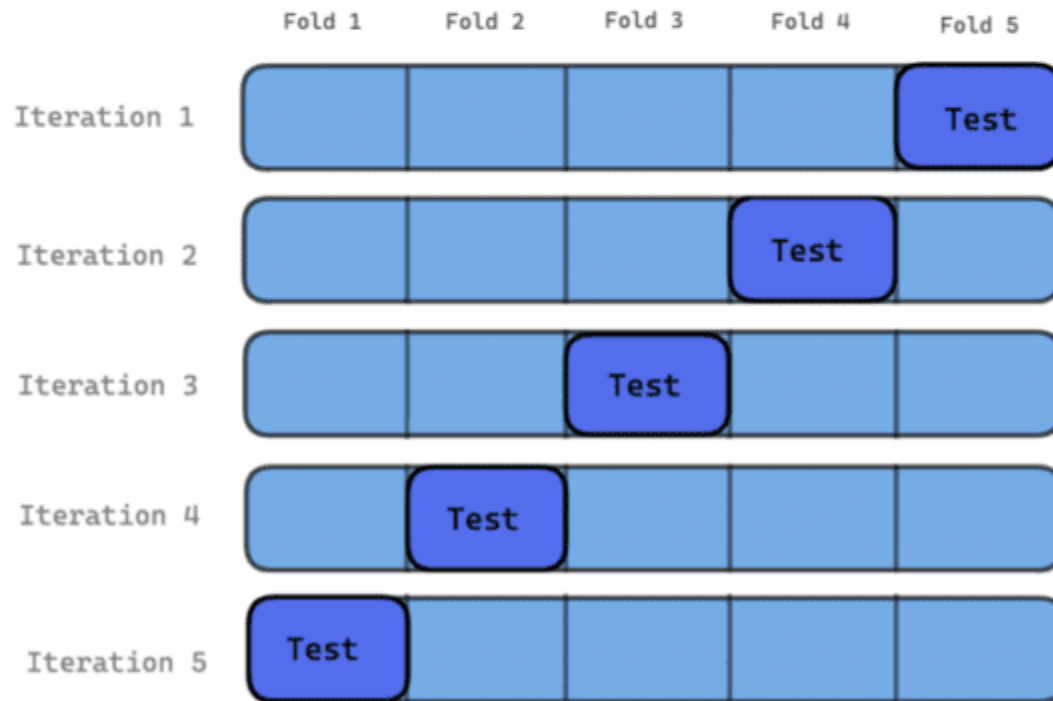
Bootstrap sample n



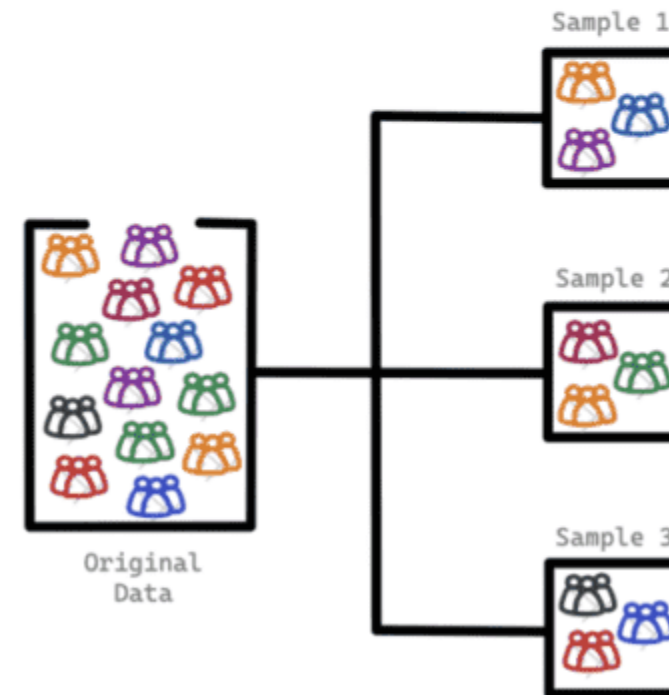


# Resampling Techniques in Data Science

## K-Fold Cross-Validation



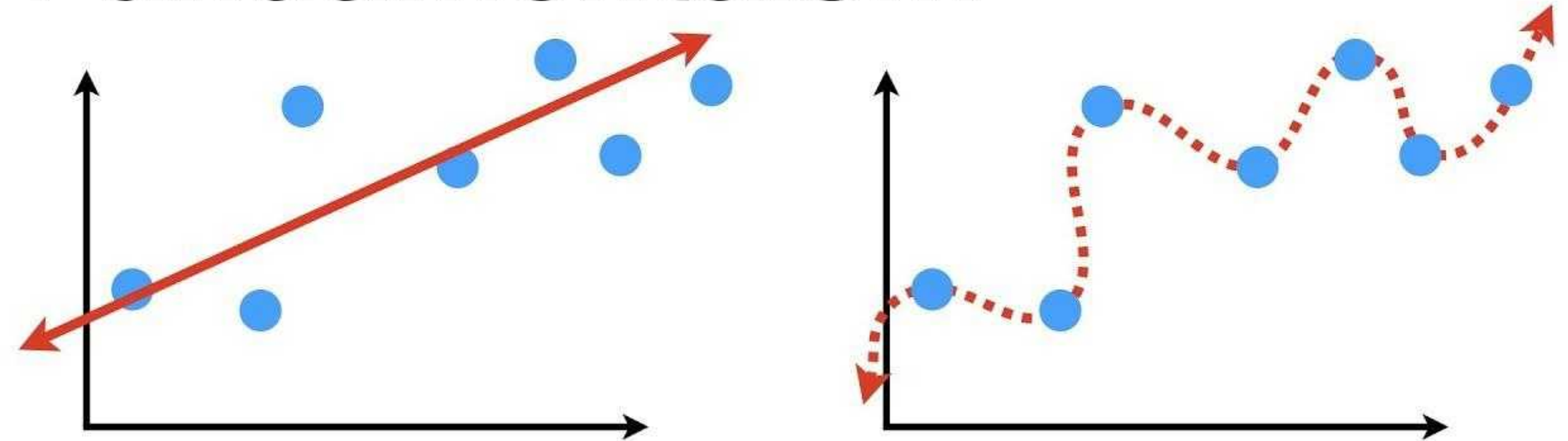
## Bootstrapping



| Method                    | Definition                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Limitations                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Holdout</b>            | The original data is divided into two sets, training and test set. The model is induced using the training set and then its performance is evaluated using the test set. The accuracy of the induced model on the test set is used to estimate the accuracy of the classifier.                                                                                                                                                                                                                                                                                    | <ol style="list-style-type: none"> <li>1. Because the data was divided into two sets fewer label examples are available for training. As a result, the induced model may not be as good</li> <li>2. It may be highly dependent on the composition of the training and test sets.</li> <li>3. The two sets are no longer independent because they are subsets of a larger set.</li> </ol> |
| <b>Random Subsampling</b> | Repeats the holdout method several times to improve estimation of a classifier's performance. Overall accuracy, $acc_{sub}$ is based on the accuracy for each run, $acc_i$ . $acc_{sub} = \sum_{i=1}^k acc_i / k$                                                                                                                                                                                                                                                                                                                                                 | <ol style="list-style-type: none"> <li>1. Still encounters some of the holdout problems.</li> <li>2. Some records might be used for training more often than others.</li> </ol>                                                                                                                                                                                                          |
| <b>Cross-Validation</b>   | Partitions the data into k disjoint subset. It then uses k-1 subsets for the training model and one set for the test set. It does this k number of times switching the test set each time thereby each record is used the same number of times for training and exactly one time for testing. The total error is found by summing the errors of all k runs.<br><br>Leave-one-out k=n:-basically all the data except for one is used for the training set and the one is used for the test set. This is good because it uses as much data as possible for training | <ol style="list-style-type: none"> <li>1. Training algorithm has to be rerun from scratch k times.</li> <li>2. When the data set size is small splitting it compromises it integrity.</li> </ol><br><ol style="list-style-type: none"> <li>1. Computationally expensive</li> <li>2. The variance of the estimated performance metric is high</li> </ol>                                  |
| <b>Bootstrap</b>          | The training records are sampled with replacement. "A common variation used is the .632 bootstrap which computes the overall accuracy by combining the accuracies of each bootstrap sample with the accuracy computed from a training set that contains all the labeled examples in the original data" <sup>14</sup>                                                                                                                                                                                                                                              | <ol style="list-style-type: none"> <li>1. Bootstrap can yield poor results in certain situations</li> <li>2. Though the bootstrap easily accommodates some violations of traditional statistical assumptions (e.g., non-normality), it is susceptible to others (e.g., non-independence)<sup>2</sup>.</li> </ol>                                                                         |



# Machine Learning Fundamentals...



**...Bias and Variance!!!**

# BIAS-VARIANCE TRADEOFF

The bias-variance tradeoff represents the balance between a model's simplicity (bias) and its sensitivity to fluctuations in the training data (variance). Striking the right balance minimizes the total error and achieves the best performance.

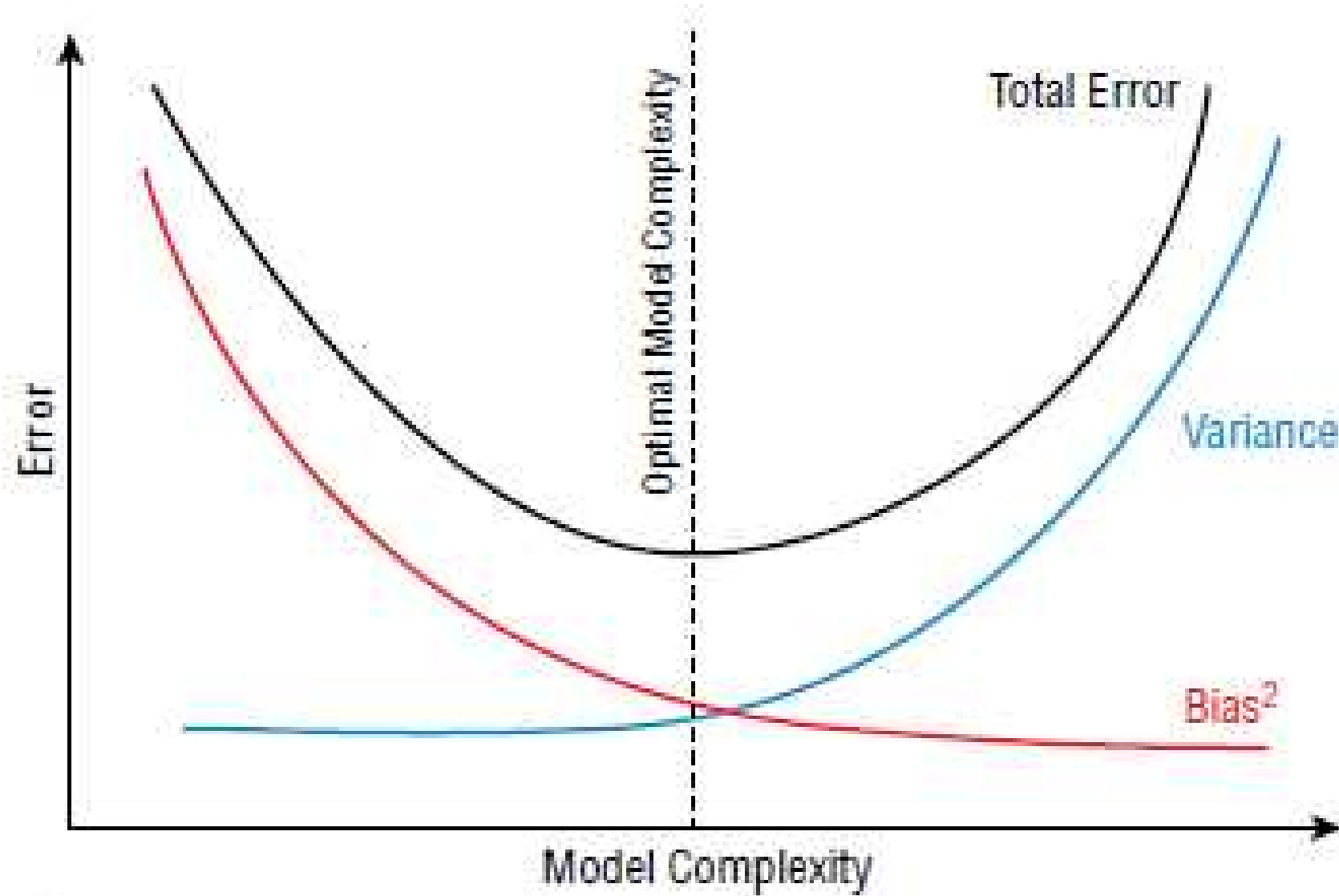
Bias<sup>2</sup> is the squared difference between the model's average prediction and the true values

Irreducible Error is the inherent noise in the data, which cannot be reduced by improving the model

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Variance measures the model's inconsistency across different training sets



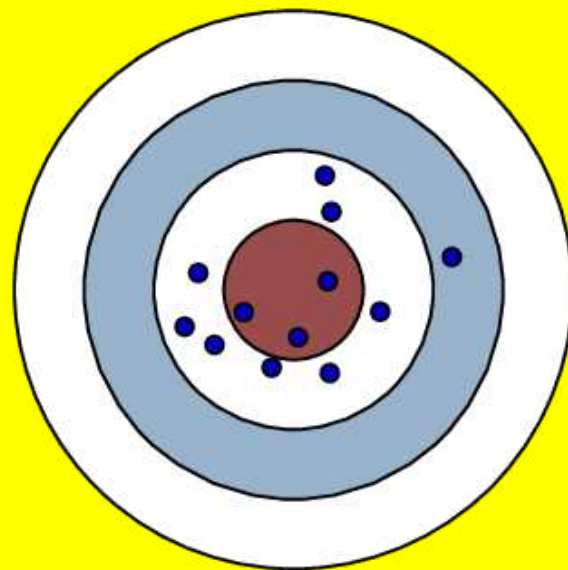
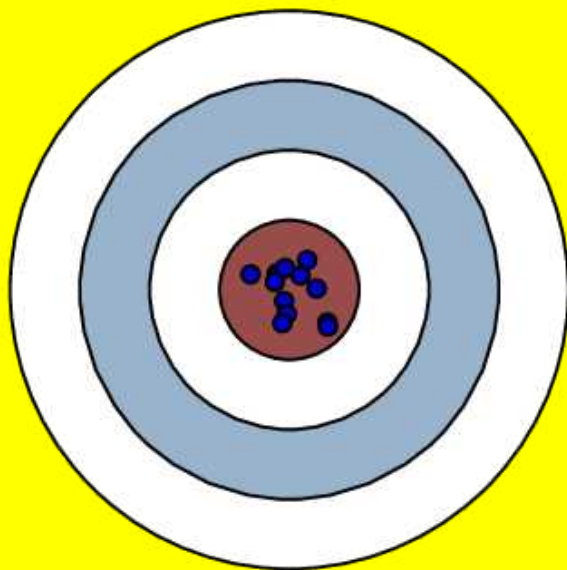


**Figure 1.11** The bias/variance trade-off

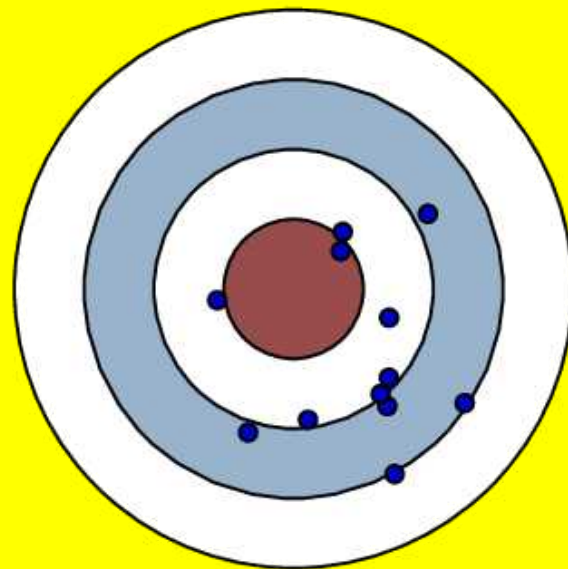
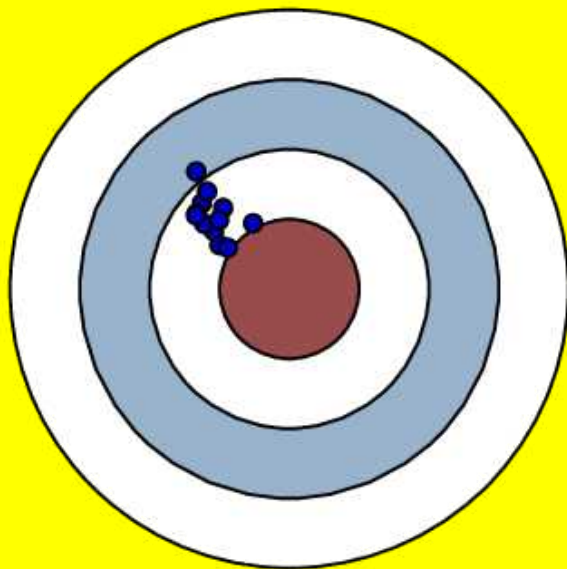
Low Variance

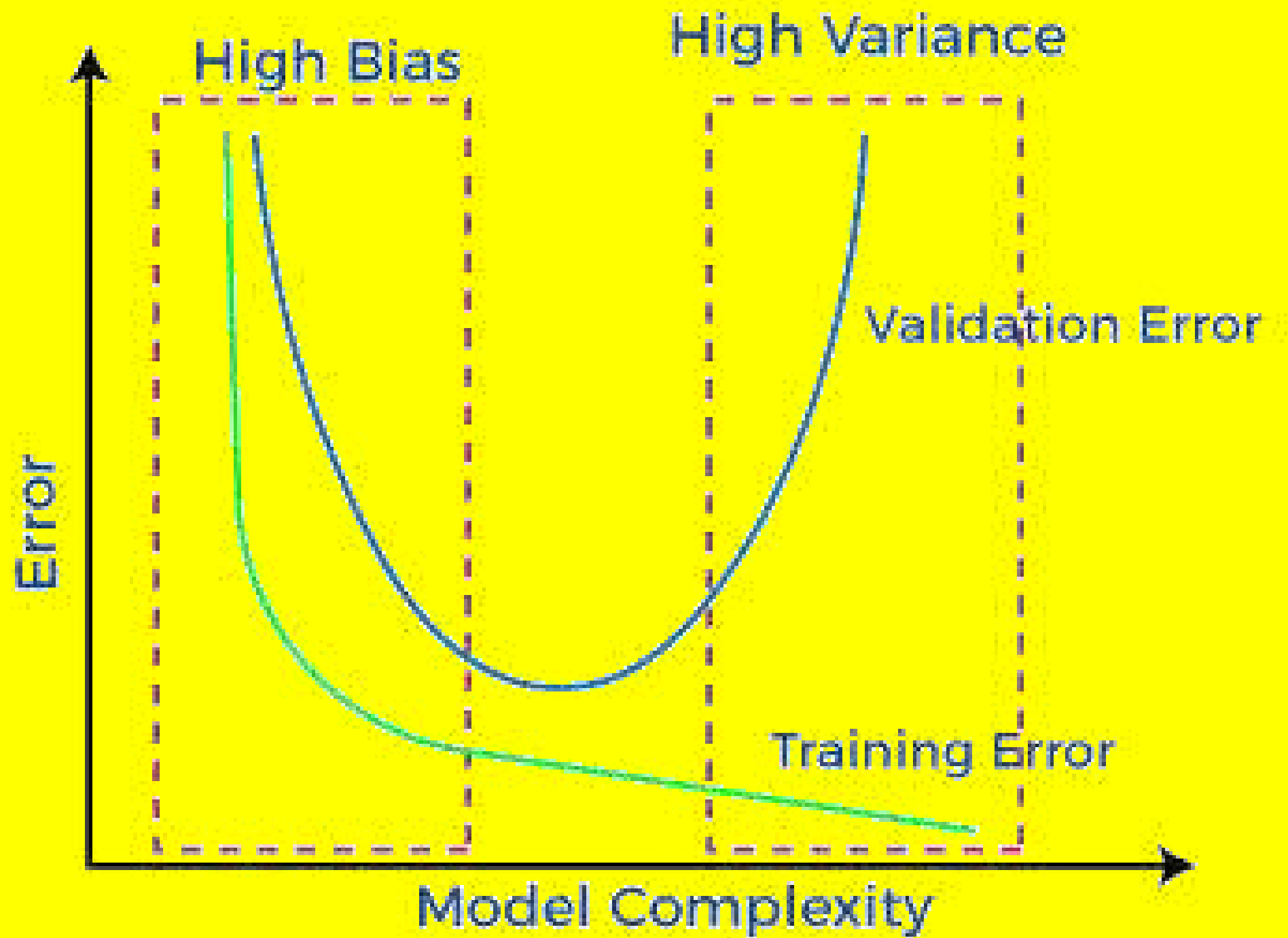
High Variance

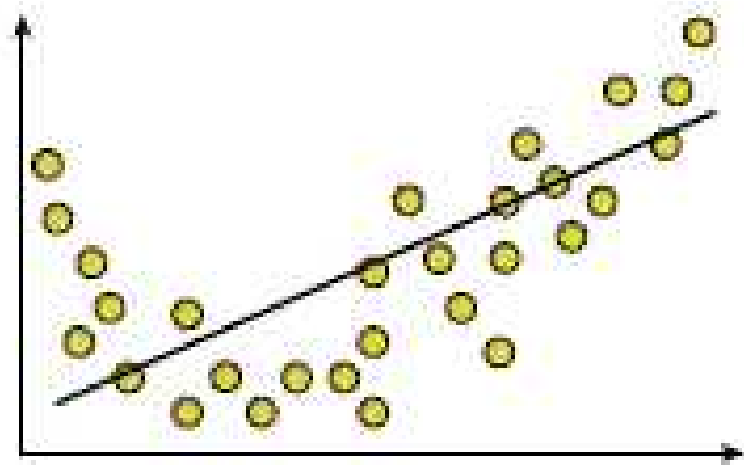
Low Bias



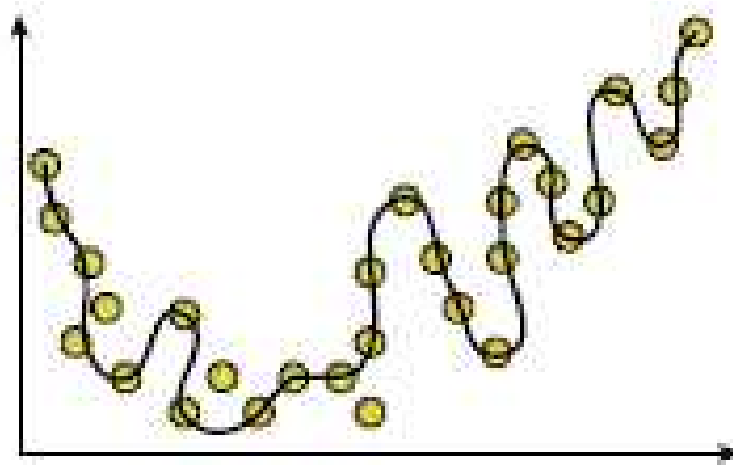
High Bias



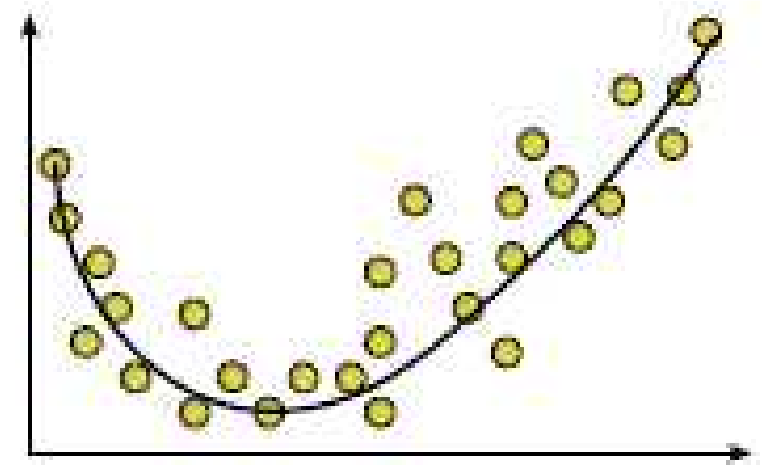




Underfitting  
(High Bias, Low Variance)



Overfitting  
(Low Bias, High Variance)

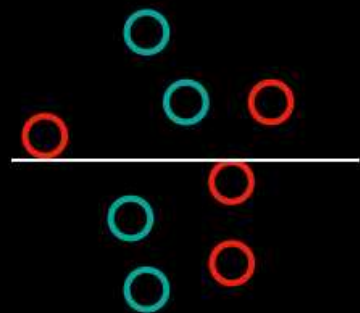


Good fit  
(Optimal Bias and Variance)

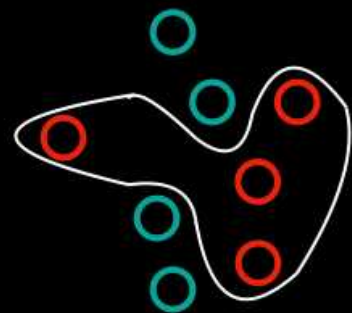
**Figure 1.12** Underfitting, overfitting, and optimal fit



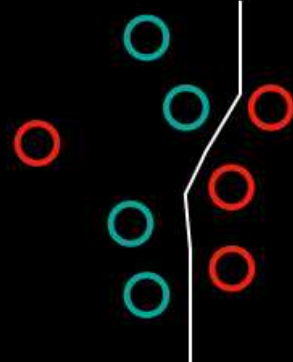
### Training Data: Classification



Underfitting



Overfitting



Good-Fit

High bias, low variance  
The model is too simple and not able to fit the training data.

Low bias, high variance  
The model is memorising the exact training data. Even outliers.

Low bias, 'low' variance  
The model is able to capture the trend, but ignores outliers.

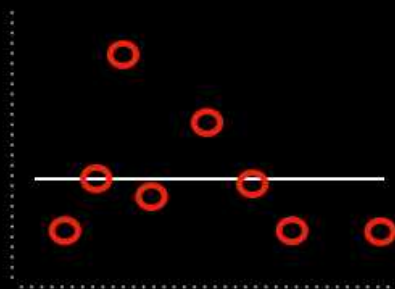
### Test Data

Classified as green

Classified as red

Thus, the model is able to classify new unseen data points.

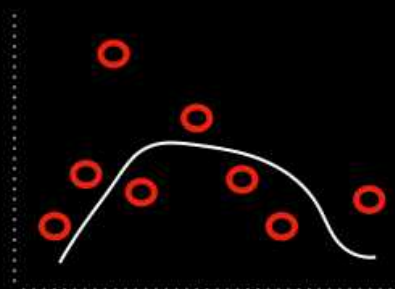
### Training Data: Regression



Underfitting

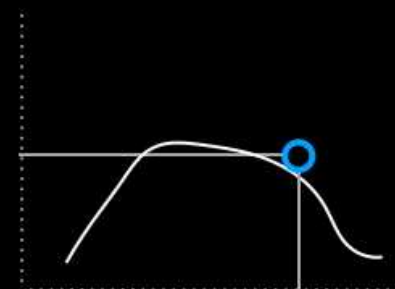


Overfitting



Good-Fit

### Test Data





# Mastering the Art of Hyperparameter Tuning for Machine Learning Models







# Hyperparameter Tuning

01

Model Selection

02

Review the  
Parameter

03

Selection Right  
Searching Method

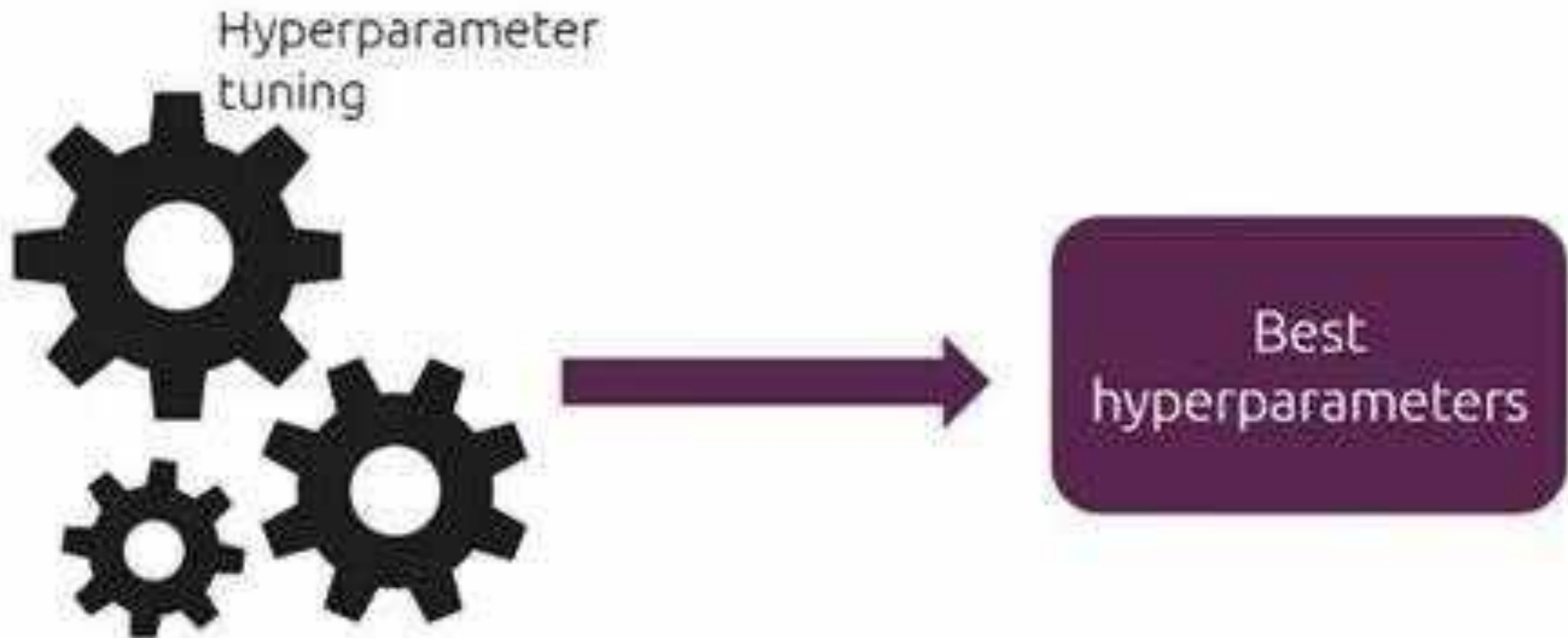
04

Applying cross-  
validation Approach

05

Access the score  
of the Model

# Tuning the Hyperparameters





# Popular Hyperparameter Tuning Techniques



[dataaspirant.com](https://dataaspirant.com)

1

Grid Search

2

Random Search

3

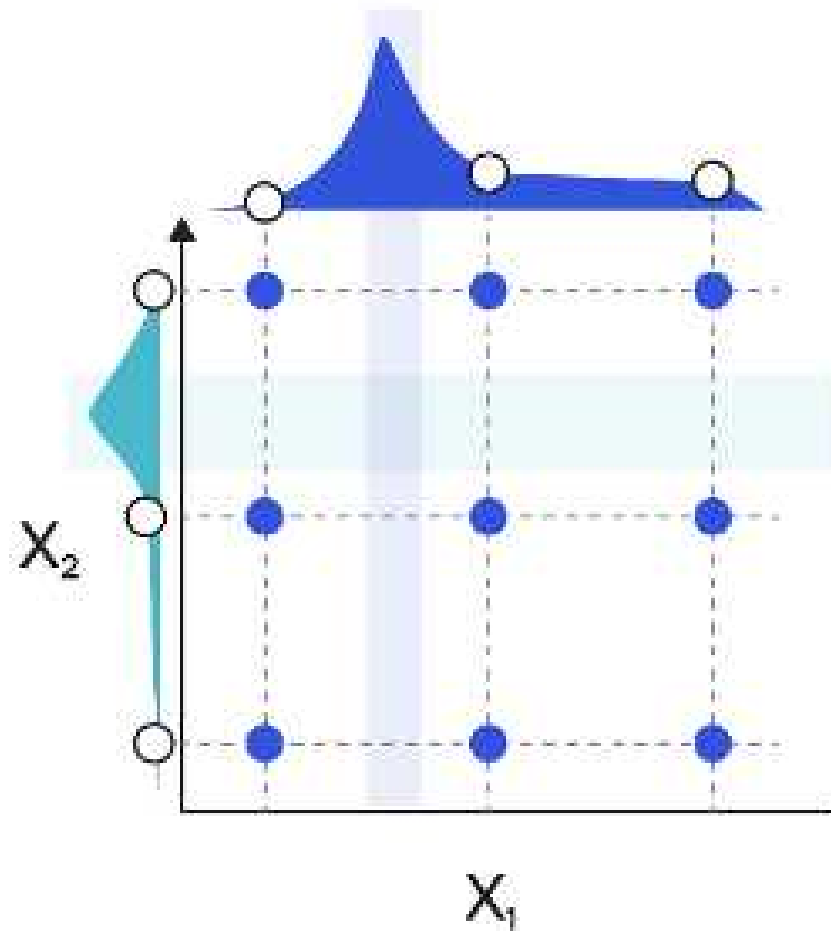
Bayesian Optimization

4

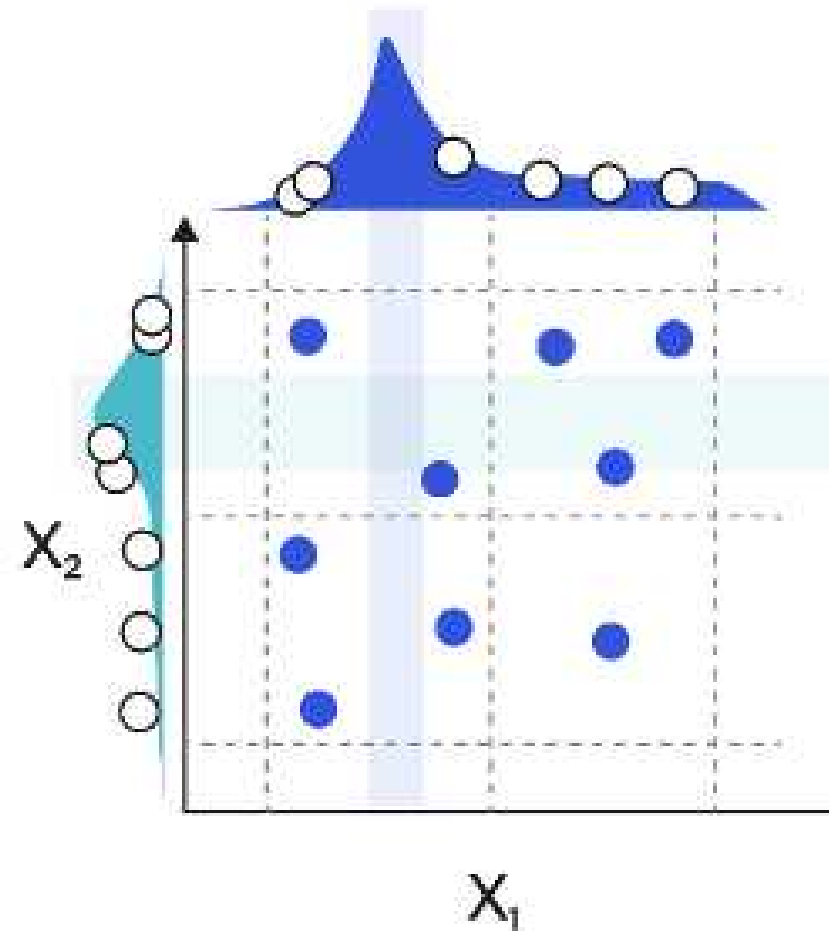
Tree-structured Parzen Estimators (TPE)

5

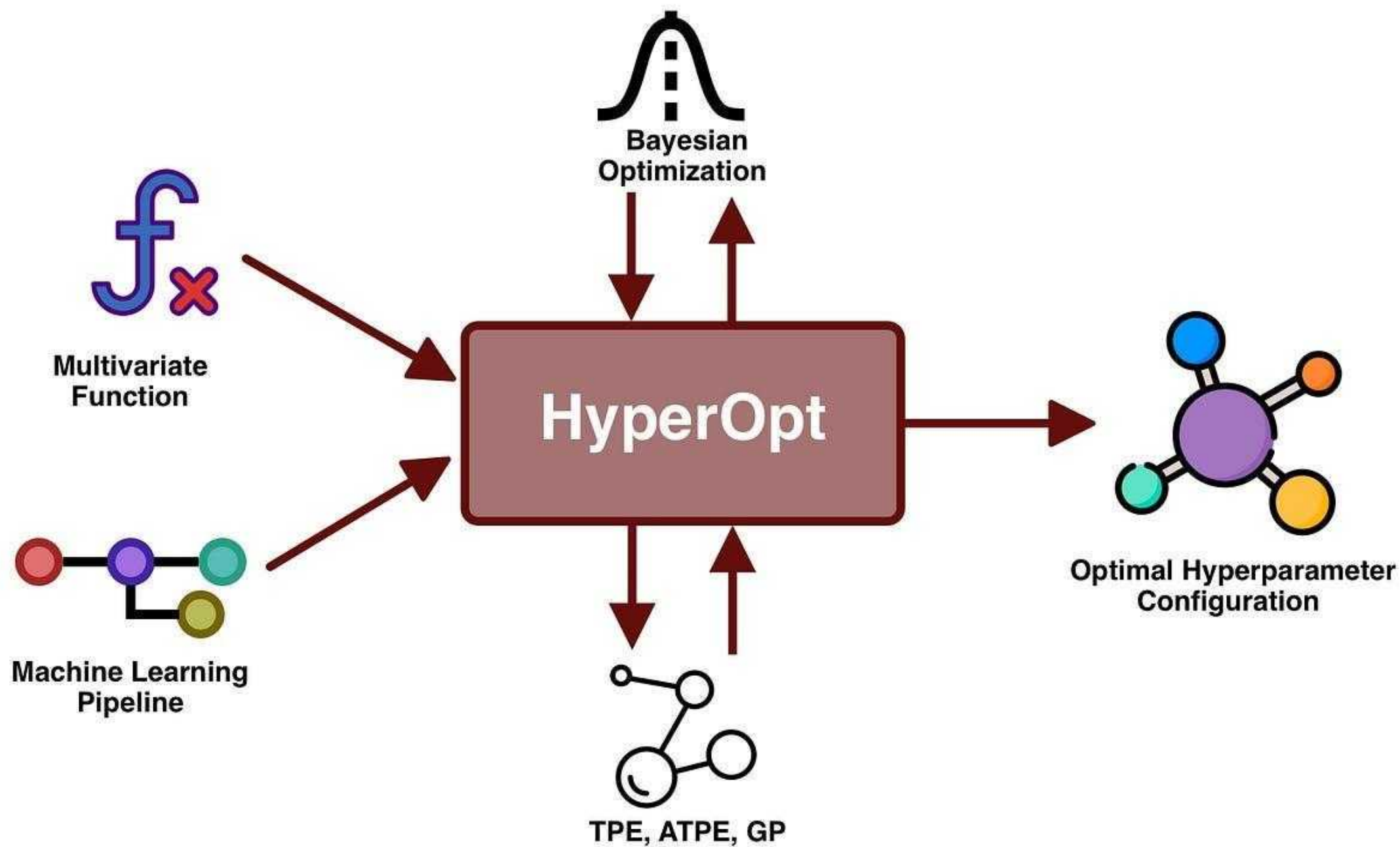
Gradient-based optimization



(a) Standard Grid Search

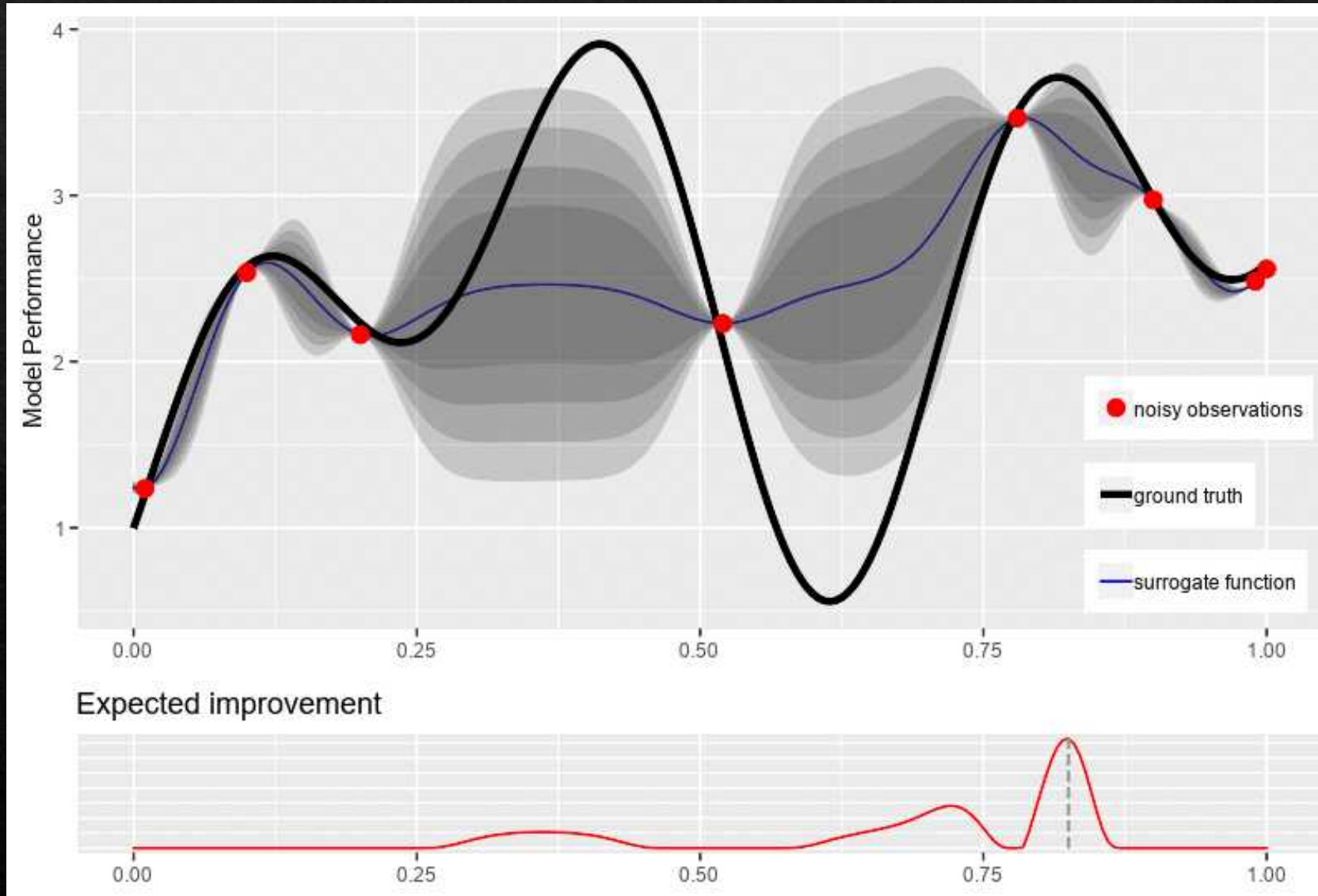


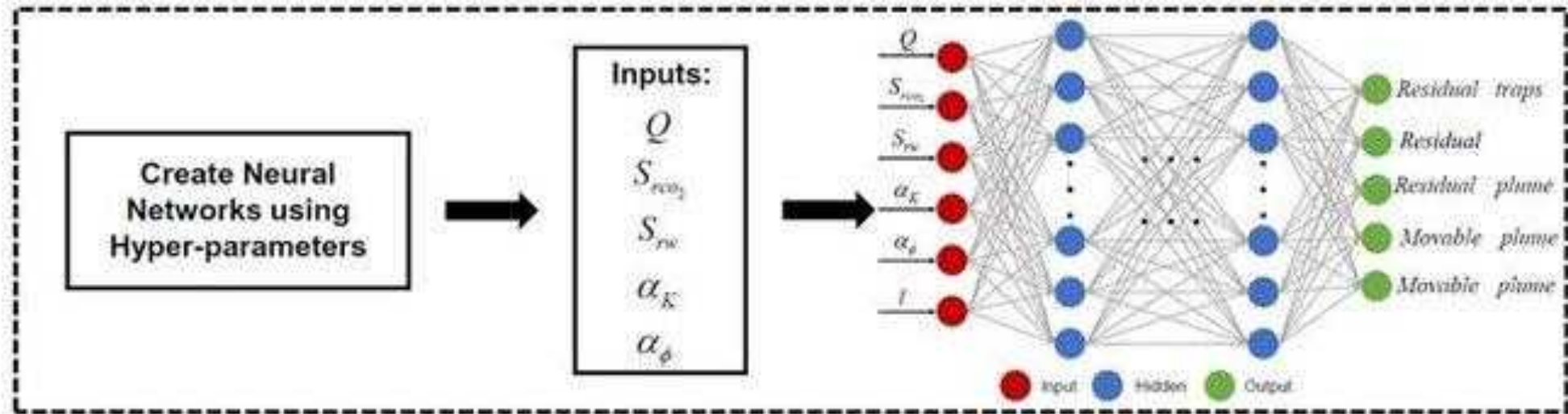
(b) Random Search





# Bayesian Optimization





## ANN Building & Training

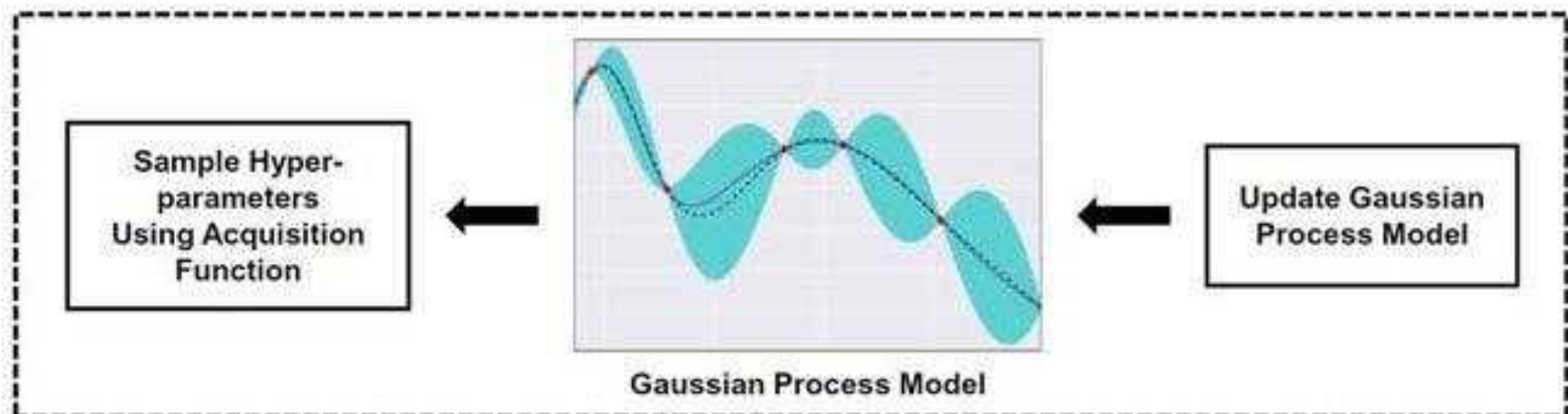
### Hyper-Parameters:

- Number of hidden layers
- Number of neurons
- Activation function
- Ratio of train to validation

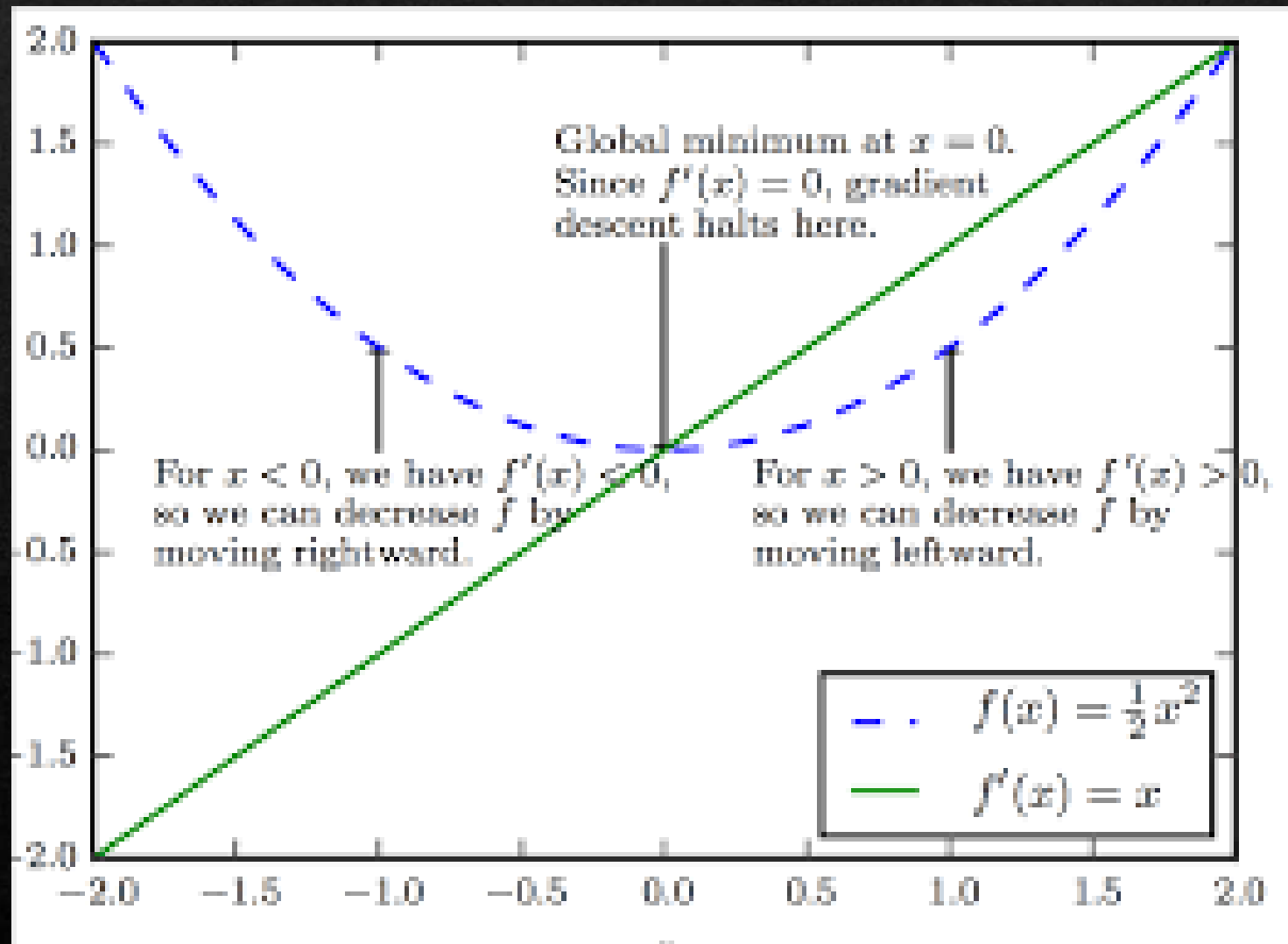
### Evaluation Metric:

- APE - Validation
- PPE - Validation

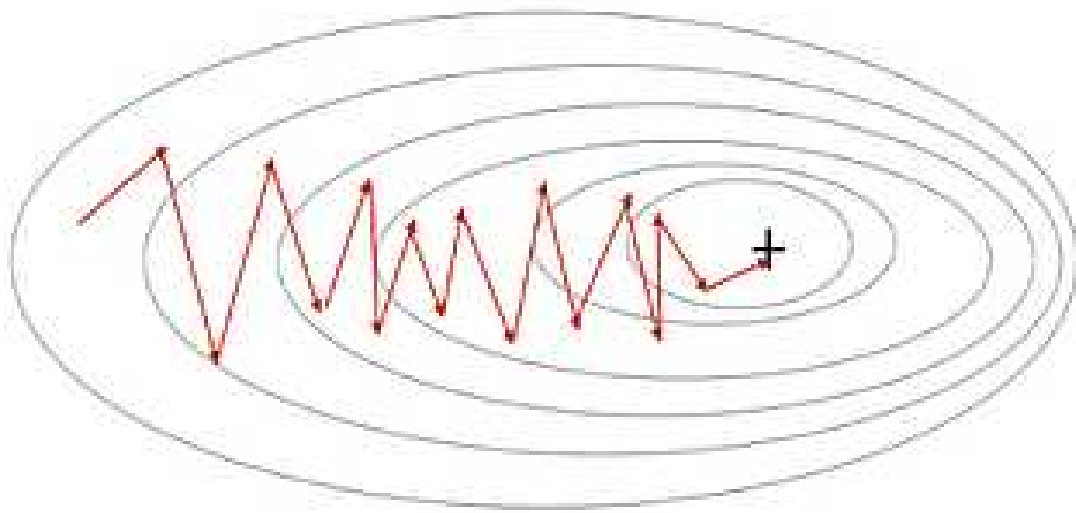
## Bayesian Optimization



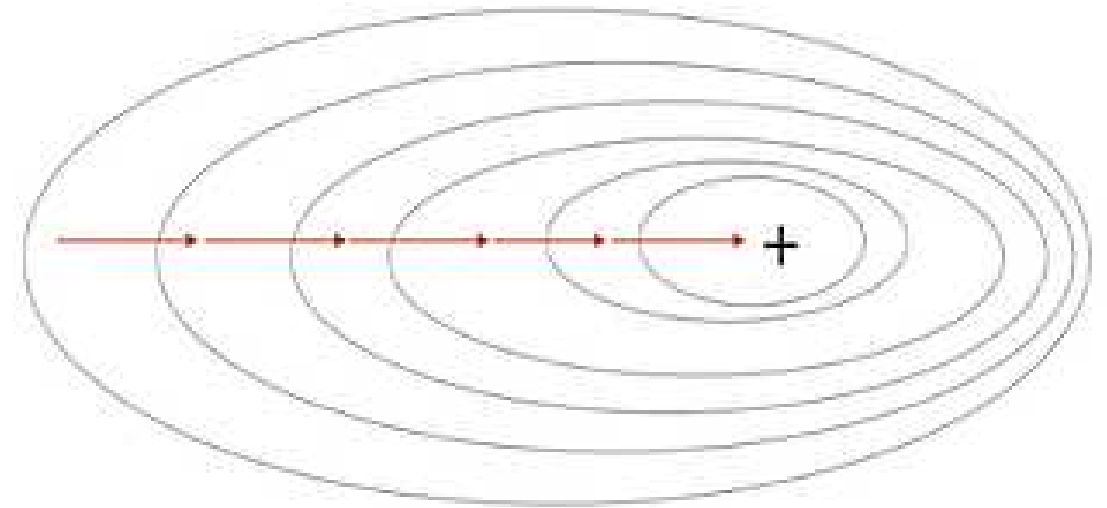
# Gradient-Based Optimization



## Stochastic Gradient Descent



## Gradient Descent



**Figure 1 : SGD vs GD**

"+" denotes a minimum of the cost. SGD leads to many oscillations to reach convergence. But each step is a lot faster to compute for SGD than for GD, as it uses only one training example (vs. the whole batch for GD).



Gradient  
Descent

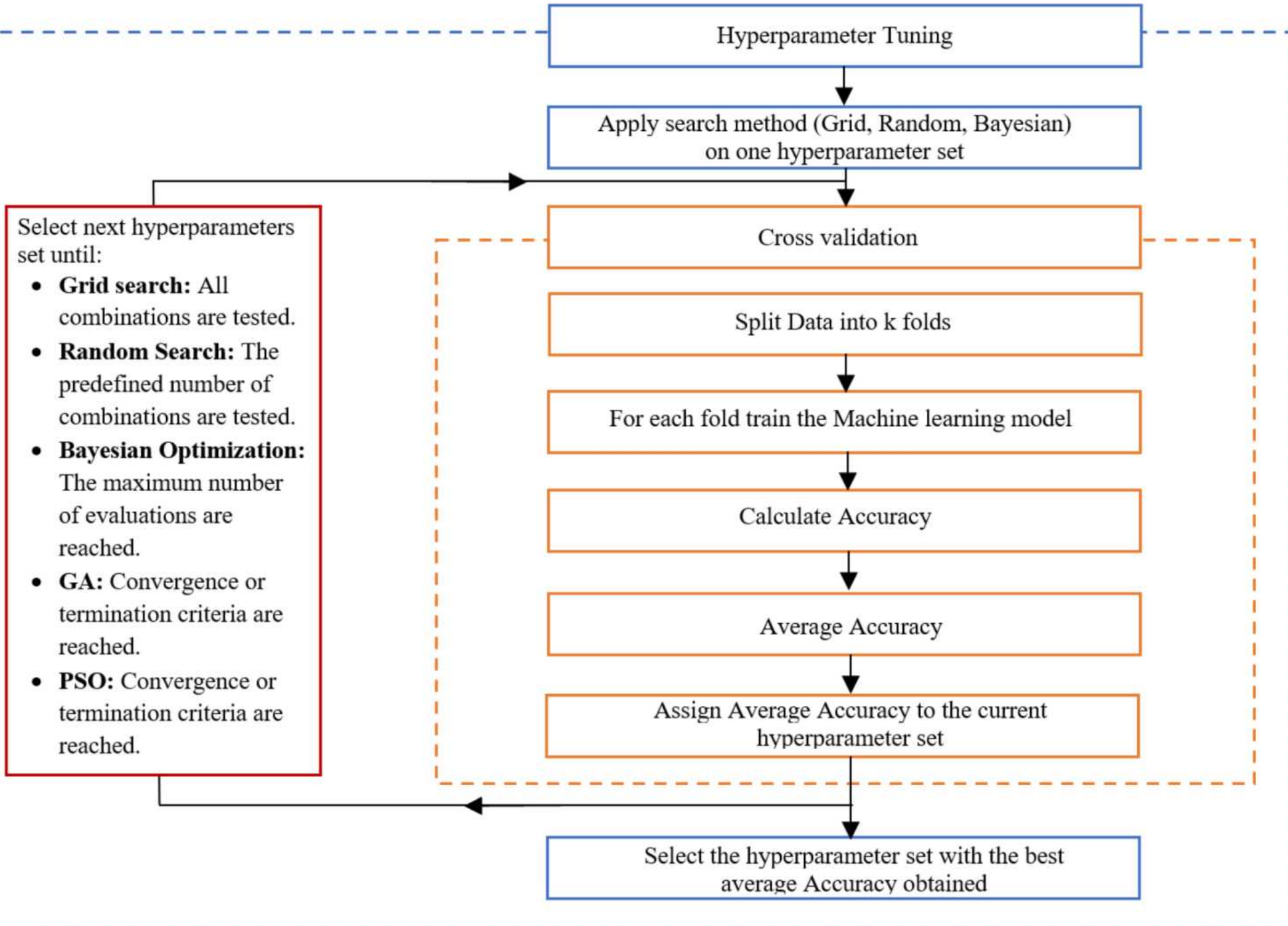
Follow the  
gradient

I'm  
oscillating...  
what do I do?

Learned  
Optimizer

Aha! I've seen  
this before...



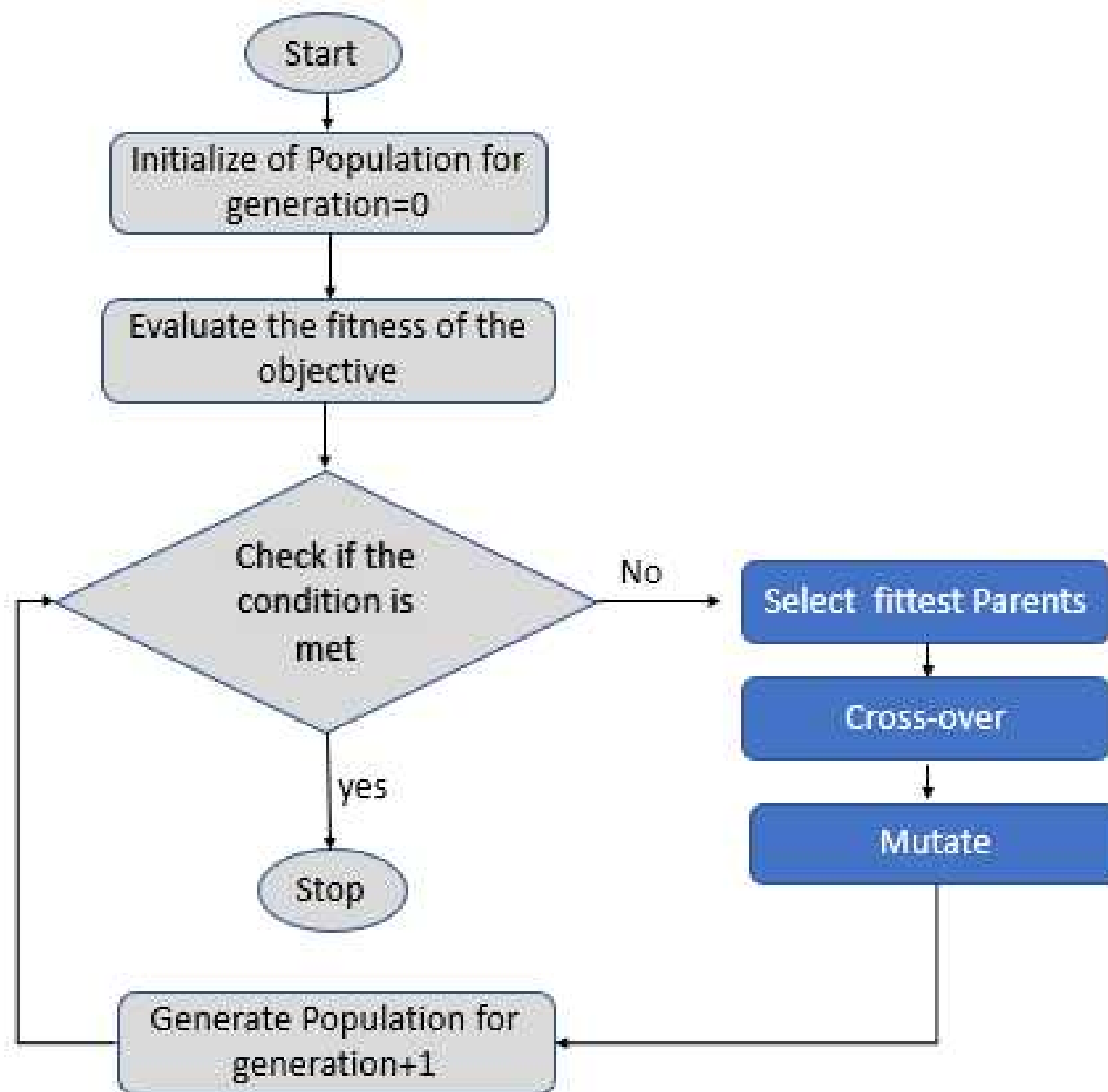




# What are Genetic Algorithms?

---

- Genetic Algorithm (GA) is a search-based optimization technique based on the principles of **Genetics and Darwin's Principle of Natural Selection**.
- It is frequently used to find optimal or near-optimal solutions to difficult problems which otherwise would take a lifetime to solve.
- GAs are a subset of a much larger branch of computation known as **Evolutionary Computation**.
- GAs were developed by John Holland (1975).

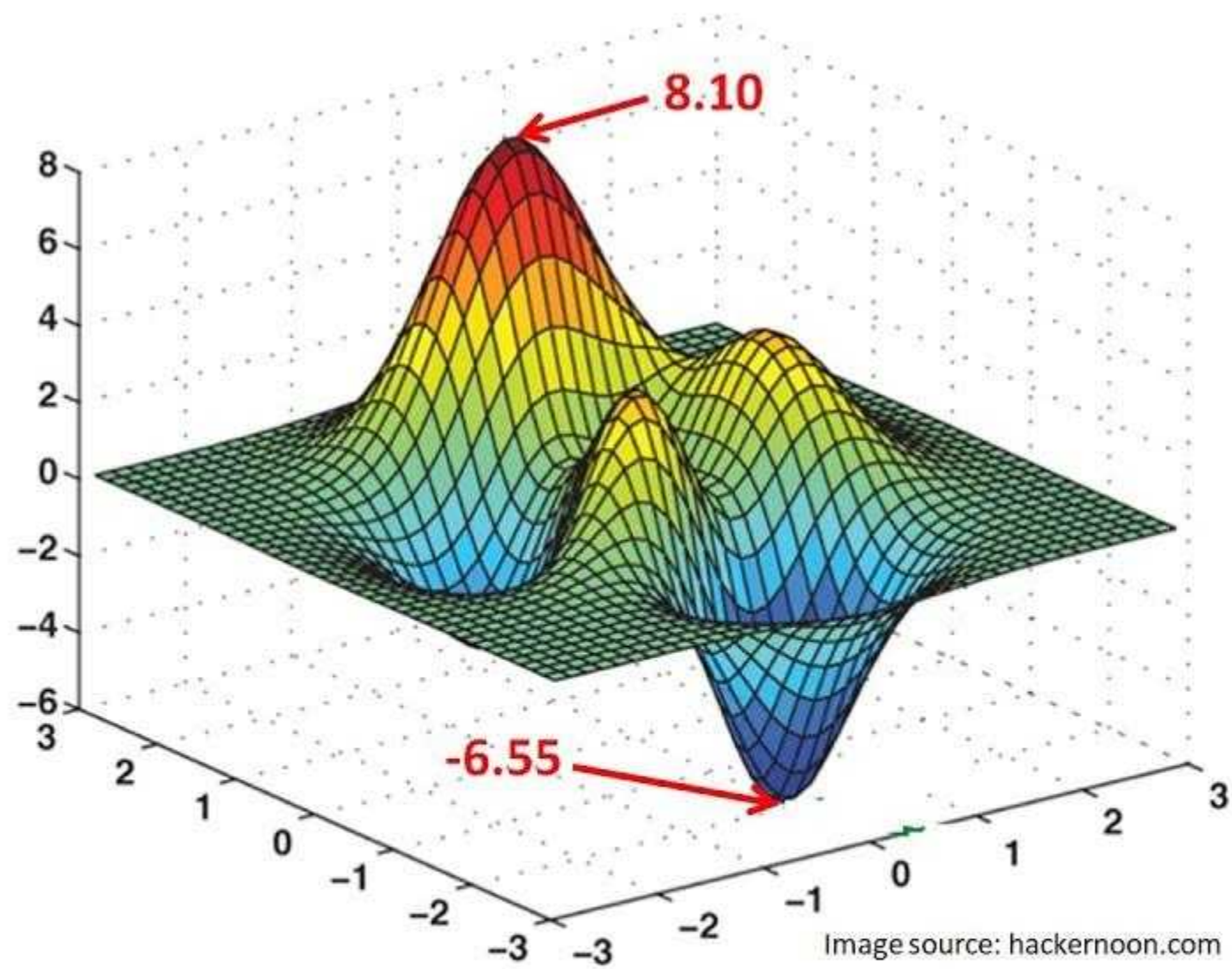


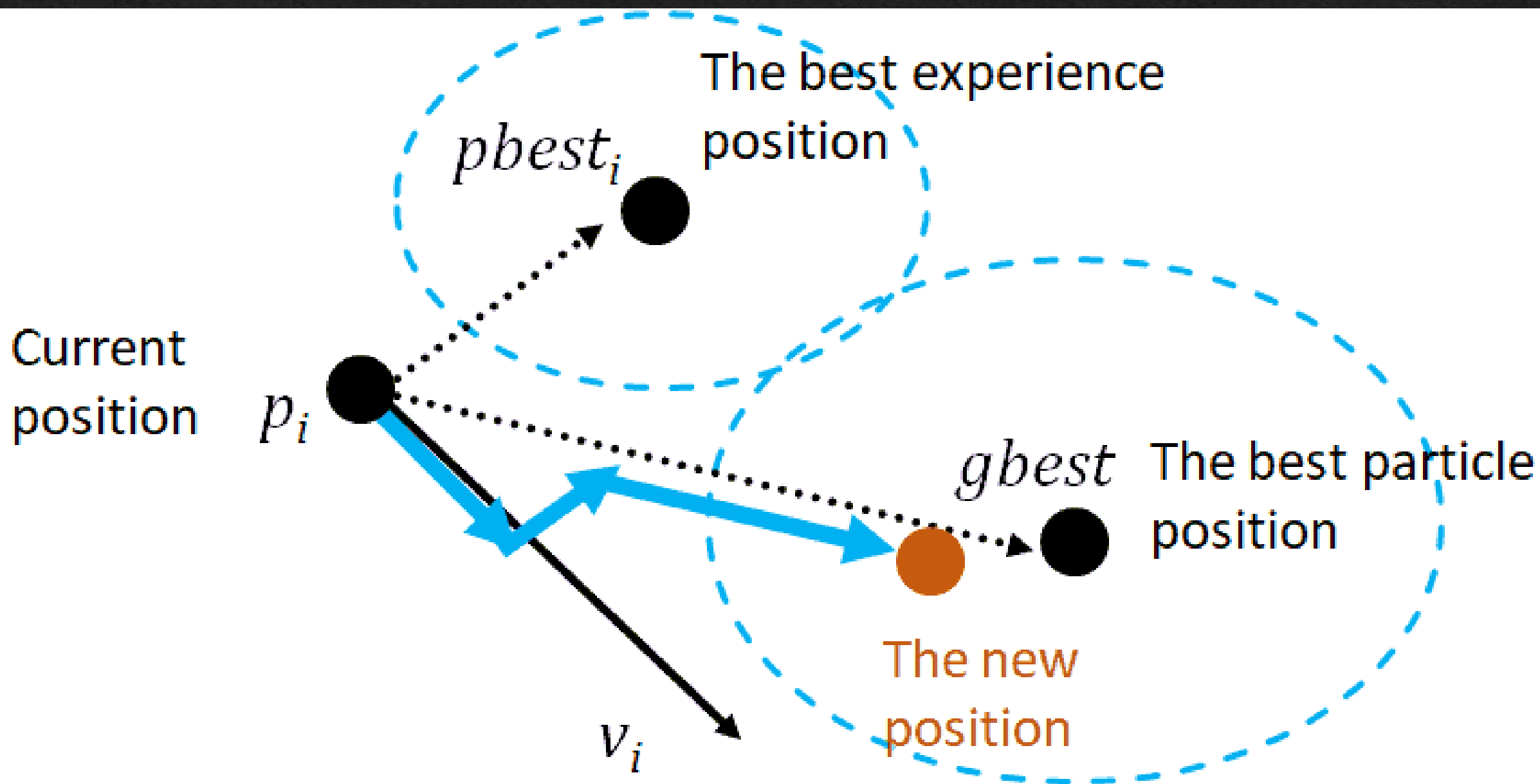


# Particle Swarm Optimization (PSO)

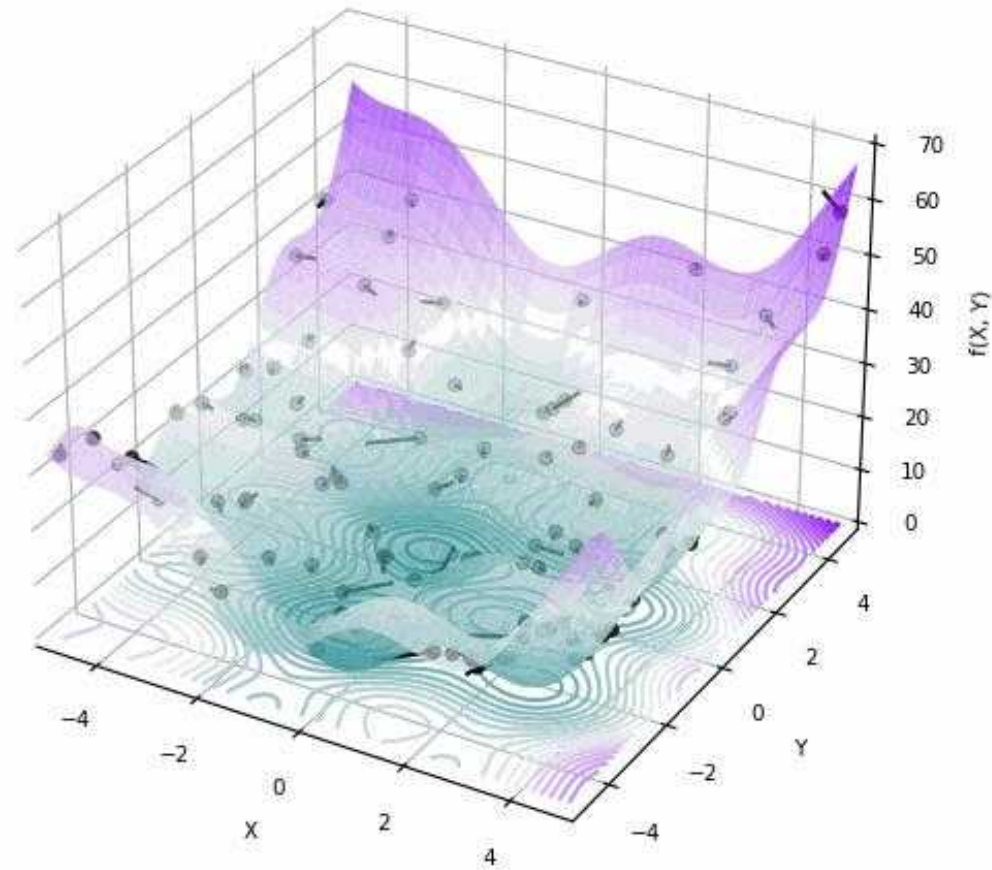
With

## Python Code & Simple Examples





[1/100]  $w:0.800 - c_1:3.500 - c_2:0.500$



First run of Particle Swarm Optimization



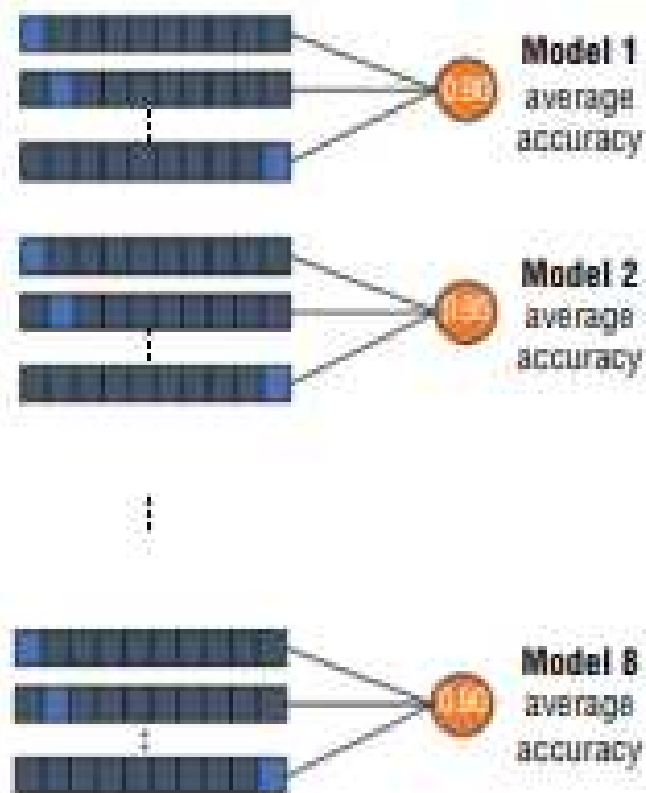
# How to improve the performance of a machine learning (ML) model



1. Create grid of parameter combinations to be considered.

| Model | Alpha | Beta | Gamma |
|-------|-------|------|-------|
| 1     | 1     | A    | TRUE  |
| 2     | 1     | A    | FALSE |
| 3     | 1     | B    | TRUE  |
| 4     | 1     | B    | FALSE |
| 5     | 2     | A    | TRUE  |
| 6     | 2     | A    | FALSE |
| 7     | 2     | B    | TRUE  |
| 8     | 2     | B    | FALSE |

2. Use k-fold cross-validation to evaluate the performance of each model.



3. Select the model with the best-performing parameter combination.

| Model | Alpha | Beta | Gamma | Accuracy |
|-------|-------|------|-------|----------|
| 1     | 1     | A    | TRUE  | 0.90     |
| 2     | 1     | A    | FALSE | 0.95     |
| 3     | 1     | B    | TRUE  | 0.92     |
| 4     | 1     | B    | FALSE | 0.91     |
| 5     | 2     | A    | TRUE  | 0.91     |
| 6     | 2     | A    | FALSE | 0.90     |
| 7     | 2     | B    | TRUE  | 0.89     |
| 8     | 2     | B    | FALSE | 0.80     |

**Figure 10.1** The grid search process showing eight models with different parameter combinations, which are each evaluated using k-fold cross-validation. The model with the best-performing parameter combination is chosen (model 2).



# Figures of Merit: Classification Tasks

- ◆ Kappa Index
- ◆ Sensitivity or Recall
- ◆ Specificity
- ◆ Accuracy
- ◆ ROC curve
- ◆ AUC
- ◆  $F_I$ -score

# Cohen Kappa Statistic

Measure The Performance of Classification Models

Kappa  
Statistic

$$k = \frac{2 * (TP * TN - FN * FP)}{(TP + FP) * (FP + TN) + (TP + FN) * (FN + TN)}$$

Assess the level of agreement between an actual and predicted

| Predicted<br>(rater 2) \ Actual<br>(rater 1) | YES     | NO      |    |
|----------------------------------------------|---------|---------|----|
|                                              |         |         |    |
| YES                                          | 45 (TP) | 15 (FN) | 60 |
| NO                                           | 25 (FP) | 15 (TN) | 40 |
|                                              | 70      | 30      |    |

## Kappa Score Interpretation

| Kappa     | Agreement                  |
|-----------|----------------------------|
| <0        | Less than chance agreement |
| 0.01-0.20 | Slight agreement           |
| 0.21-0.40 | Fair agreement             |
| 0.41-0.60 | Moderate agreement         |
| 0.61-0.80 | Substantial agreement      |
| 0.81-0.99 | Almost perfect agreement   |



$$k = \frac{2 * (45 * 15 - 15 * 25)}{(45 + 25) * (25 + 15) + (45 + 15) * (15 + 15)} = 0.13 \text{ (13\%)}$$



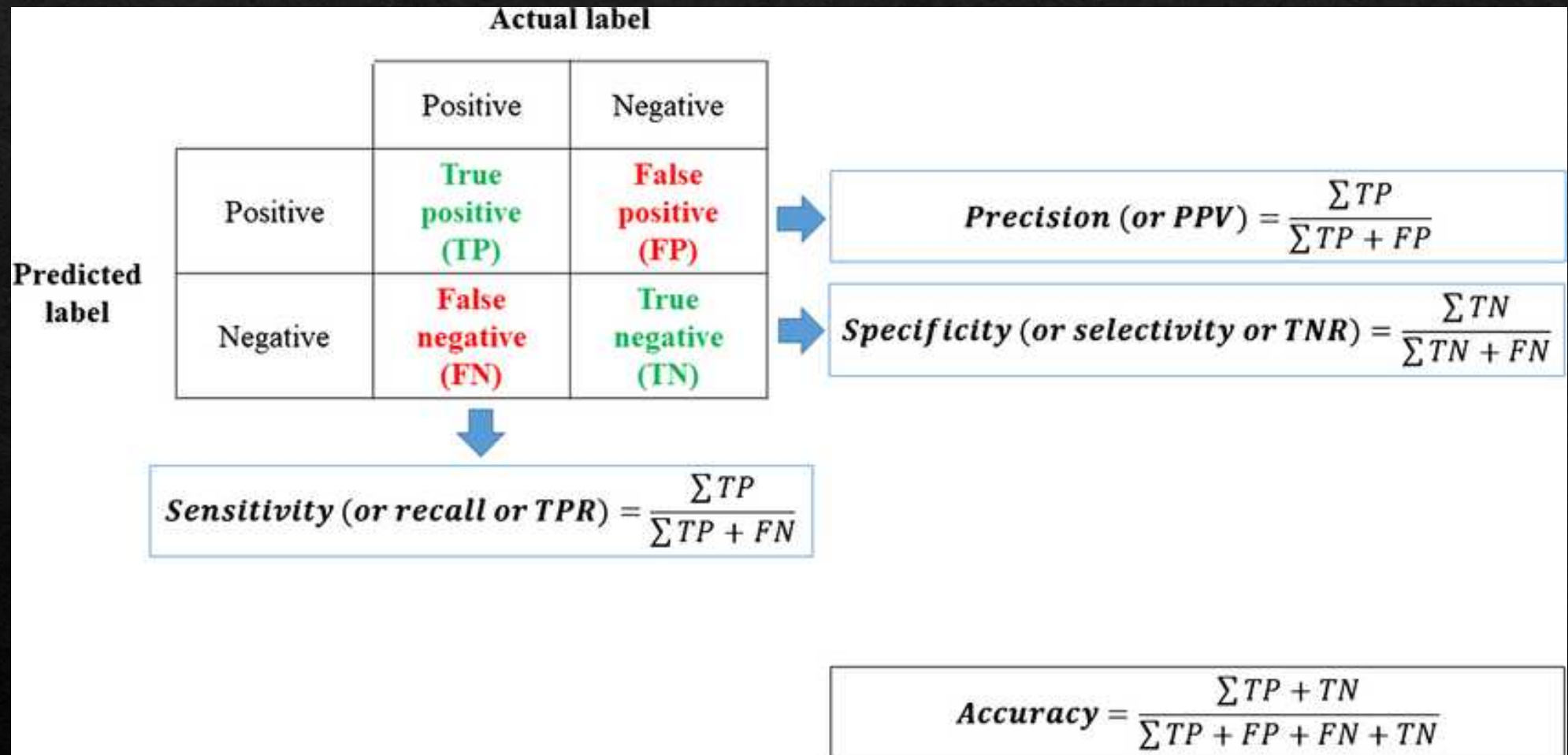
**Figure 9.8** A sample confusion matrix showing actual versus predicted values

|        |     | Predicted                                                                                              |                                                                                                       |
|--------|-----|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
|        |     | Yes                                                                                                    | No                                                                                                    |
| Actual | Yes | <br>True Positive   | <br>False Negative |
|        | No  | <br>False Positive | <br>True Negative |

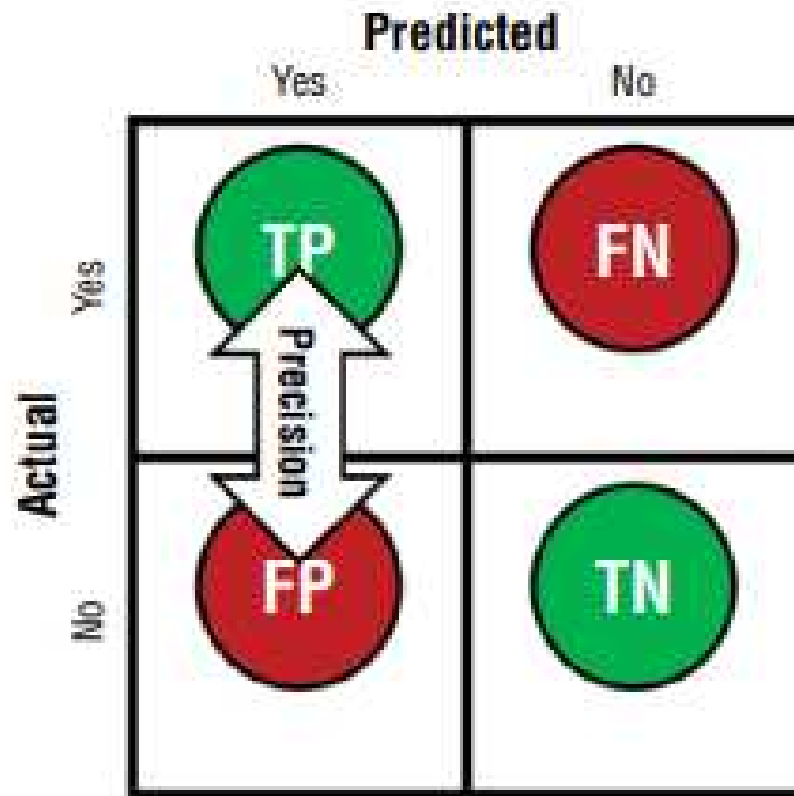
**Figure 9.9** Spam filter confusion matrix

|        |      | Predicted |     |
|--------|------|-----------|-----|
|        |      | Spam      | Ham |
| Actual | Spam | 135       | 80  |
|        | Ham  | 2         | 203 |

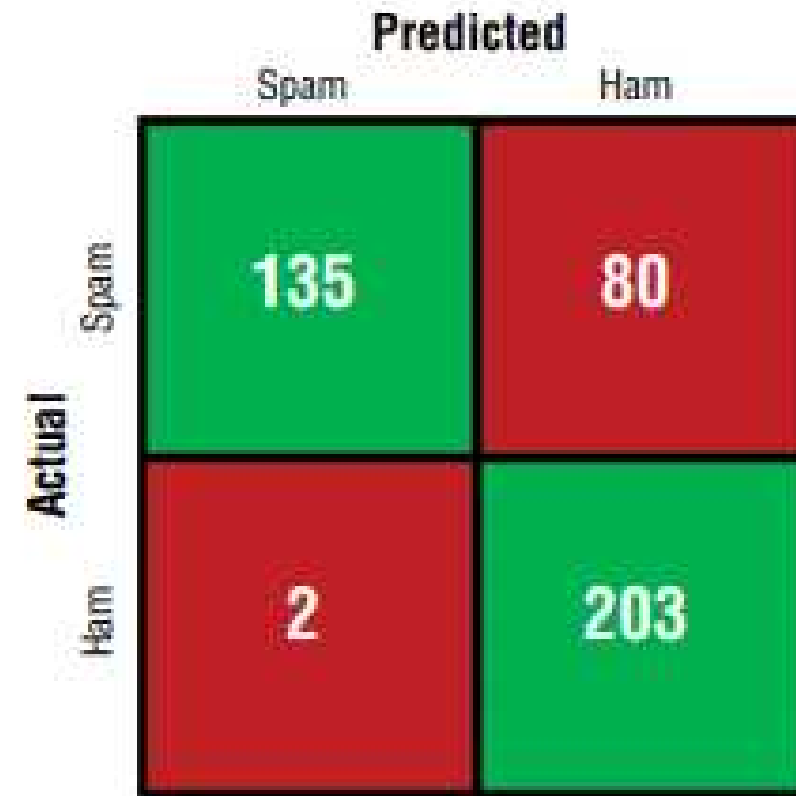
# Figures of Merit





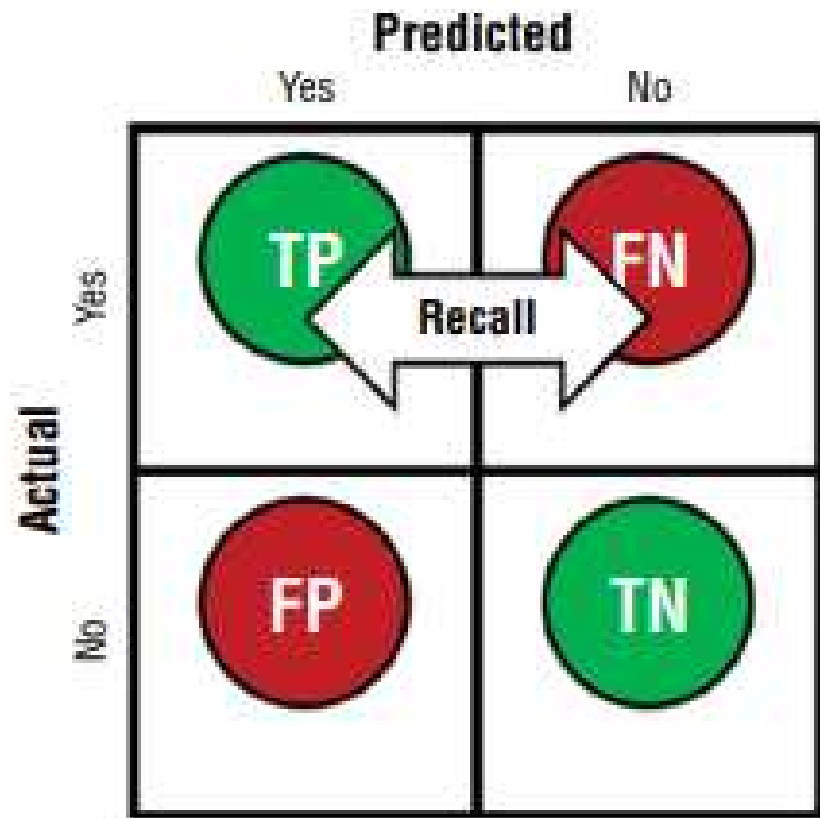


(a)

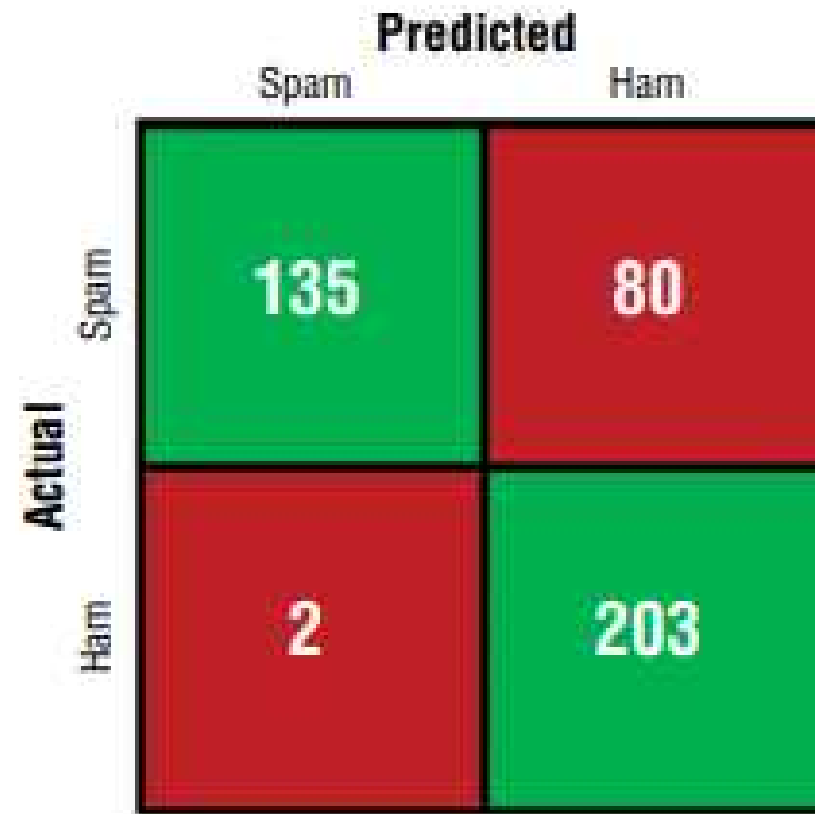


(b)

**Figure 9.10** (a) Precision as a measure of model performance based on (b) the spam filter confusion matrix

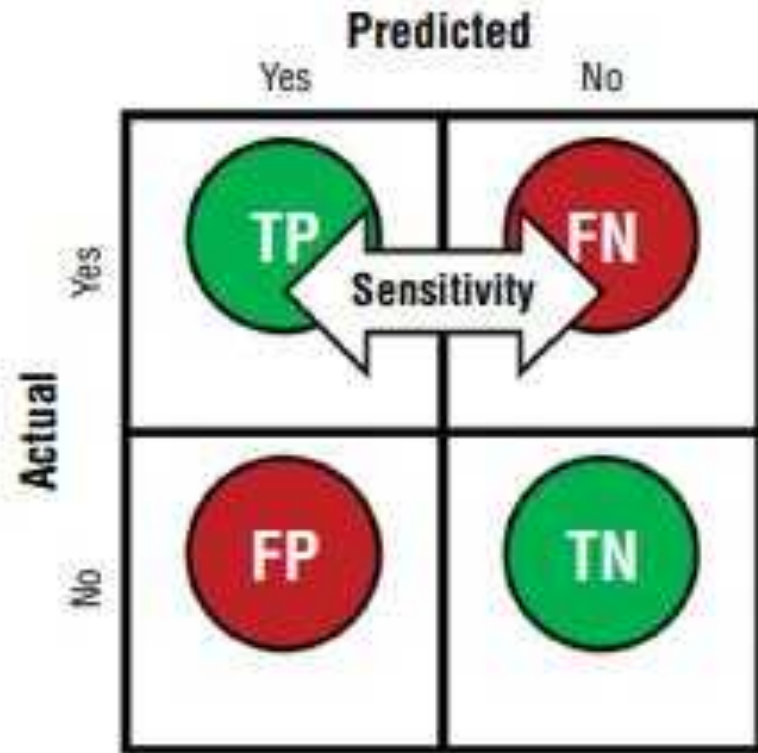


(a)

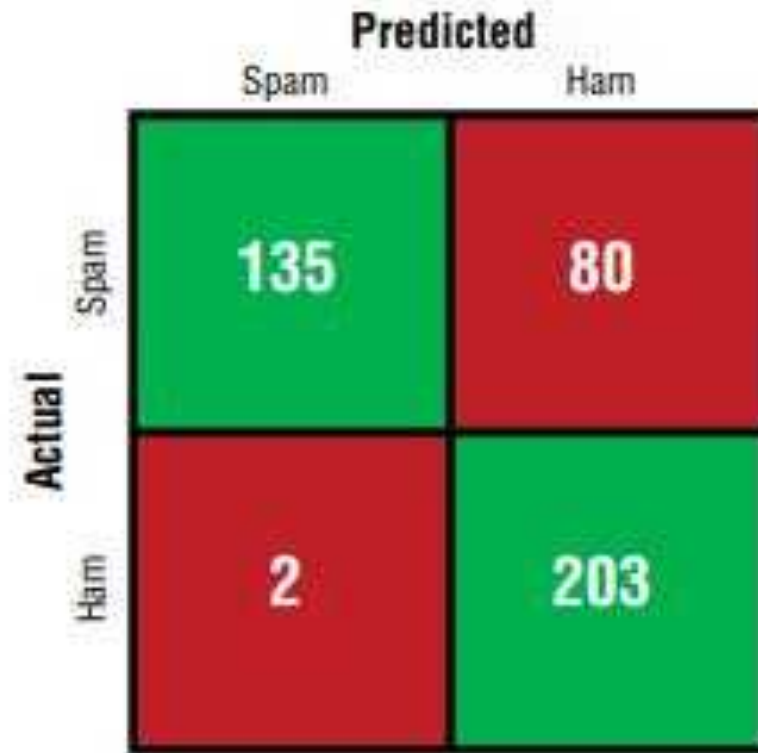


(b)

**Figure 9.11** (a) Recall as a measure of model performance based on (b) the spam filter confusion matrix



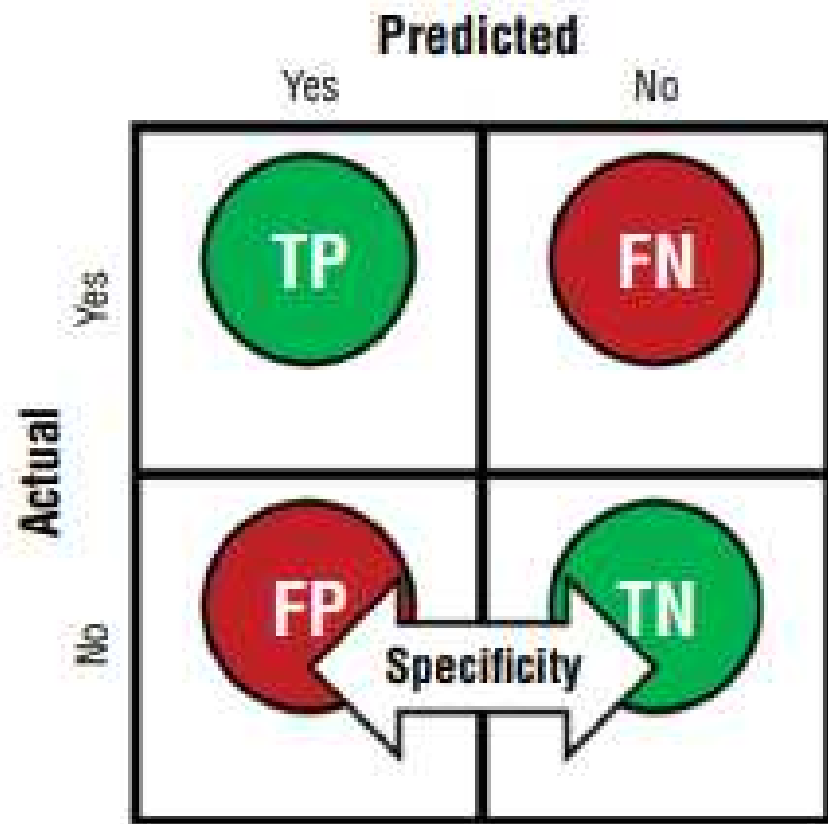
(a)



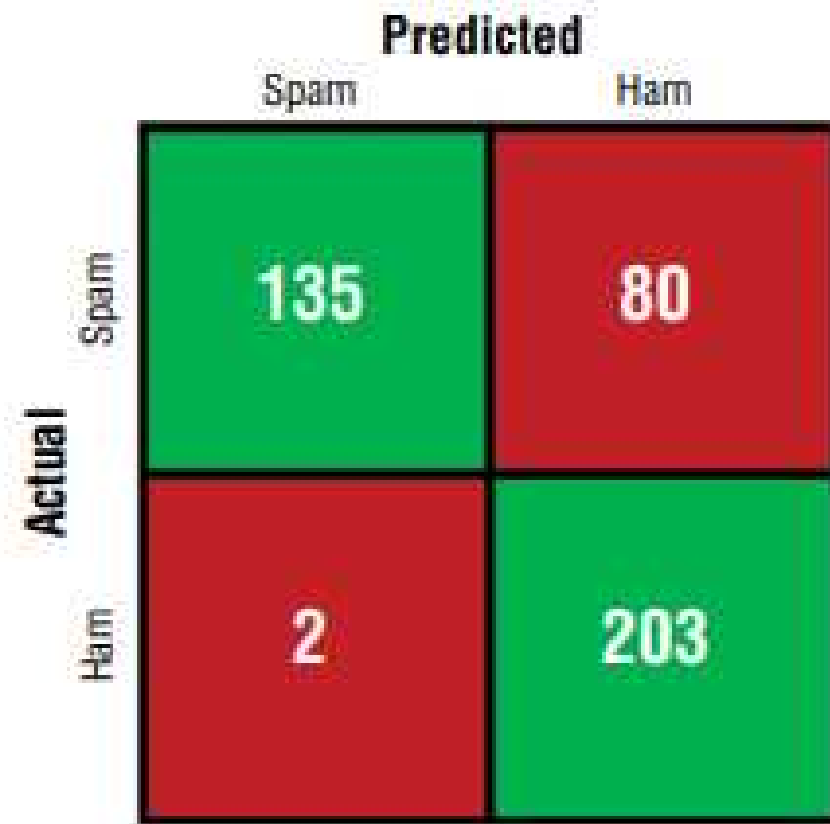
(b)

**Figure 9.12** (a) Sensitivity as a measure of model performance based on (b) the spam filter confusion matrix



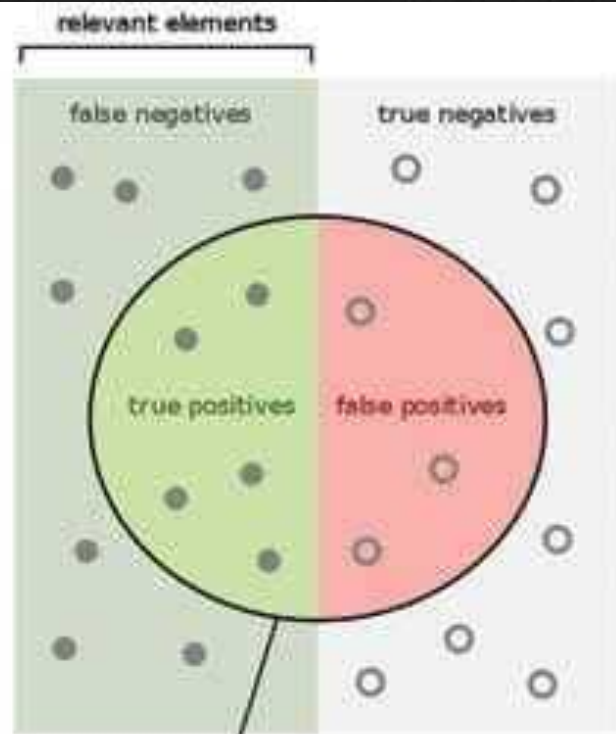


(a)



(b)

**Figure 9.13** (a) Specificity as a measure of model performance based on (b) the spam filter confusion matrix



selected elements

How many relevant items are selected?  
e.g. How many sick people are correctly identified as having the condition.

Sensitivity =



How many negative selected elements are truly negative?  
e.g. How many healthy people are identified as not having the condition.

Specificity =



# Sensitivity and Specificity

$$\text{Sensitivity} = \frac{\text{Number of true positives}}{(\text{Number of true positives} + \text{Number of false negatives})}$$

$$= \frac{\text{Number of true positives}}{\text{Total number of individuals with the illness}}$$

$$\text{Specificity} = \frac{\text{Number of true negatives}}{(\text{Number of true negatives} + \text{number of false positives})}$$

$$= \frac{\text{Number of true negatives}}{\text{Total number of individuals without the illness}}$$

# Receiver Operating Characteristics (ROC)

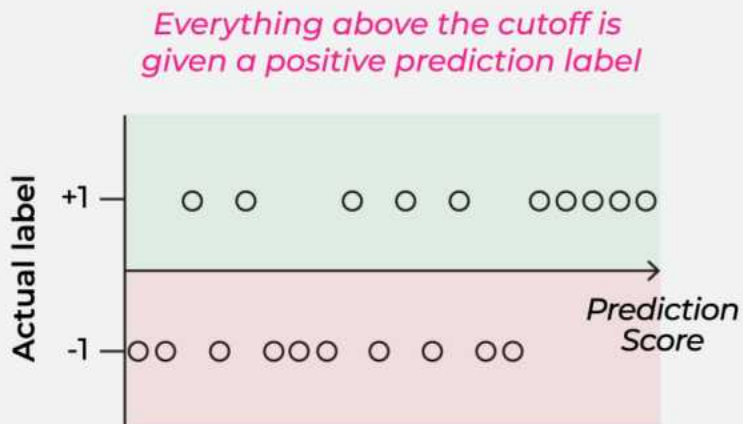
- ◊ ROC curves show the classifier's performance by plotting the true positive rate (TPR) and false positive rate (FPR).
- ◊ The area under the ROC curve (AUC) measures the performance of the machine learning algorithms.



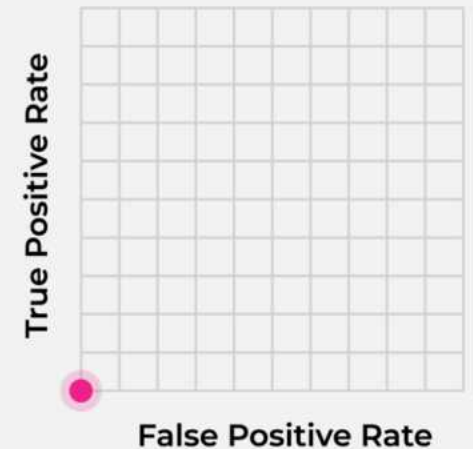
# Receiver Operating Characteristics, ROC

True Positive Rate: 0/10

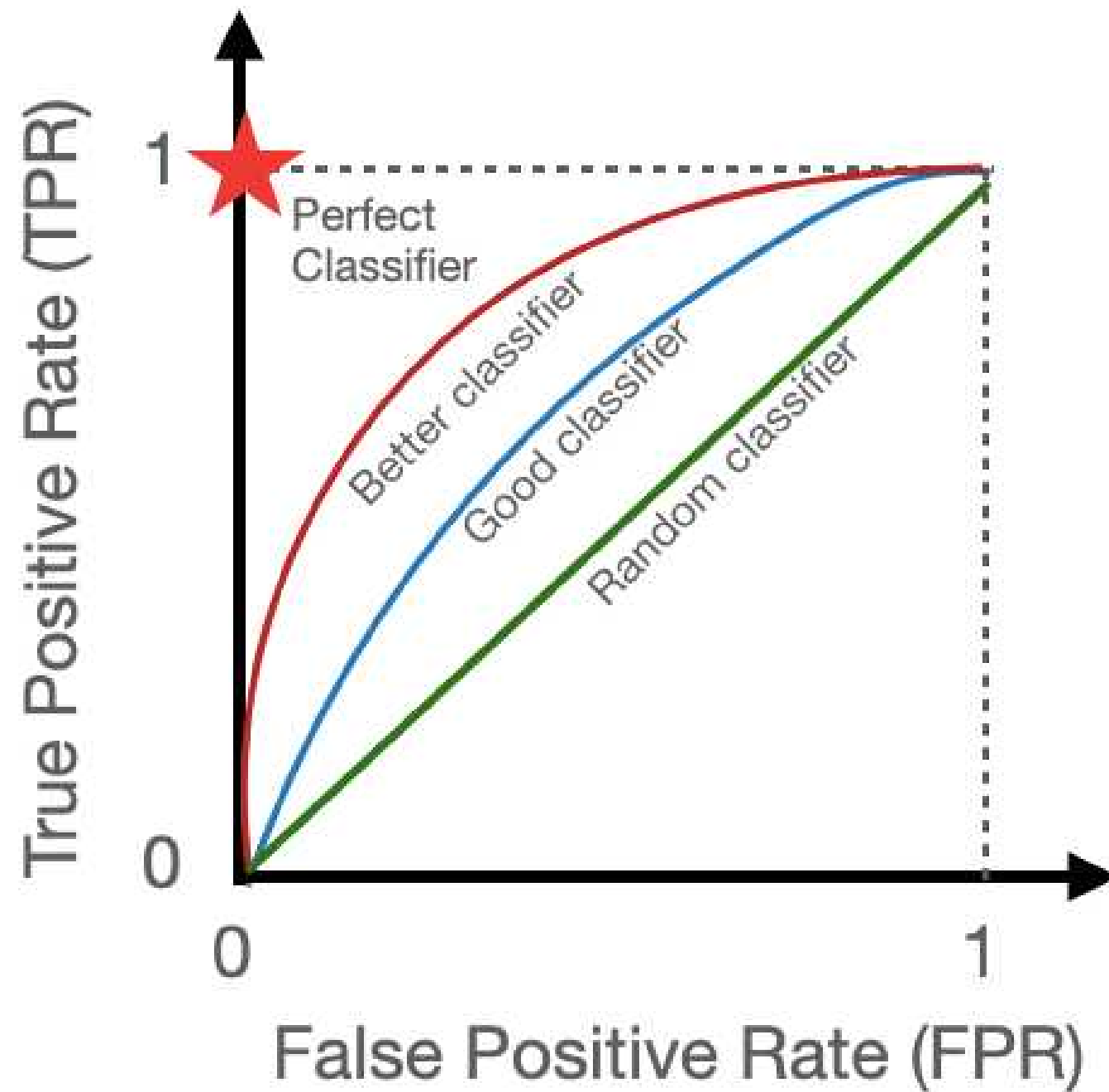
False Positive Rate: 0/10

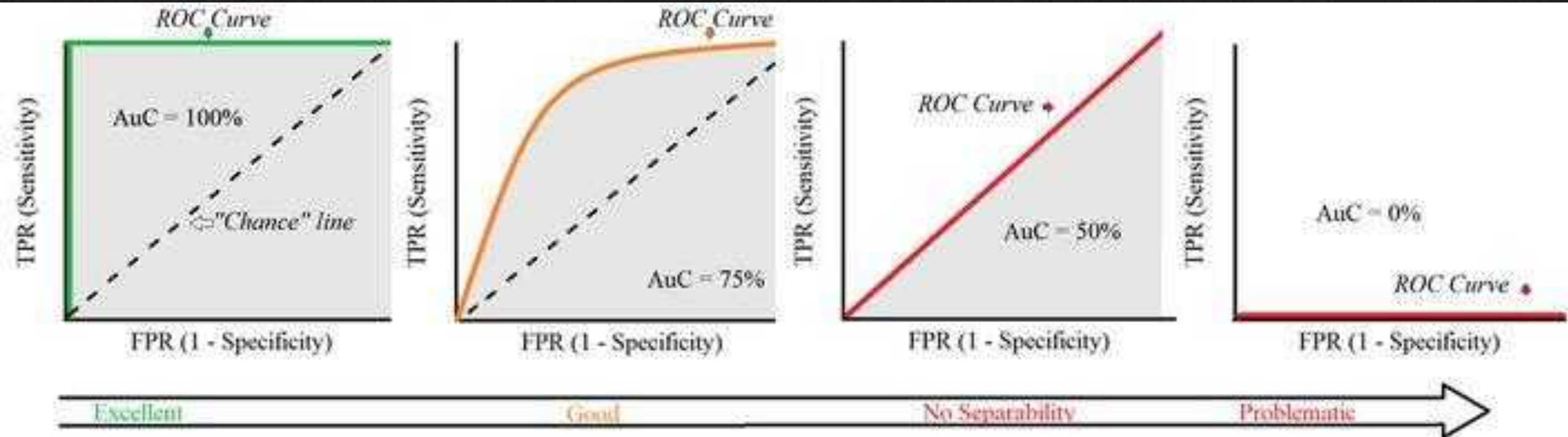


|              | Prediction label |         |
|--------------|------------------|---------|
|              | POS (+)          | NEG (-) |
| Actual label |                  |         |
| POS (+)      | 0                | 10      |
| NEG (-)      | 0                | 10      |

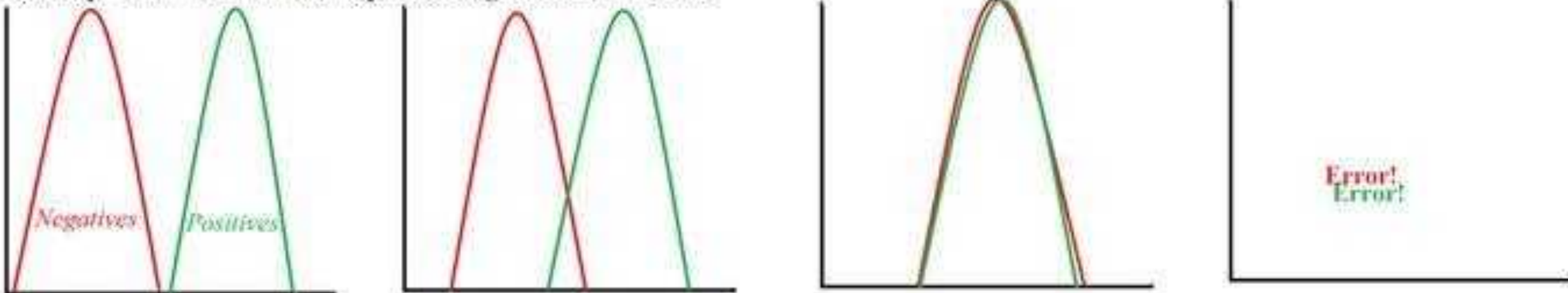


Animation of the AUC (area under the ROC) calculation process





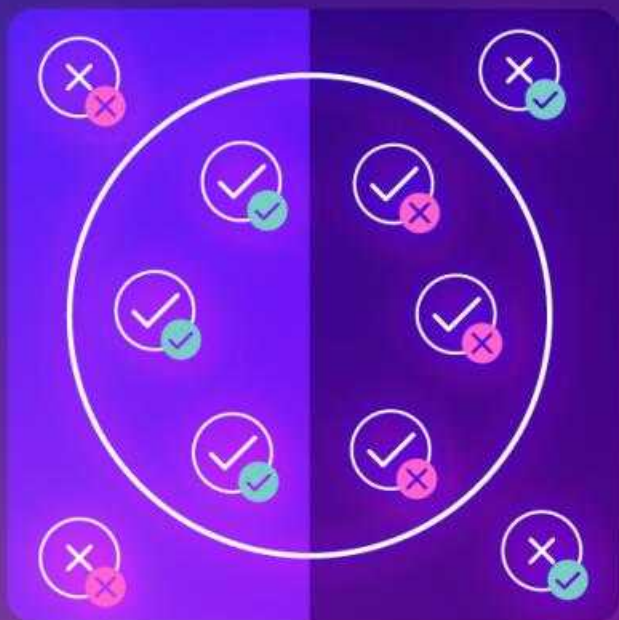
**Overlap = How well the model separates Negatives and Positives**





# Calculating a machine learning model's F1 score

**F1 SCORE**



$= 2 \times$

**Precision**

$\times$

**Recall**

**Precision**

$+$

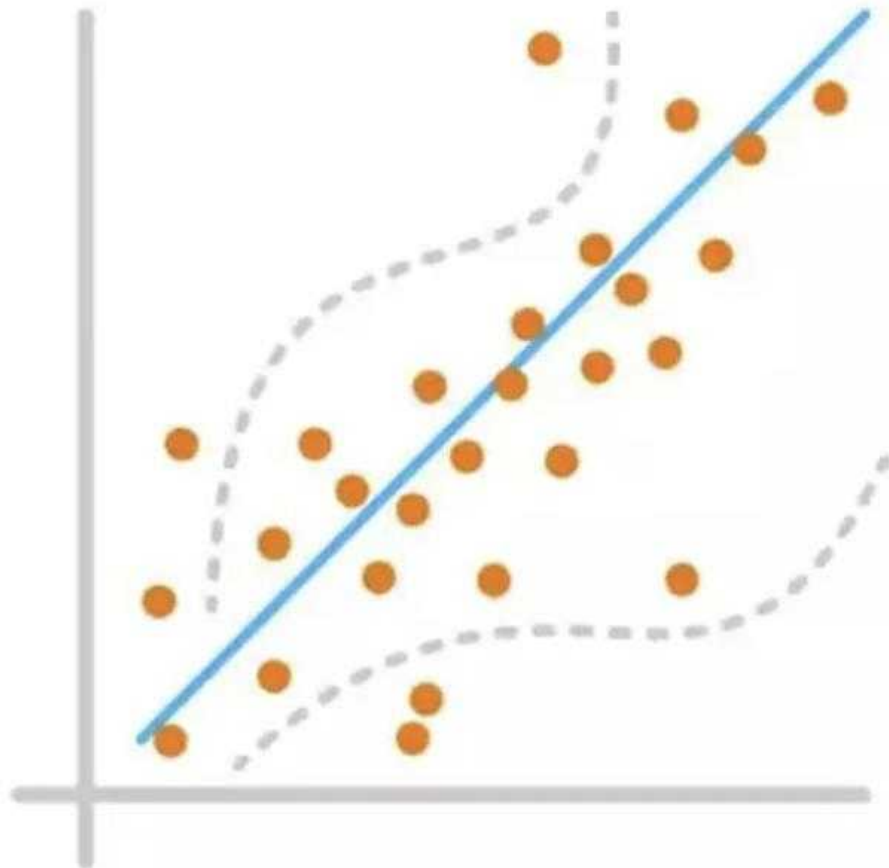
**Recall**

**Key:** **tp** = True Positive, **tn** = True Negative, **fp** = False Positive, **fn** = False Negative

| Metric Name      | Metric Forumla                                                                                            | Code                                                                                              | When to use                                                                                                                    |
|------------------|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Accuracy         | <b>Accuracy</b> = $\frac{tp + tn}{tp + tn + fp + fn}$                                                     | <code>tf.keras.metrics.Accuracy()</code><br>or<br><code>sklearn.metrics.accuracy_score()</code>   | Default metric for classification problems. Not the best for imbalanced classes.                                               |
| Precision        | <b>Precision</b> = $\frac{tp}{tp + fp}$                                                                   | <code>tf.keras.metrics.Precision()</code><br>or<br><code>sklearn.metrics.precision_score()</code> | Higher precision leads to less false positives.                                                                                |
| Recall           | <b>Recall</b> = $\frac{tp}{tp + fn}$                                                                      | <code>tf.keras.metrics.Recall()</code><br>or<br><code>sklearn.metrics.recall_score()</code>       | Higher recall leads to less false negatives.                                                                                   |
| F1-score         | <b>F1-score</b> = $2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$ | <code>sklearn.metrics.f1_score()</code>                                                           | Combination of precision and recall, usually a good overall metric for a classification model.                                 |
| Confusion matrix | NA                                                                                                        | Custom function<br>or<br><code>sklearn.metrics.confusion_matrix()</code>                          | When comparing predictions to truth labels to see where model gets confused. Can be hard to use with large numbers of classes. |

# Figures of Merit: Regression Tasks

## 4 Common Regression Metrics



1

Mean Squared Error (MSE)

2

Root Mean Squared Error (RMSE)

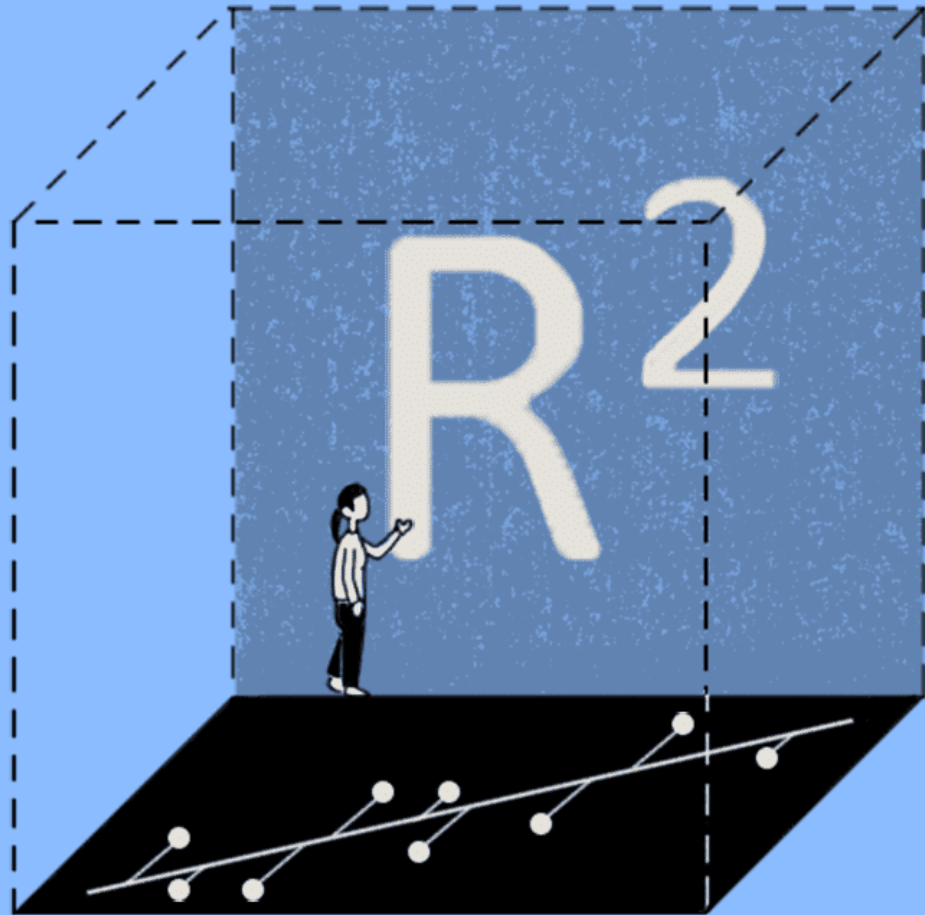
3

Mean Absolute Error (MAE)

4

R-squared ( $R^2$ )





# R-Squared

*['är 'skwerd]*

A statistical measure that represents the proportion of the variance for a dependent variable that's explained by an independent variable or variables in a regression model.

A stylized illustration of a police officer in a light blue uniform and cap, pointing their right hand towards the center text. The officer has red hair and is wearing a dark blue belt with a silver buckle.

# R-Squared

Vs

# Adjusted R-Squared

A stylized illustration of a police officer in a light blue uniform and cap, holding a red clipboard and writing with a pen. The officer is wearing a dark blue belt with a silver buckle.

Learn the key difference between R-Squared & Adjusted R-Squared in a meaningful way.

[dataaspirant.com](https://dataaspirant.com)

# R-Squared vs. Adjusted R-Squared

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

$$Adjusted R^2 = 1 - \frac{(1 - R^2)(N - 1)}{N - p - 1}$$

Where

$R^2$  Sample R-Squared

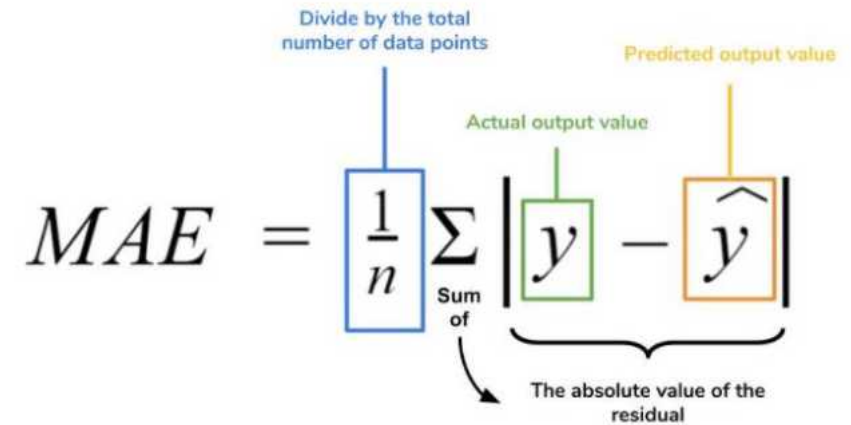
$N$  Total Sample Size

$p$  Number of independent  
variable



# Figures of Merit: Regression Tasks

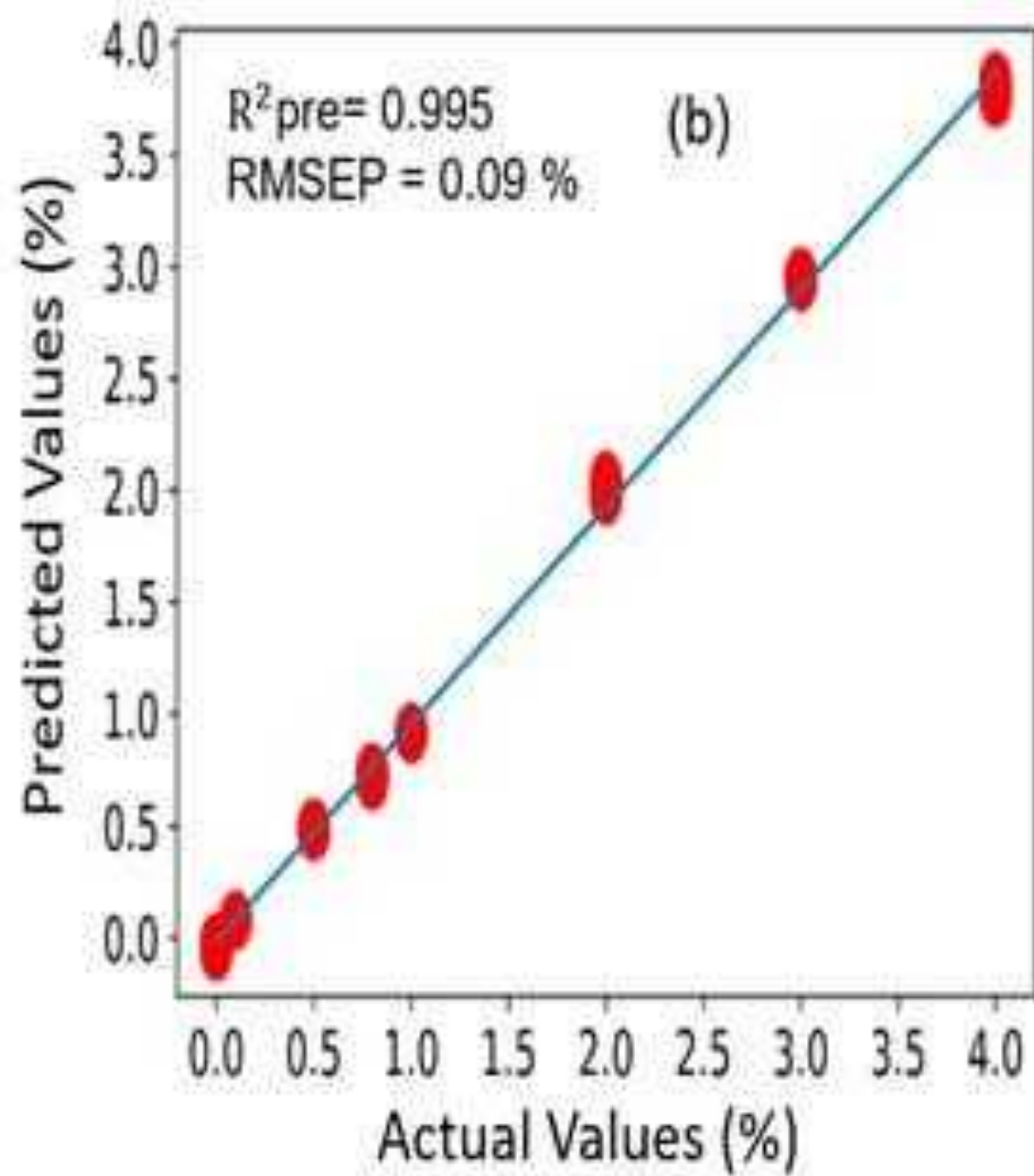
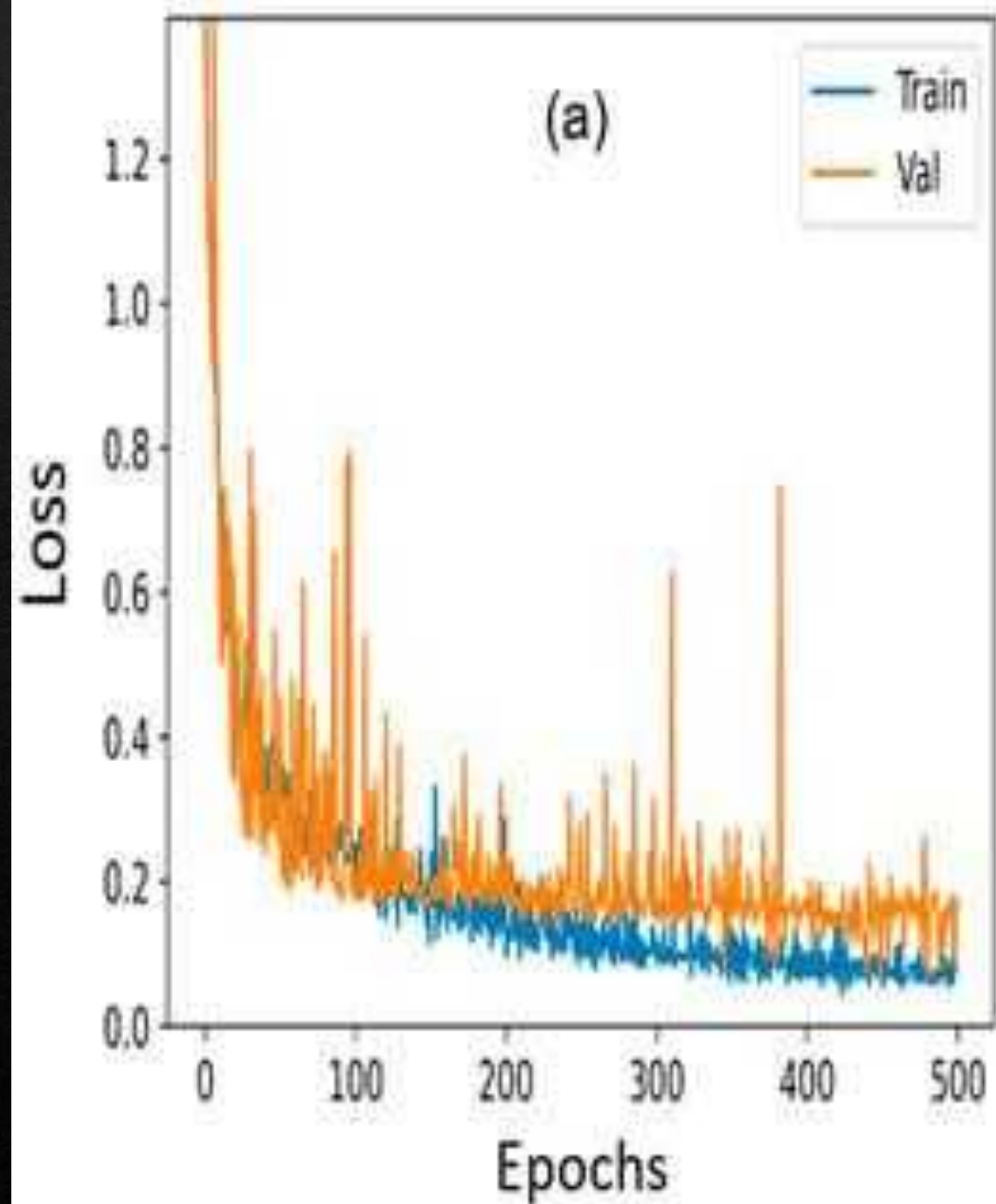
$$RMSE = \sqrt{\frac{\sum (y_i - \hat{y}_i)^2}{N - P}}$$

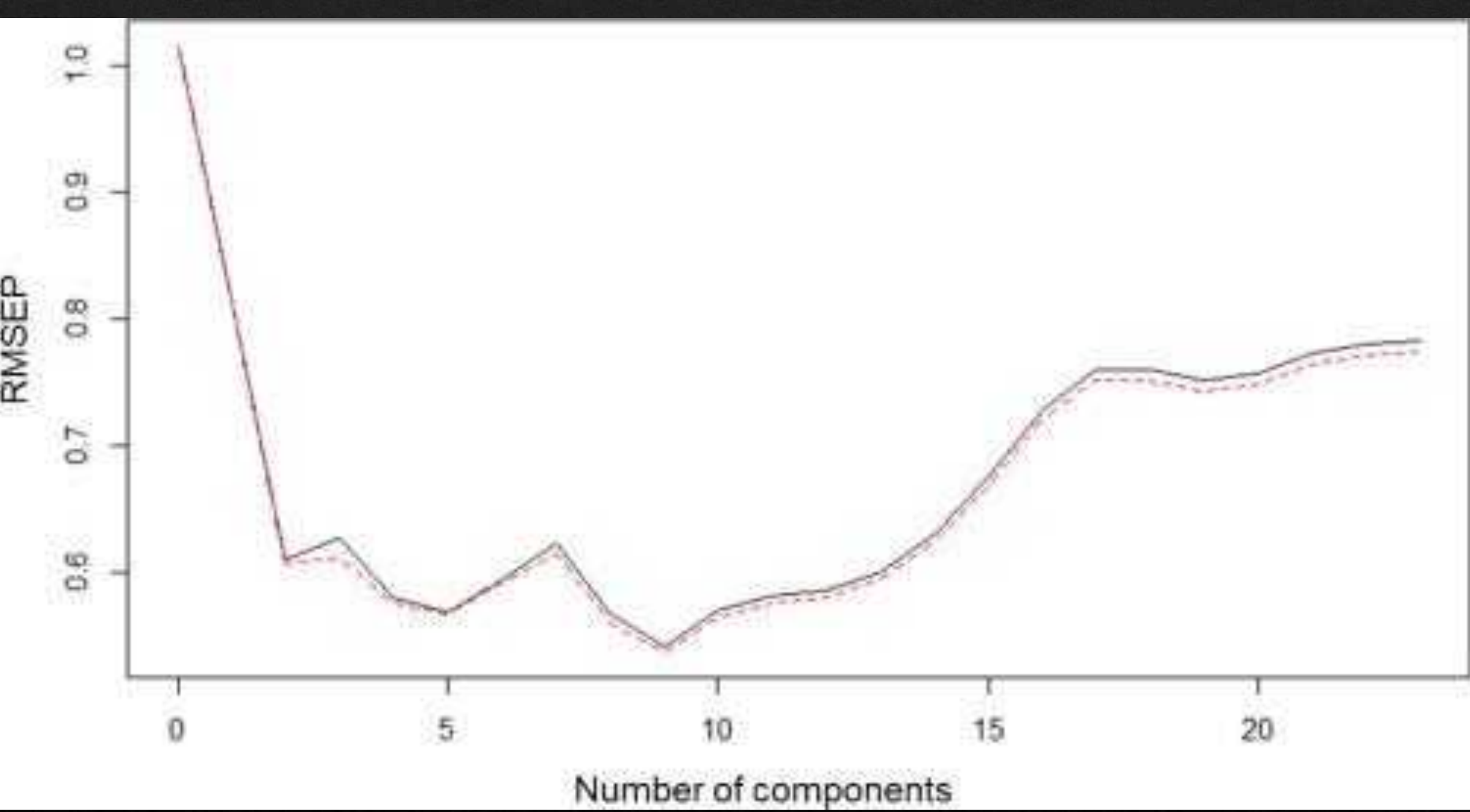


The diagram illustrates the Mean Absolute Error (MAE) formula with several annotations:

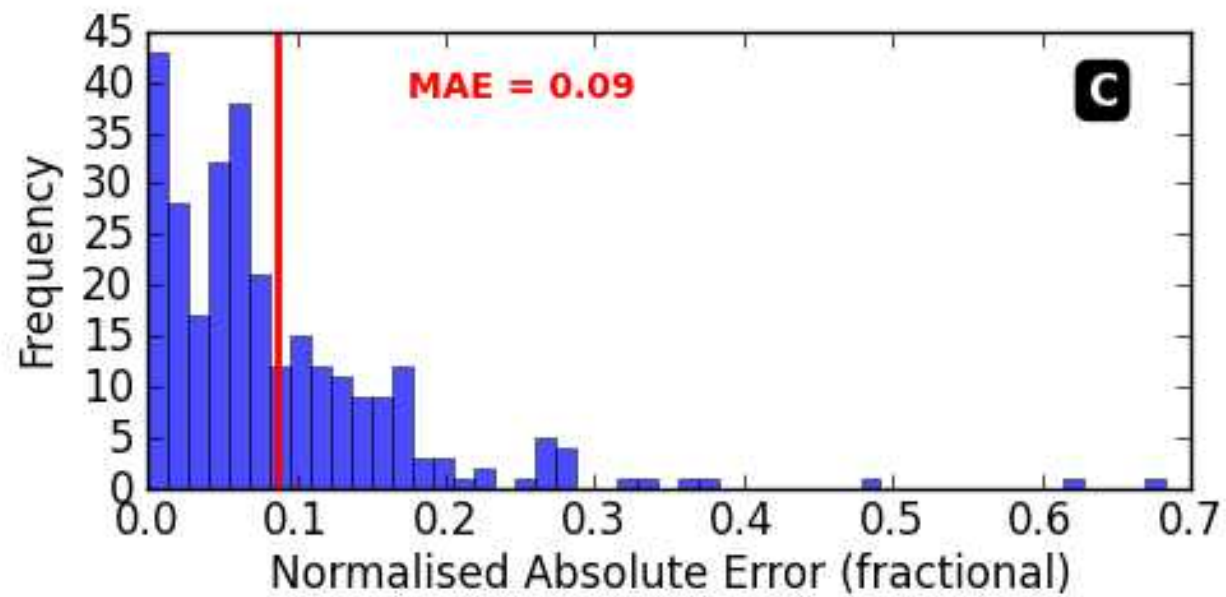
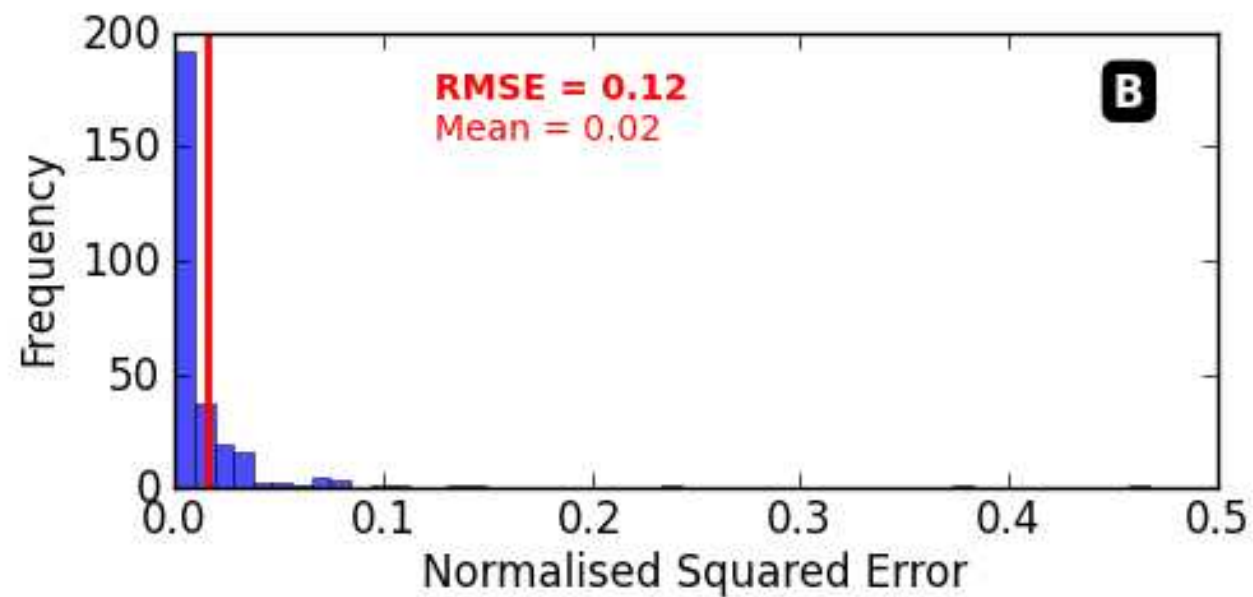
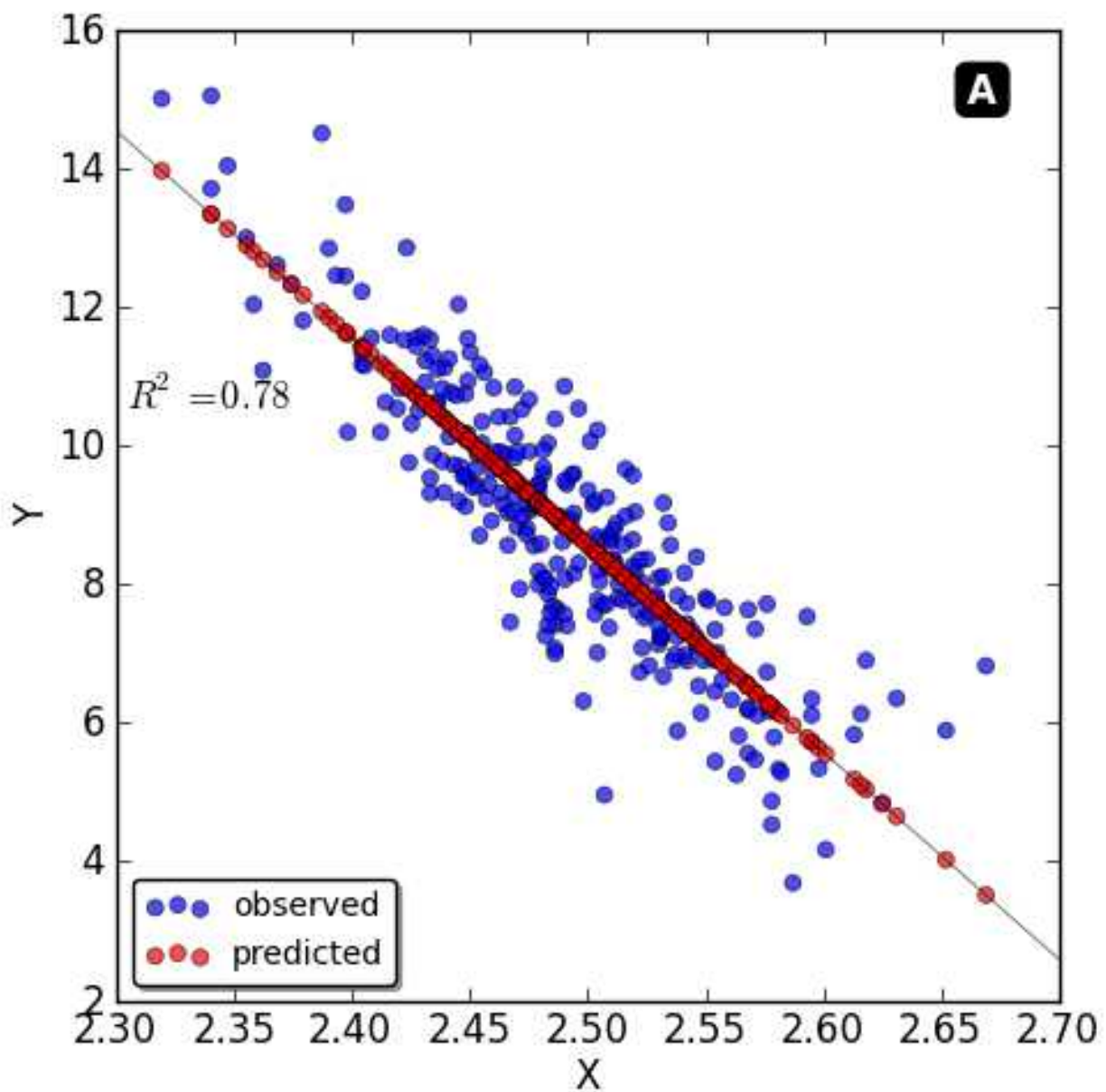
- Divide by the total number of data points:** A blue line points to the  $\frac{1}{n}$  term in the formula.
- Sum of:** A blue line points to the summation symbol  $\sum$ .
- Actual output value:** A green line points to the  $y$  term inside the absolute value.
- Predicted output value:** An orange line points to the  $\hat{y}$  term inside the absolute value.
- The absolute value of the residual:** A bracket under the  $|y - \hat{y}|$  term is labeled with this text.

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$



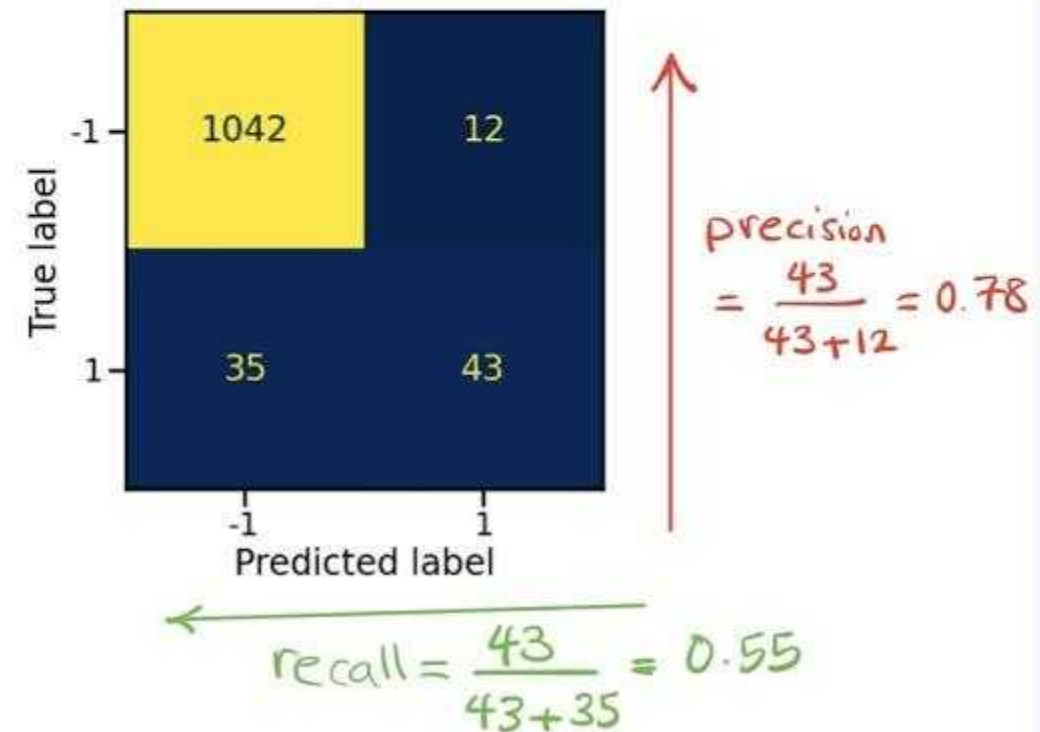
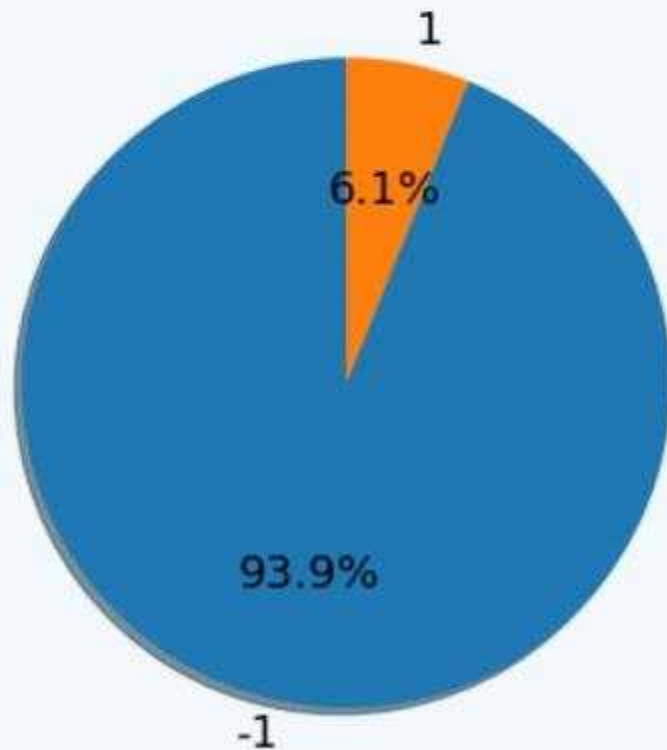




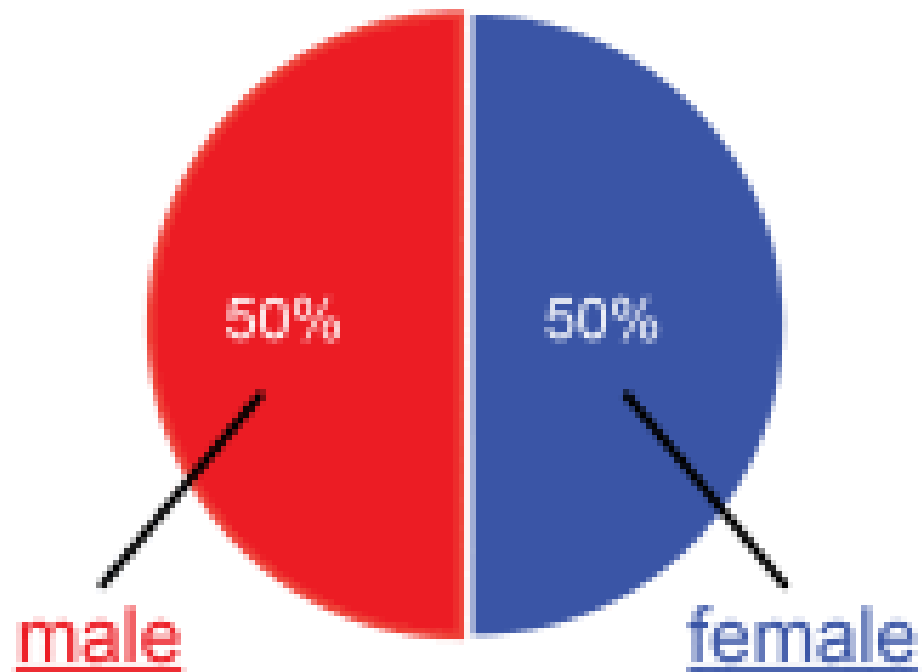


# Machine Learning with Imbalanced Data

## Part I

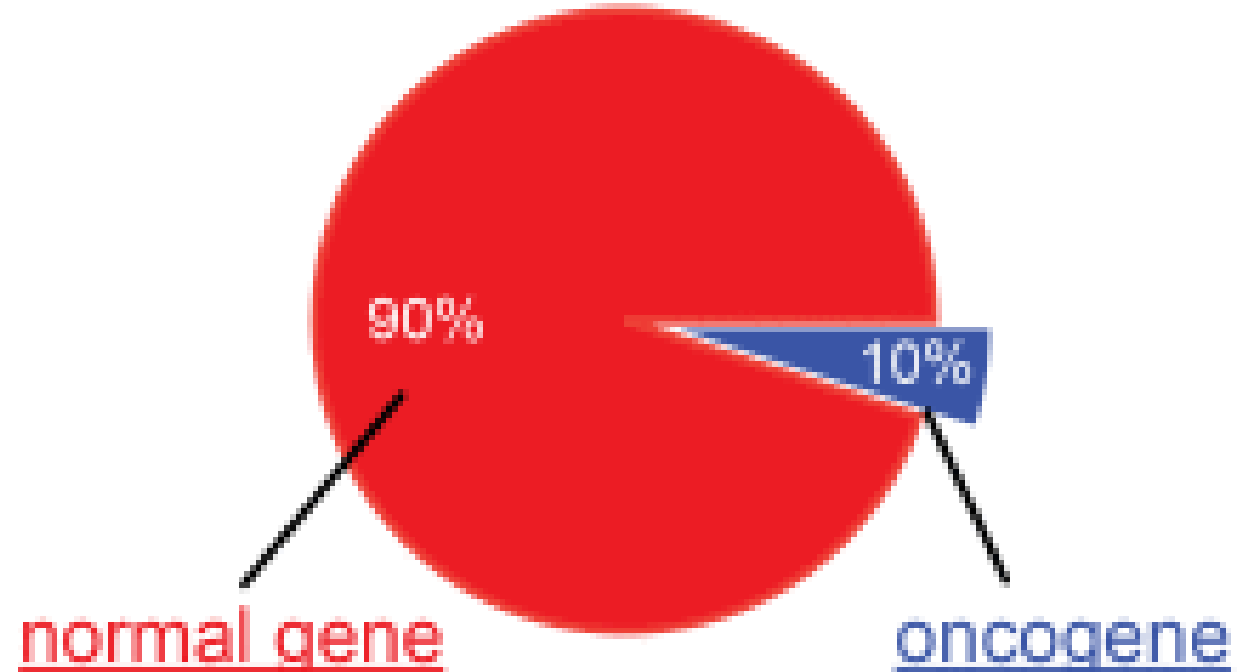


## Example of balanced and imbalanced data



Negatives = Positives

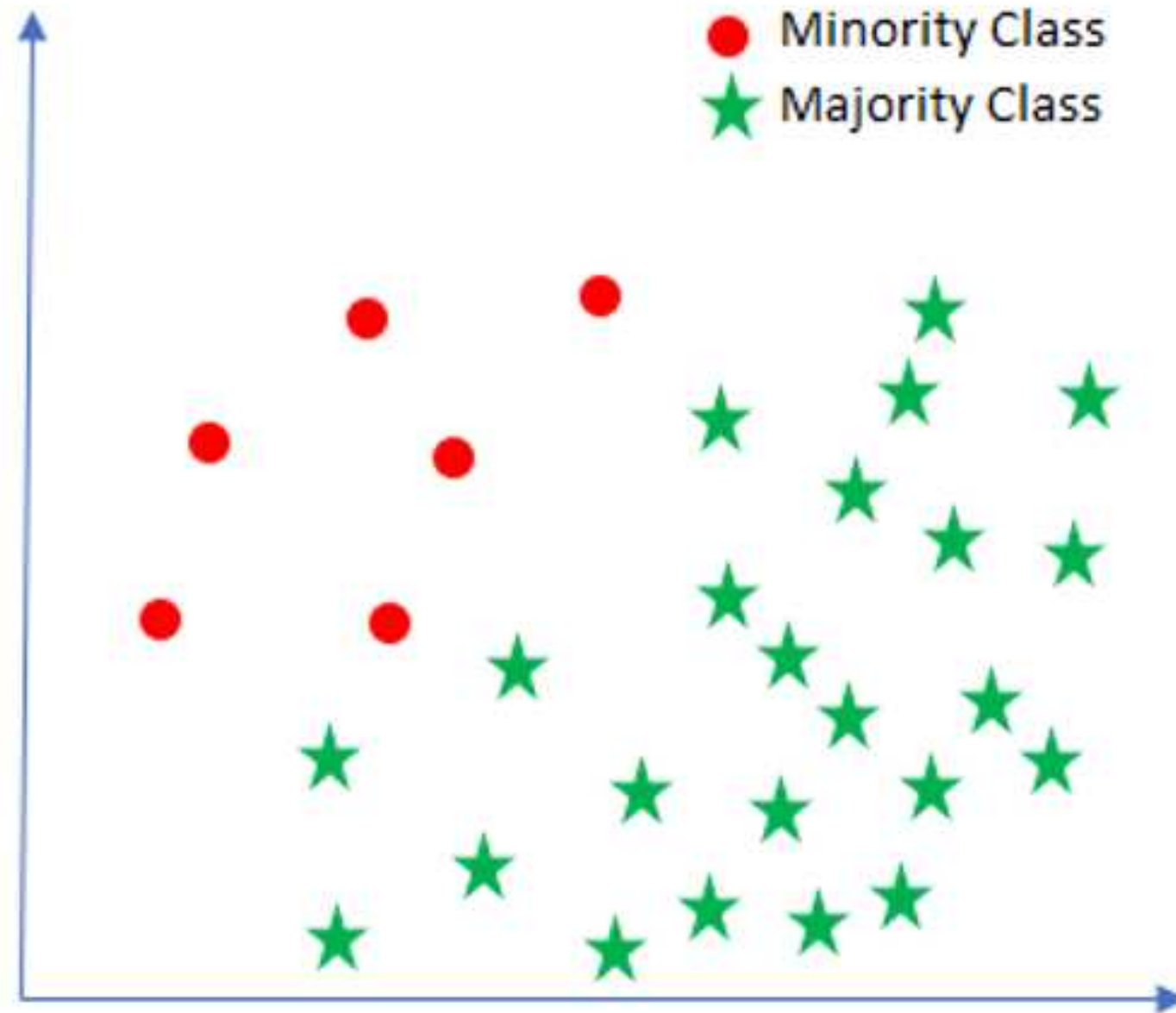
Balanced



Negatives > Positives

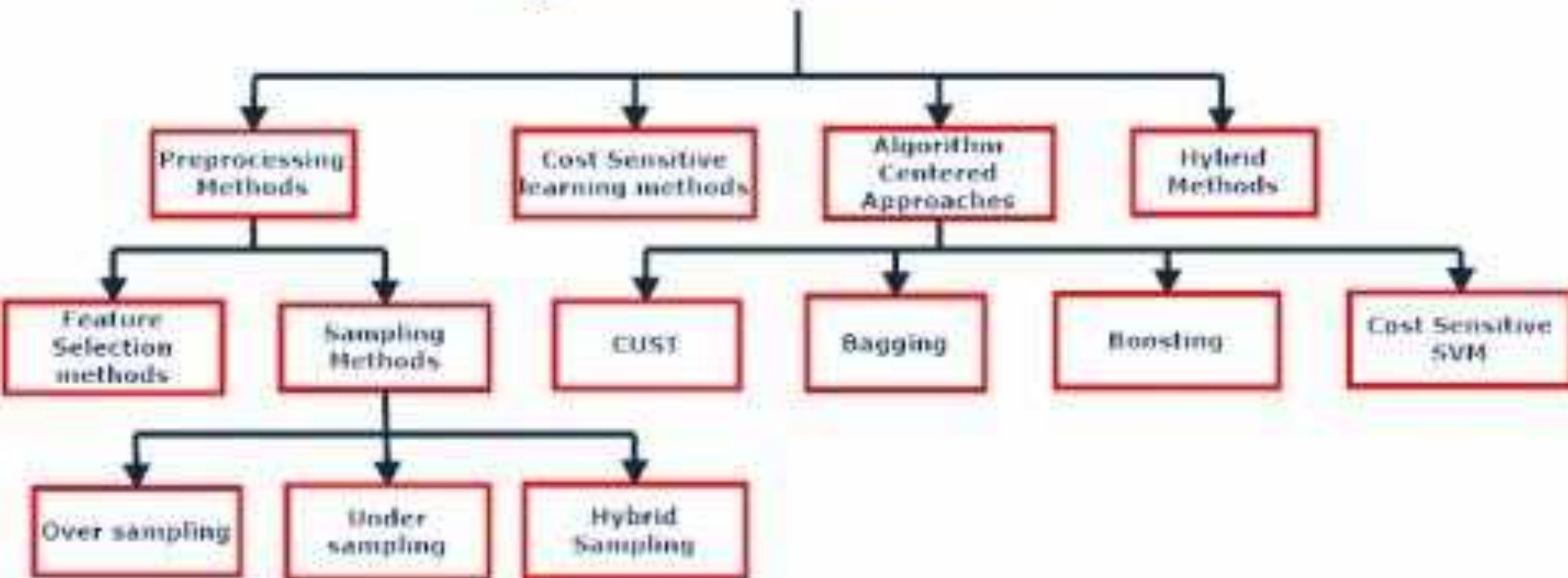
Imbalanced

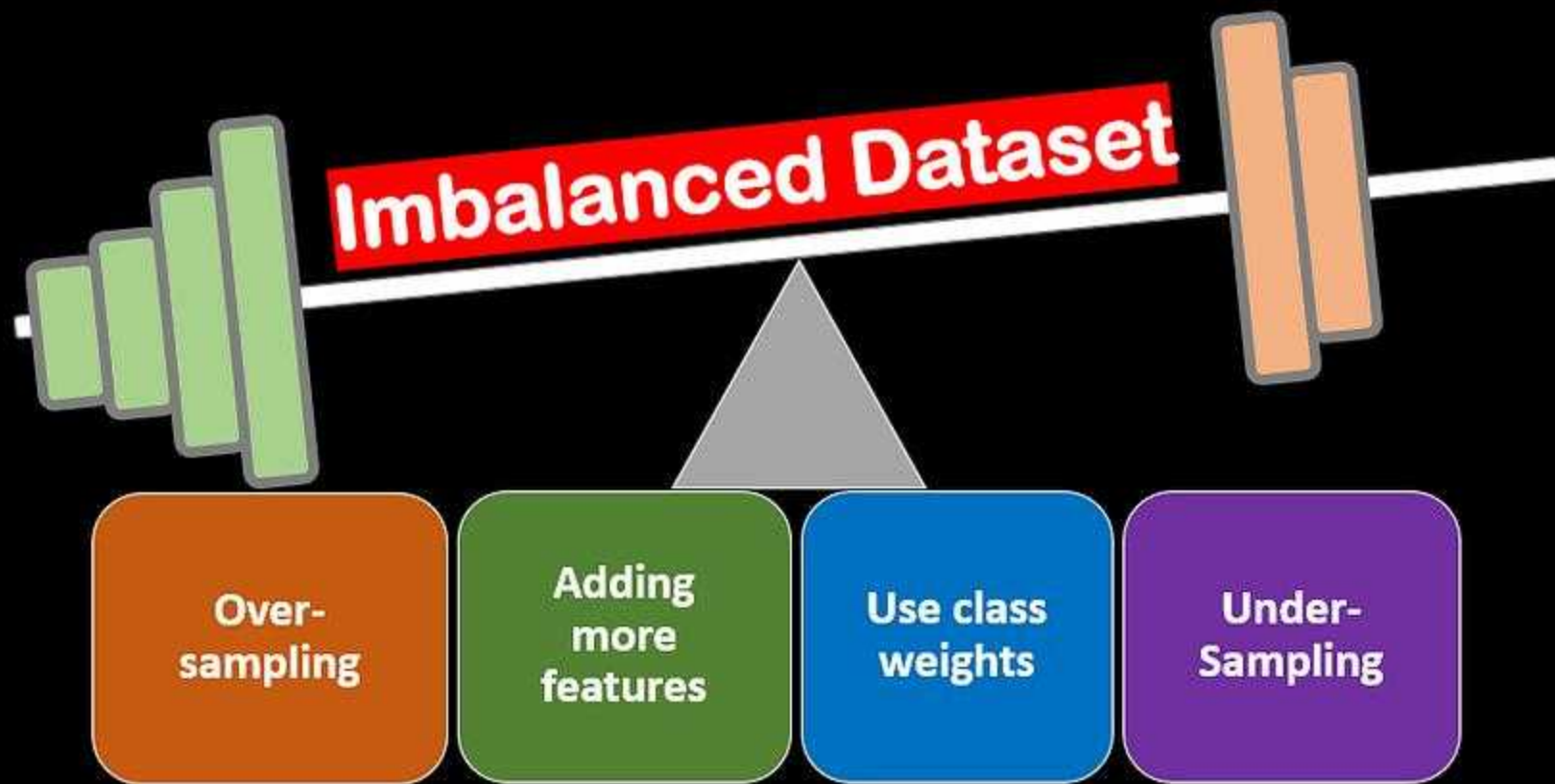
# Imbalanced Data





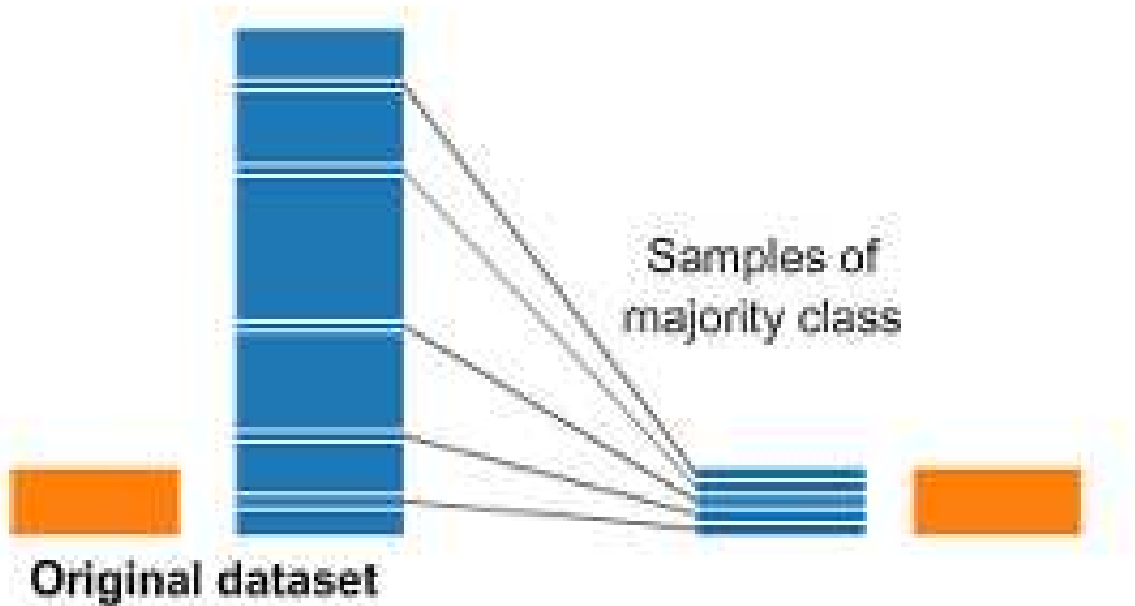
## Approaches for Imbalanced data



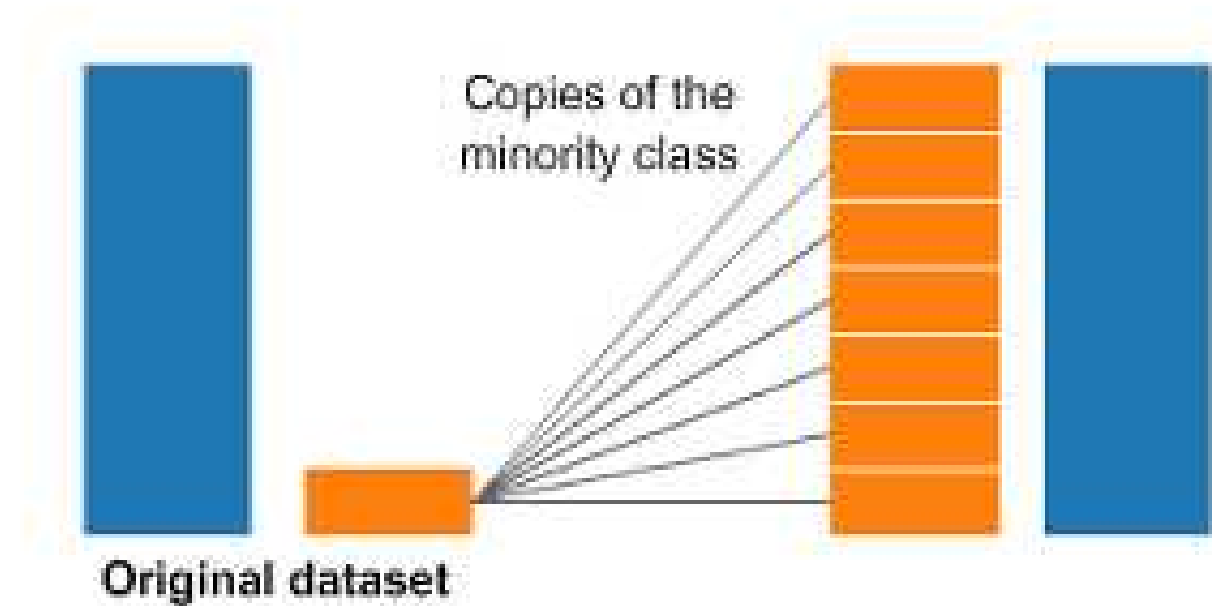


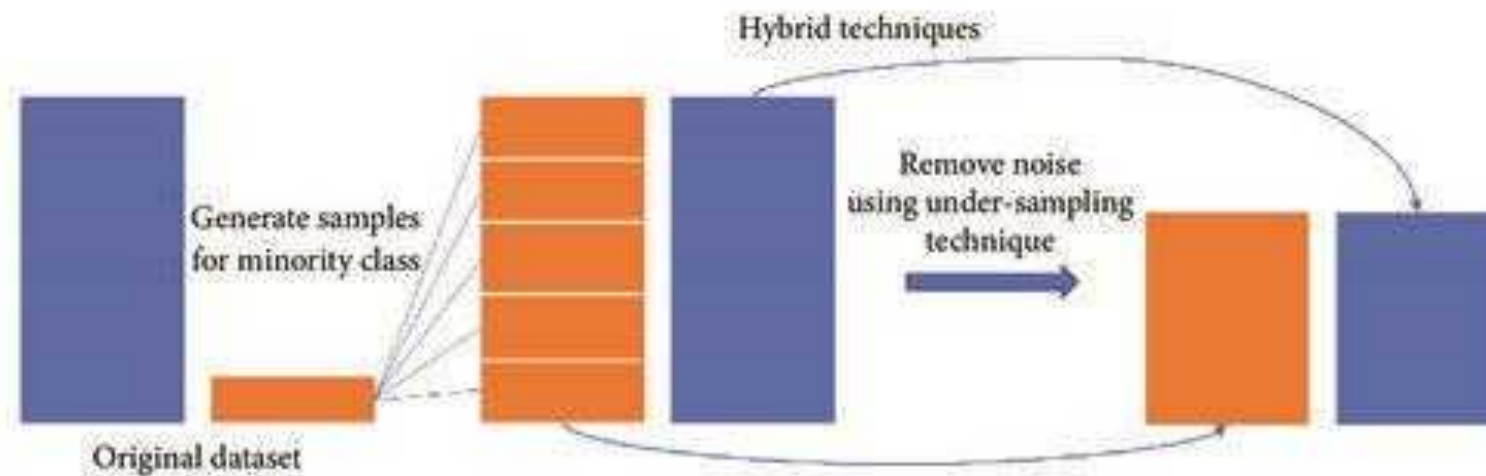
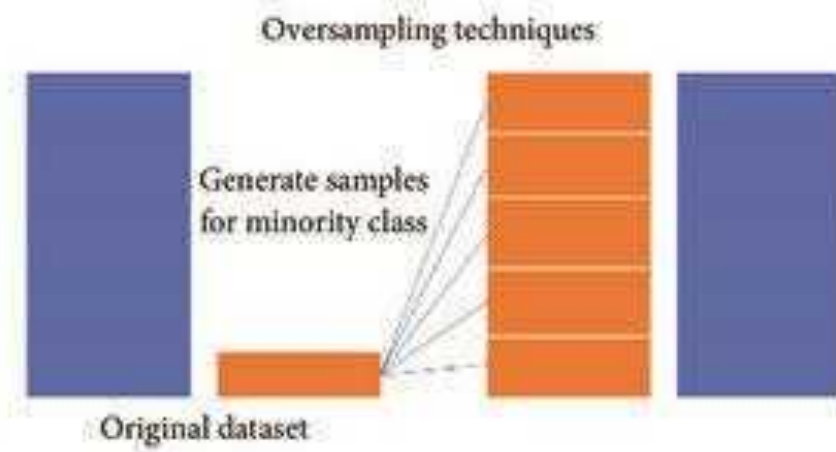
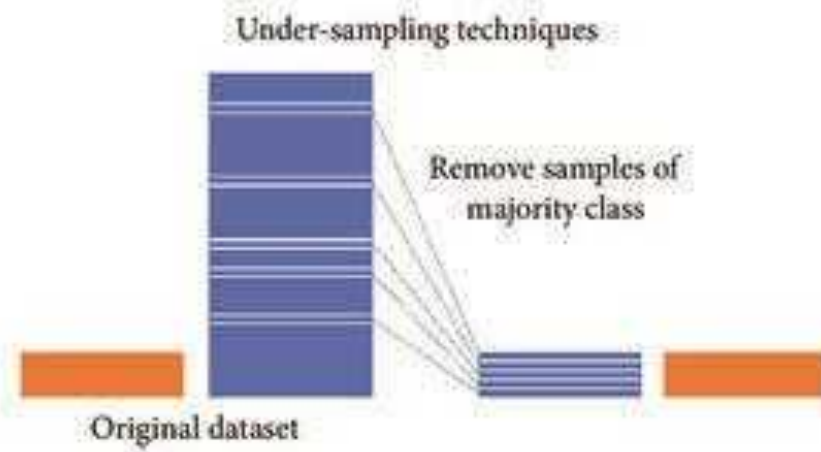
# Unbalanced Classes: How to deal?

Undersampling



Oversampling



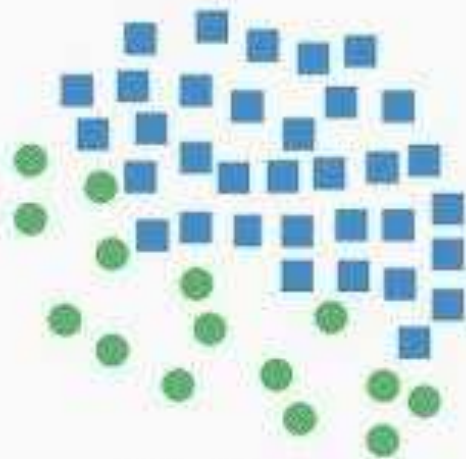




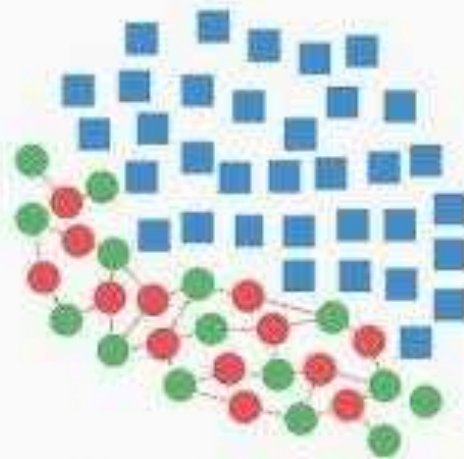
# SMOTE

## HANDLE IMBALANCED DATASET

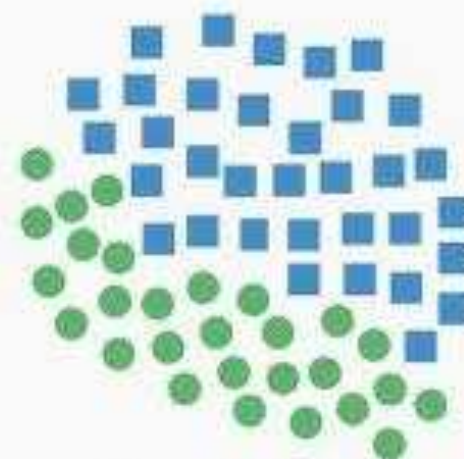
Synthetic Minority Oversampling Technique



Original Dataset



Generating Samples

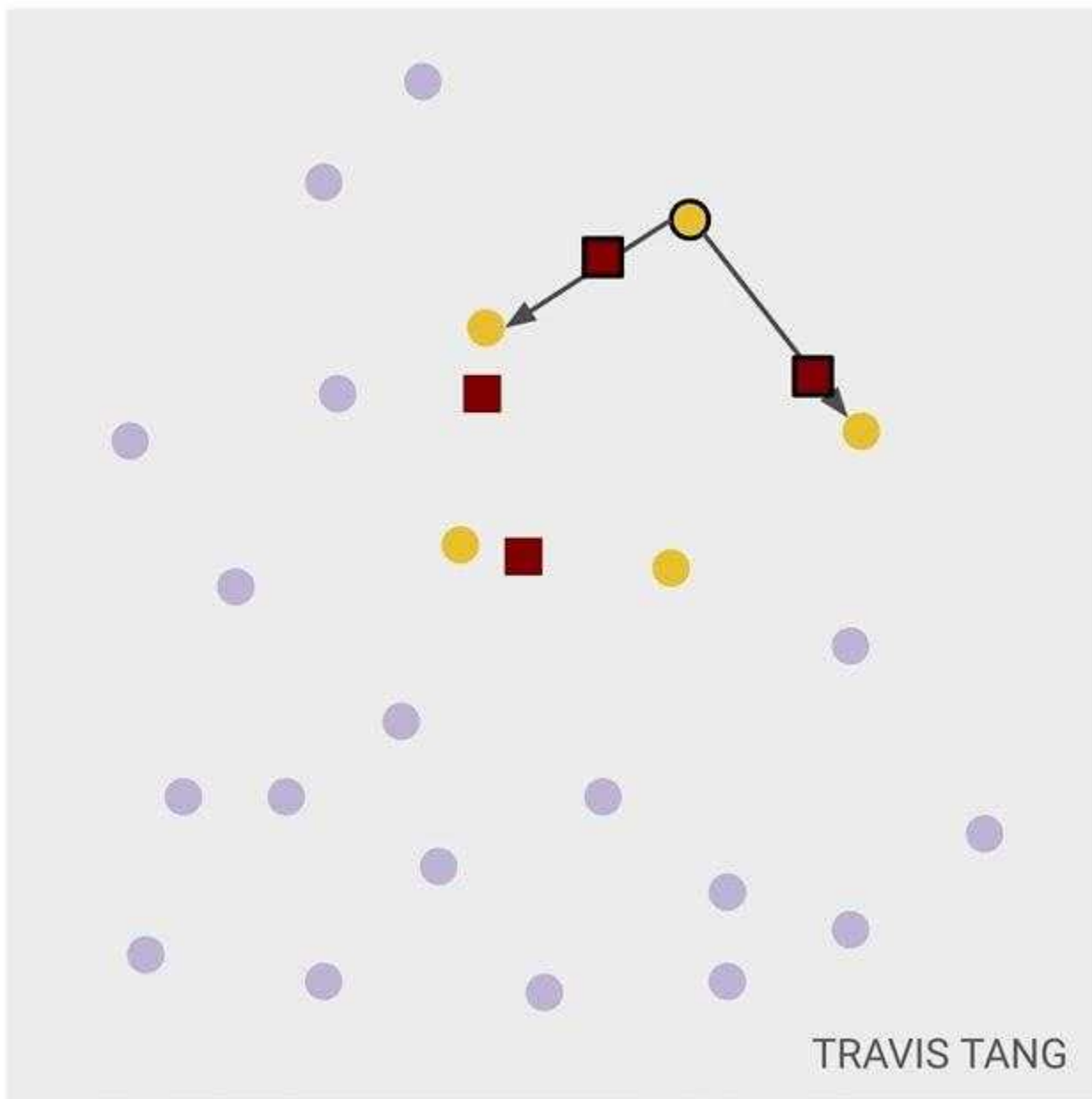


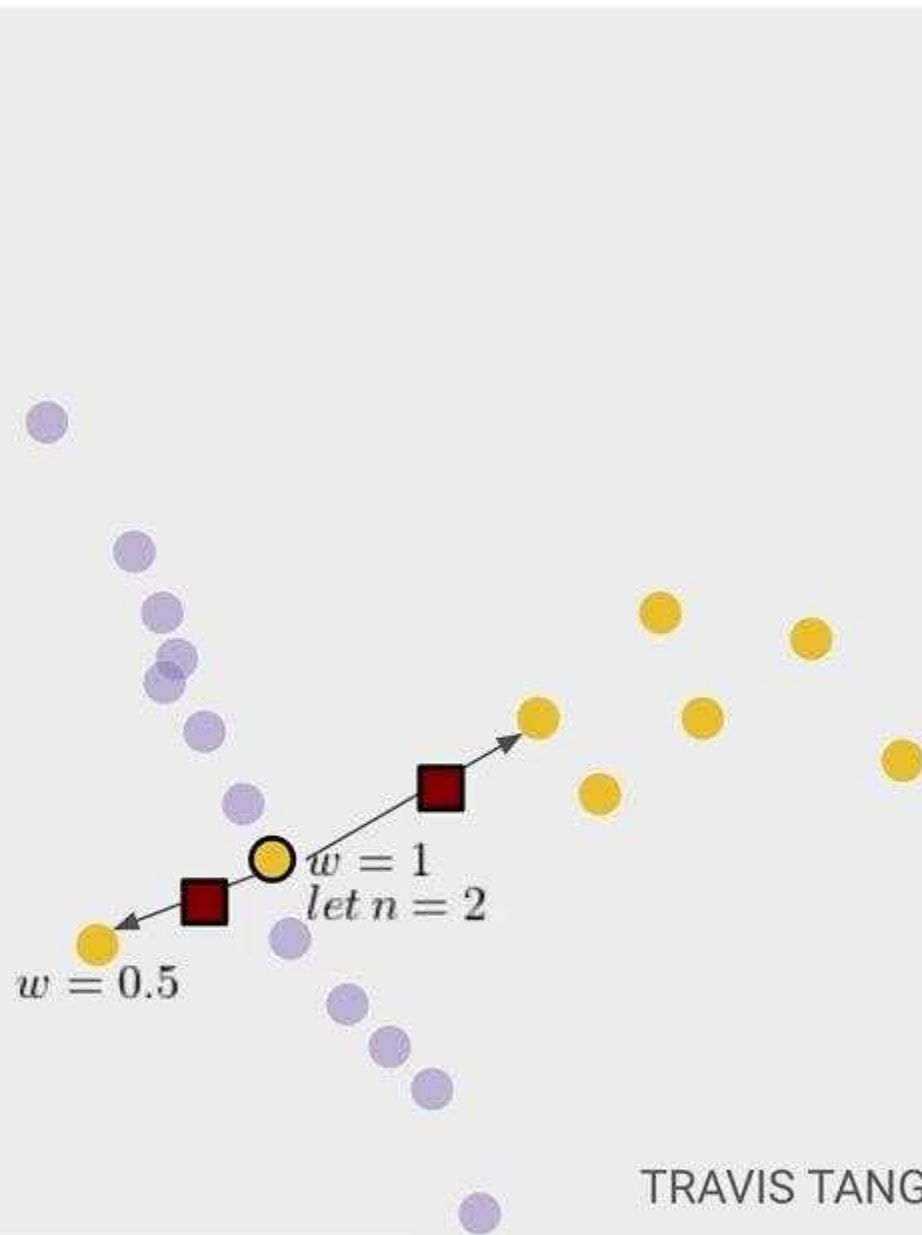
Resampled Dataset

# The SMOTE Algorithm

FOR OVERSAMPLING THE MINORITY CLASS

1. Identify a data point from the minority class.
2. Find its  $k$  nearest neighbor. Here,  $k$  is 2.
3. Create **synthetic data** between the data point and its neighbors.
4. **Repeat steps 1 to 3.**



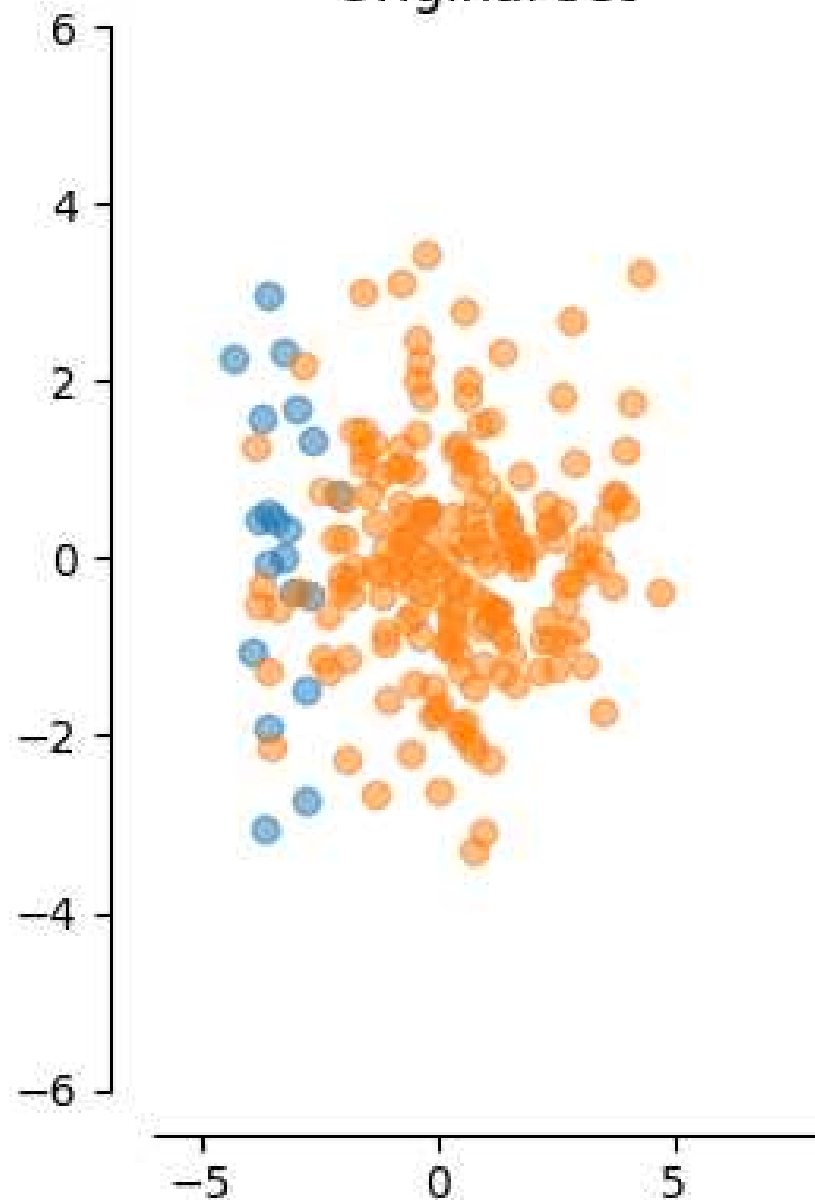


# The ADASYN Algorithm

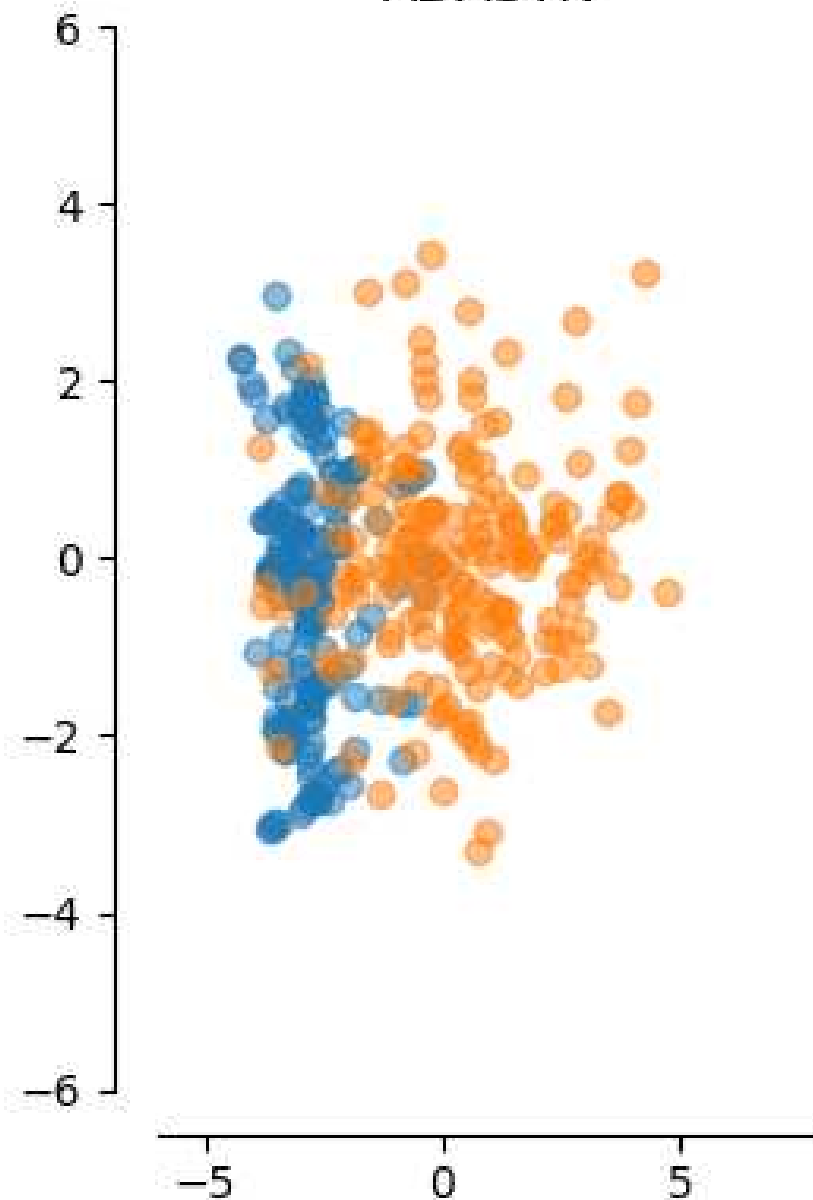
FOR OVERSAMPLING THE MINORITY CLASS

1. For each minority data point  $i$ ,
  - a. Find its  $k$  neighbors ( $k=2$ )
  - b. Find its weight
$$w_i = \frac{\text{num neighbors in majority class}}{k \text{ neighbors}}$$
2. For each point with  $w > 0$ ,
  - a. Find its minority neighbors.
  - b. **Create  $n$  data points** between itself and its minority neighbour.

Original set

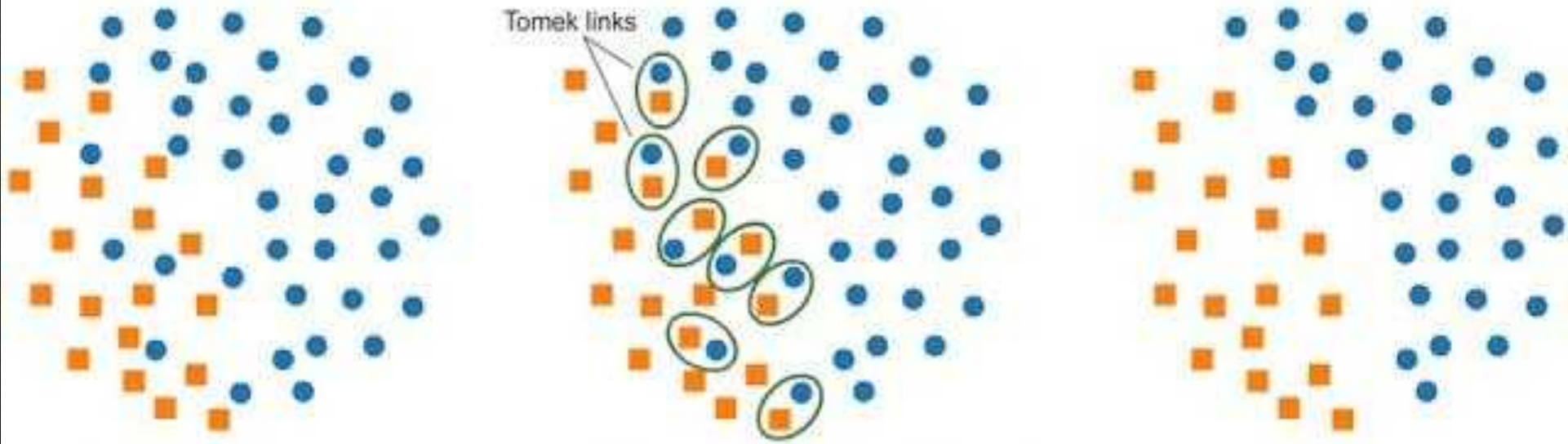


ADASYN





# Tomek links Algorithm



**Under-sampling – Imbalanced data**

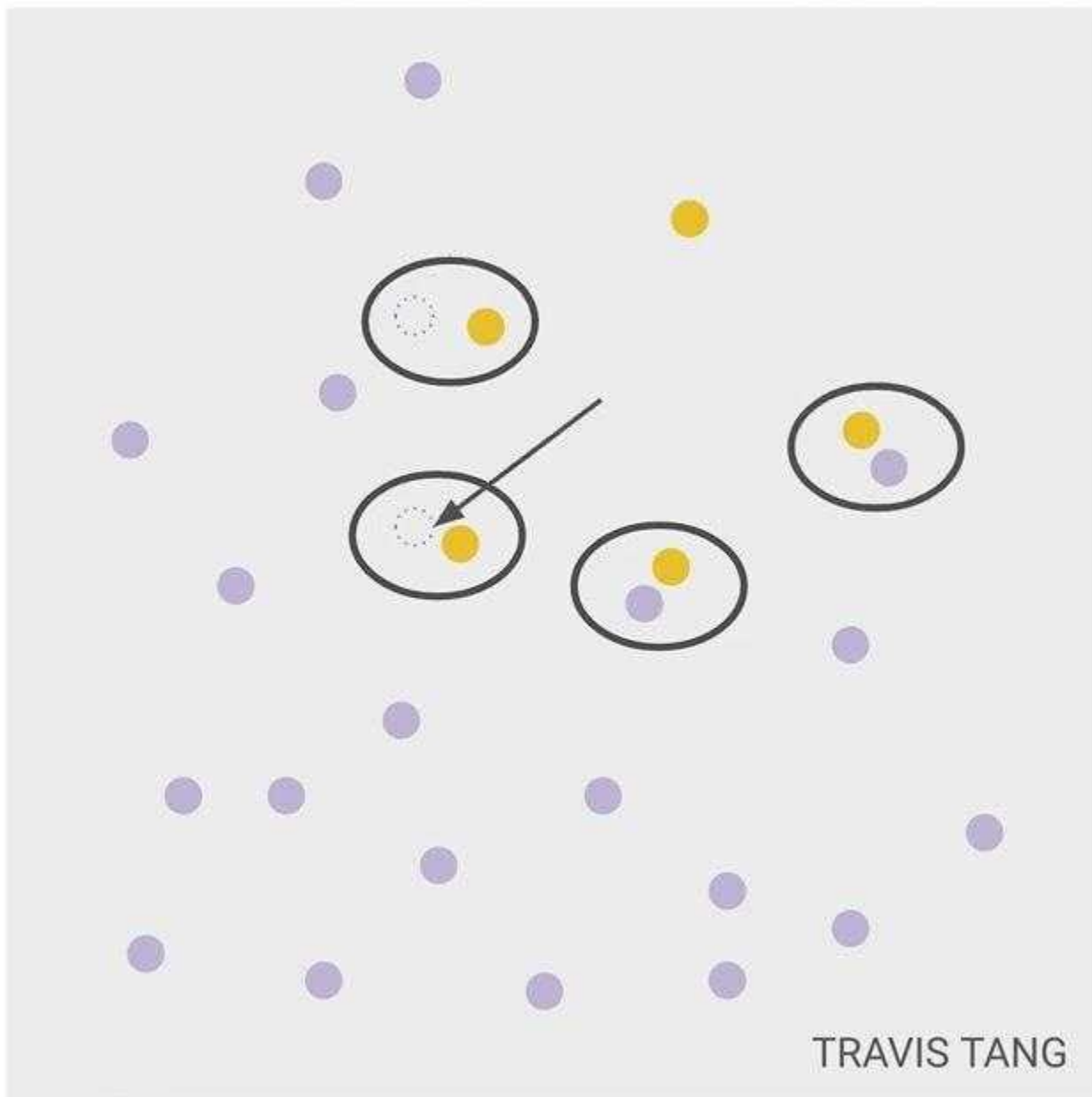
Like, Share and Subscribe to Mahesh Huddar

Visit: [vtupulse.com](http://vtupulse.com)

# Tomek Link Algorithm

FOR UNDERSAMPLING THE MAJORITY CLASS

1. Find Tomek links, which are points from different classes that are close to one another.
2. For each pair, remove the point from the majority class.



# An evaluation strategy to select and discard sampling preprocessing methods for imbalanced datasets: A focus on classification models

Alexander de P. Rodrigues, Aderval S. Luna  , Licarion Pinto  

Show more 

 Add to Mendeley  Share  Cite

<https://doi.org/10.1016/j.chemolab.2023.104933> 

[Get rights and content](#) 

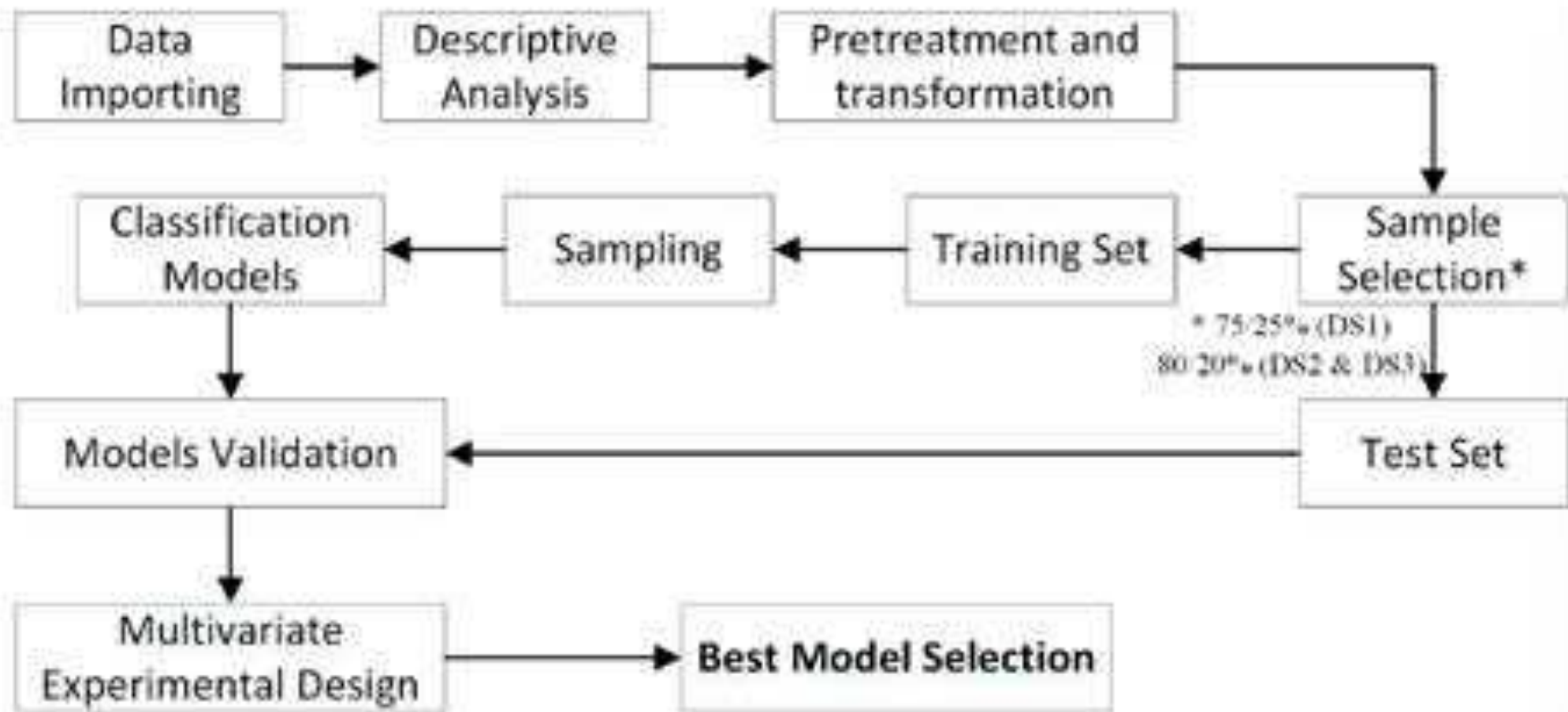
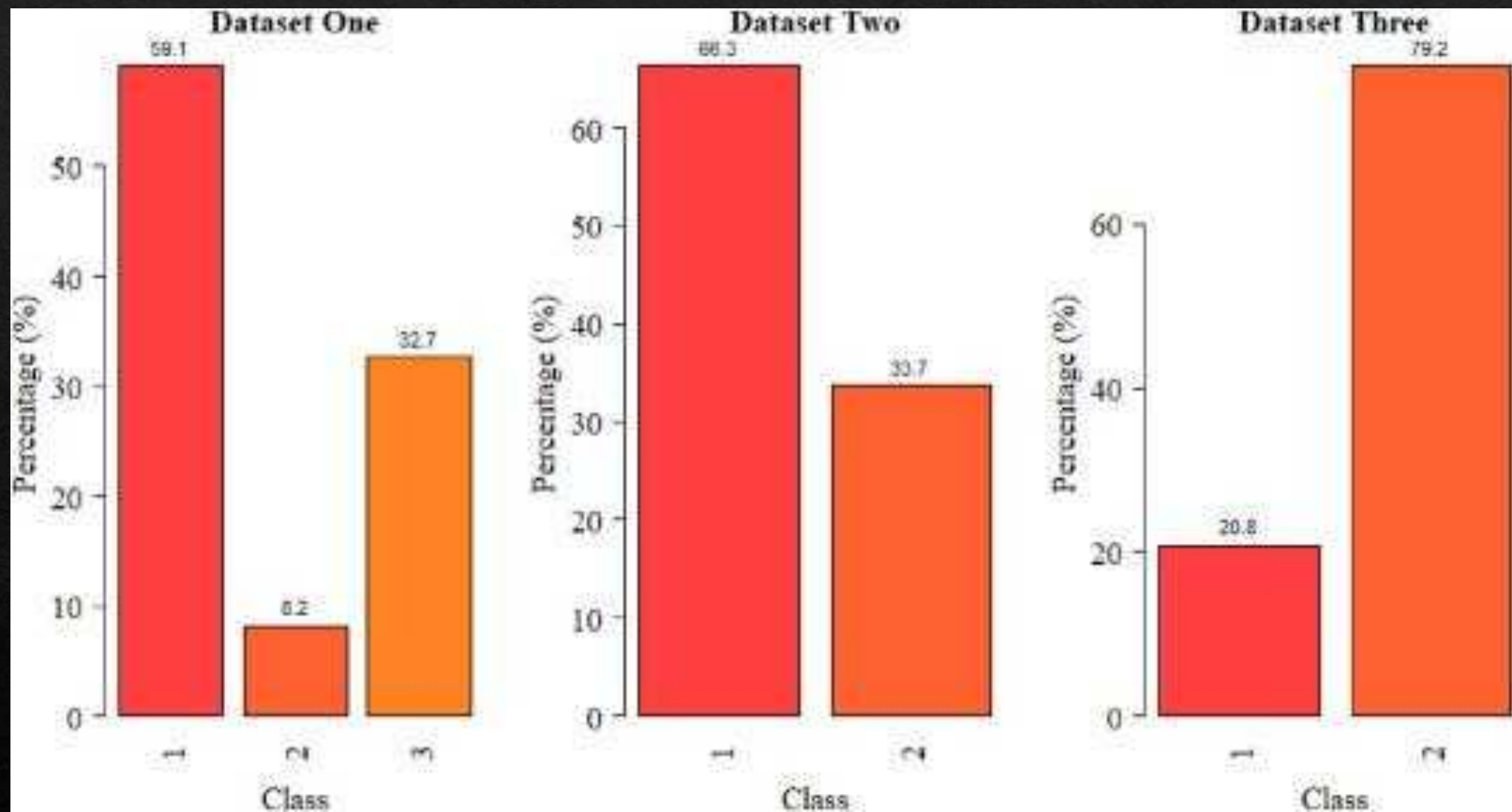
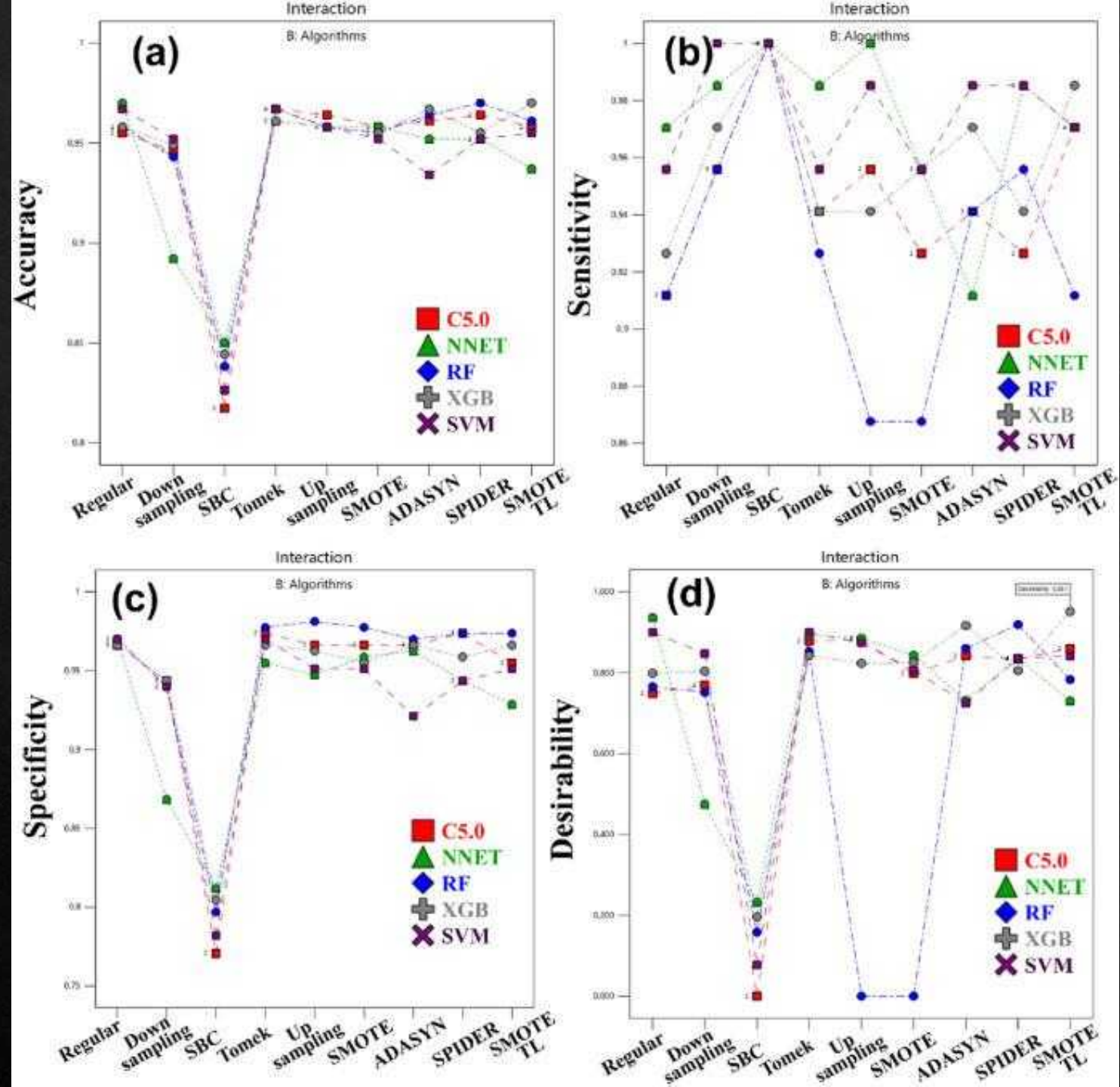


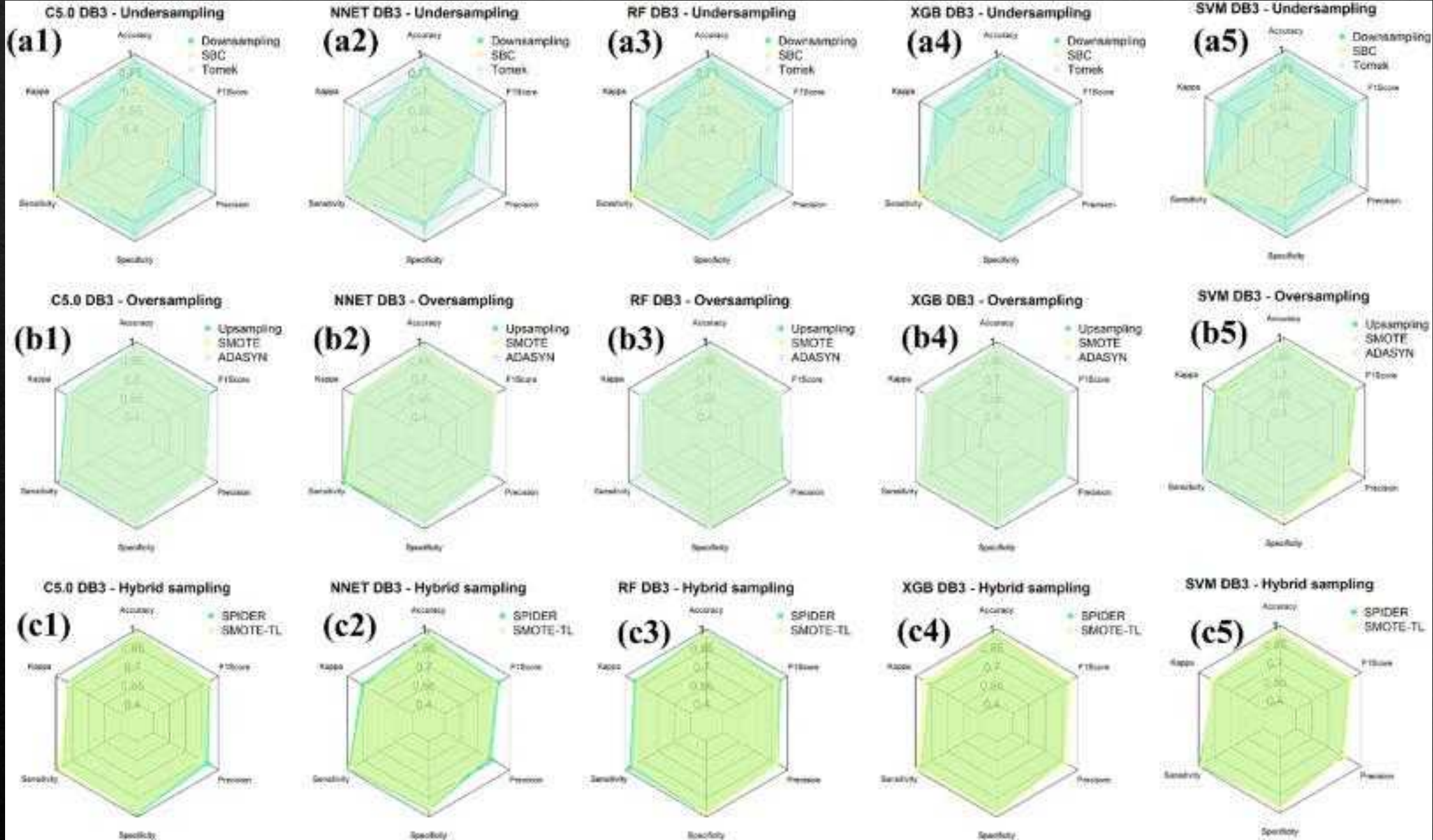
Fig. 1. Chemometric flowchart of the resampling evaluation strategy.











Alberto Fernández · Salvador García  
Mikel Galar · Ronaldo C. Prati  
Bartosz Krawczyk · Francisco Herrera

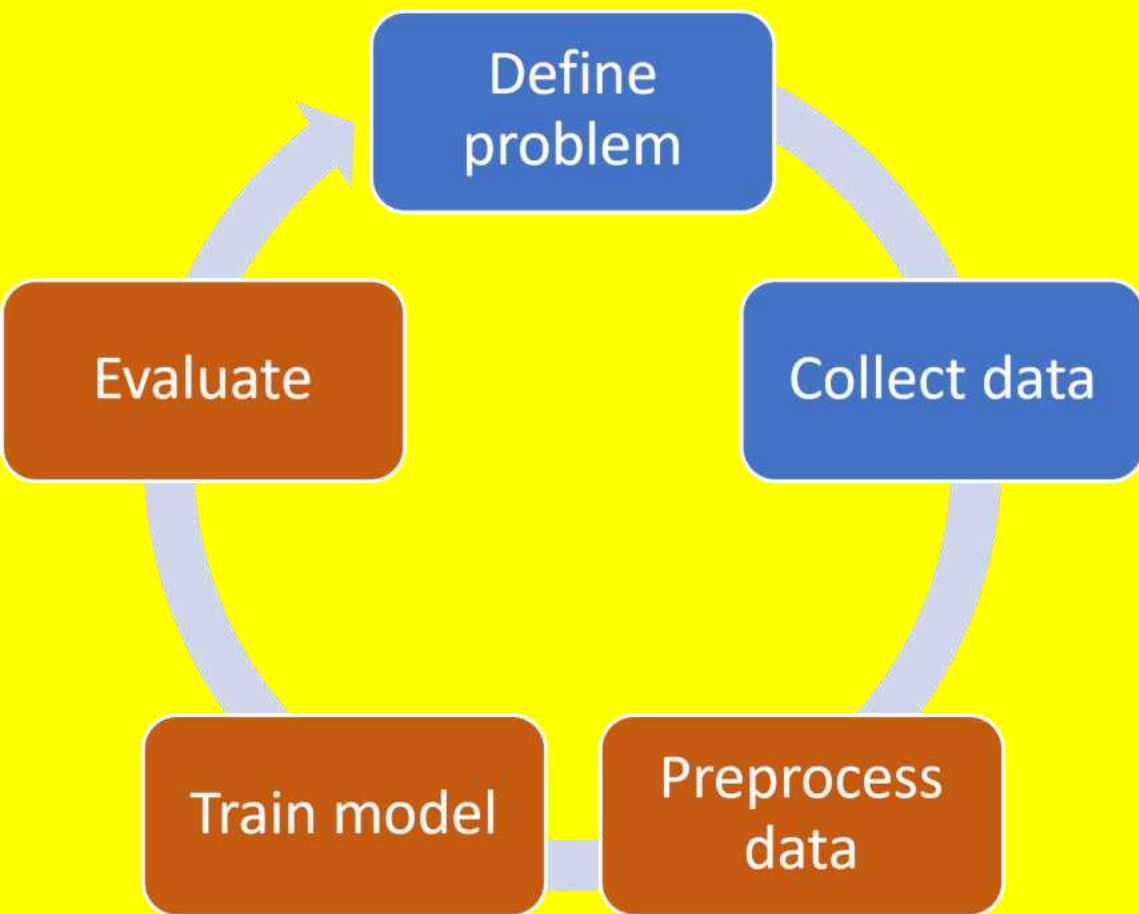
# Learning from Imbalanced Data Sets



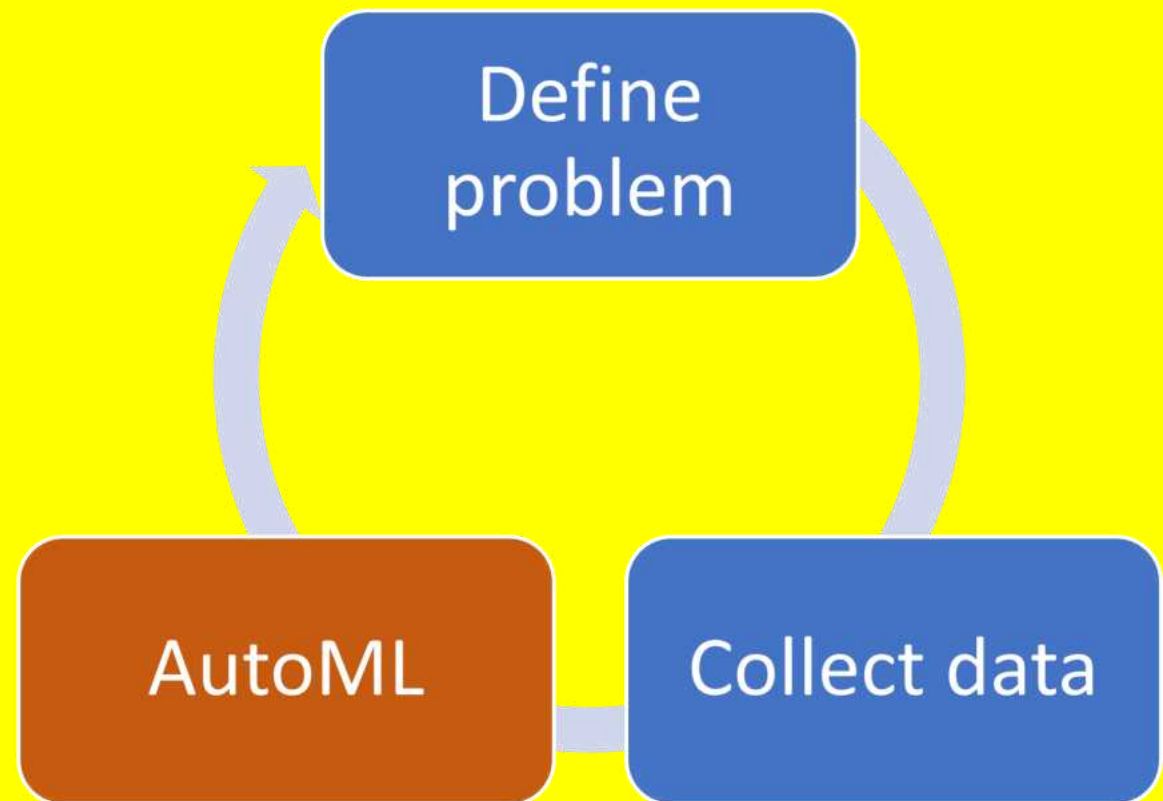
# Automated Machine Learning



## Traditional ML training workflow



## AutoML workflow



# AutoML



Simplifies Process



Building Machine Learning Models



Non - Experts

Example: Prediction of Emails

Data

Excode



1



0

AutoML

Features

Parameters

Algorithms

Result

Spam Emails

→ 5

Non-spam Emails

→ 10

## Traditional Machine Learning Workflow

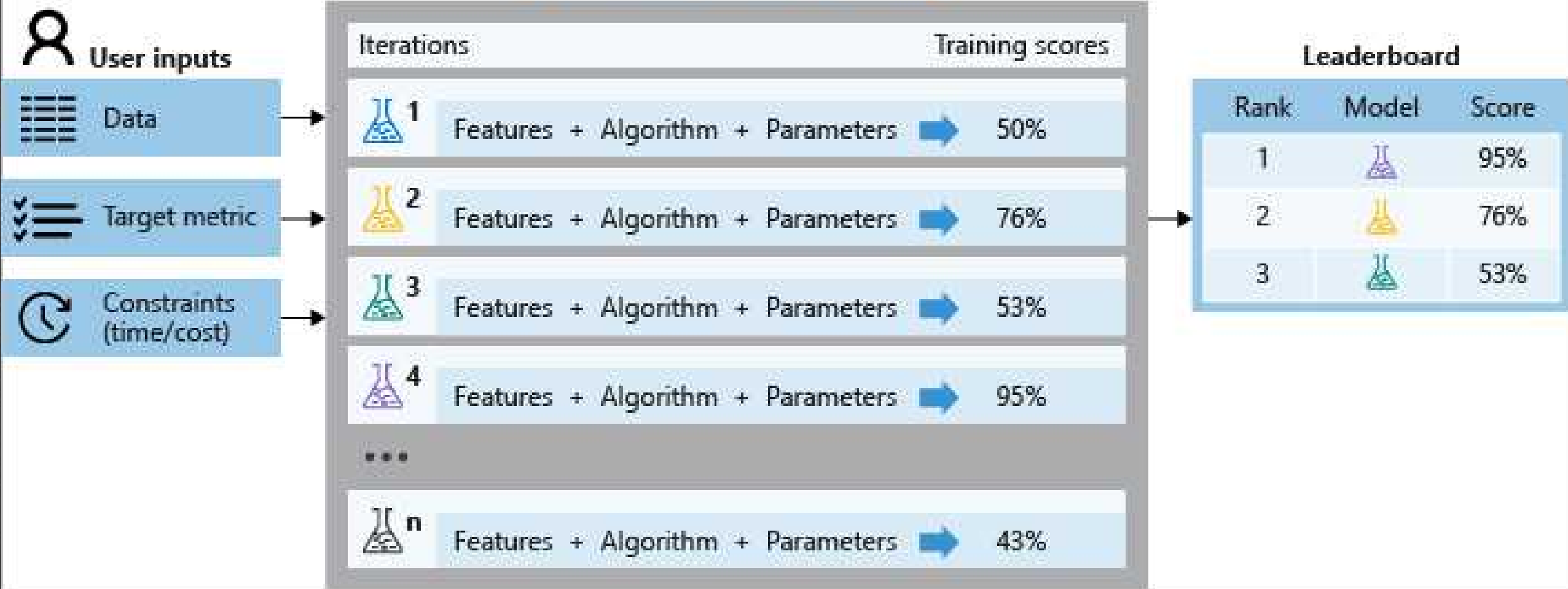


## AutoML Workflow





## Automated machine learning



## Tool Based Frameworks



## Open Source Frameworks



## Cloud Based Frameworks



## Cloud solutions



2015

2016

2017

2018

2019

2020

2021

Auto-  
Sklearn



AutoKeras



## Open-source solutions

## ARTICLE

# Employing Auto-Machine Learning Algorithms for Predicting the Cold Filter Plugging and Kinematic Viscosity at 40 °C in Biodiesel Blends using Vibrational Spectroscopy Data

Aderval Severino Luna<sup>\*1</sup>  , Alexandre Rodrigues Torres<sup>2</sup> , Camilla Lima Cunha<sup>2</sup> , Igor C. A. Lima<sup>1</sup>, Luis G. Nonato<sup>3</sup>

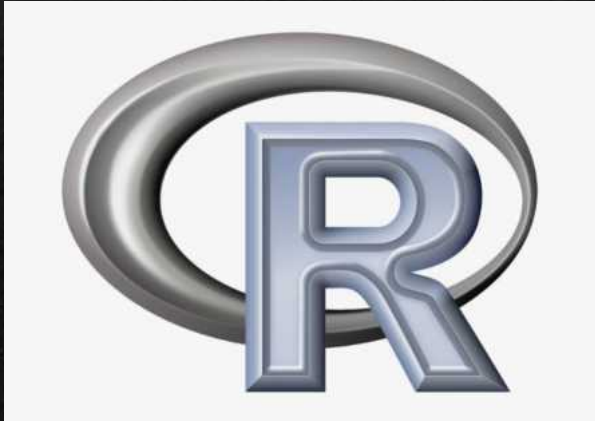
<sup>1</sup>Universidade do Estado do Rio de Janeiro, Programa de Pós-graduação em Engenharia Química, Rua São Francisco Xavier, 524, Maracanã, 20550-900, Rio de Janeiro, RJ, Brazil

<sup>2</sup>Universidade do Estado do Rio de Janeiro, Faculdade de Tecnologia, Rodovia Presidente Dutra, km 298, Polo Industrial, 27537-000, Resende, RJ, Brazil

<sup>3</sup>Universidade de São Paulo, Instituto de Ciências Matemáticas e de Computação, Avenida Trabalhador São-Carlense, 400, 13560-970, São Carlos, SP, Brazil



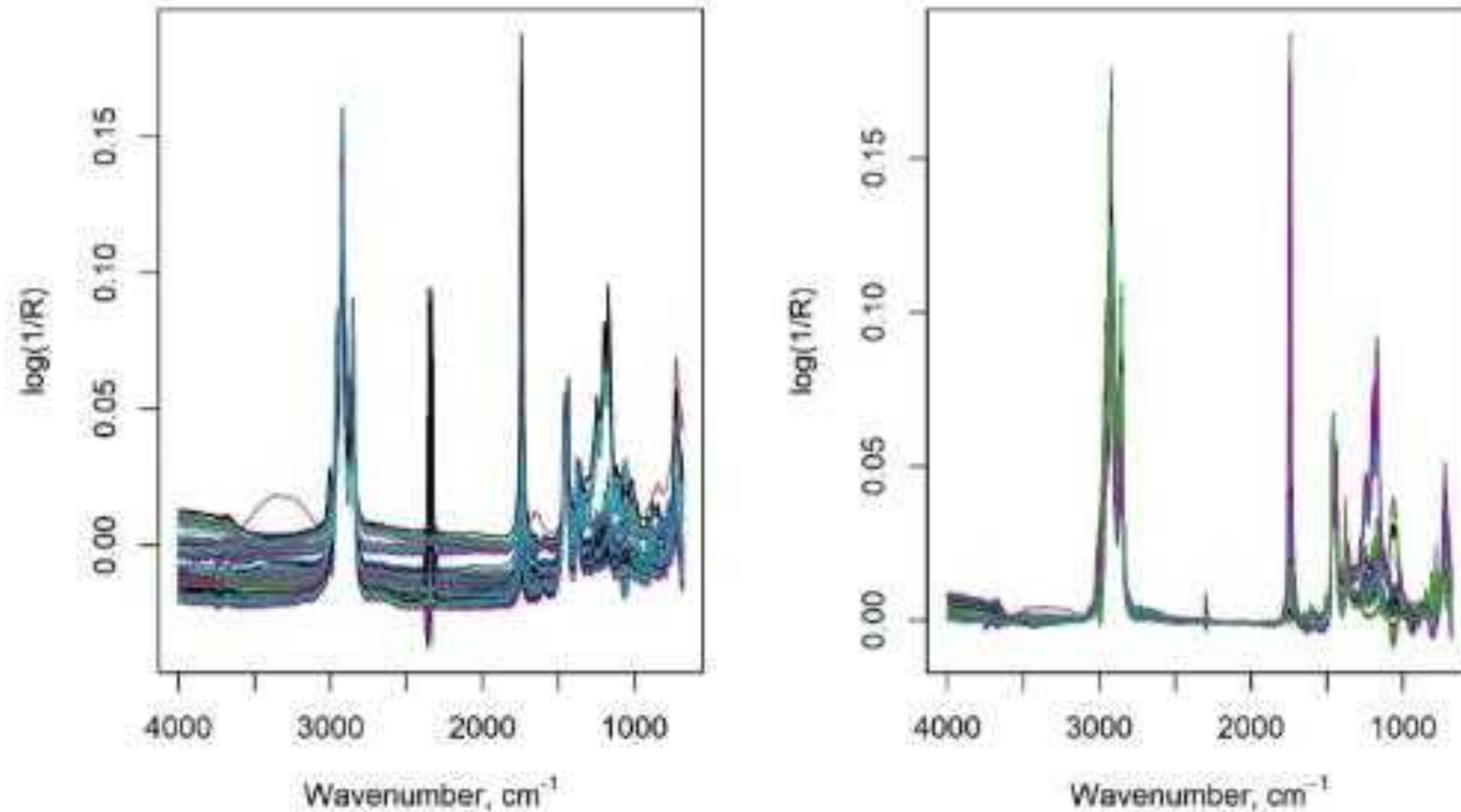
# Methodology



Notebook Samsung  
CORE i7, 8th Gen,  
16 GB RAM,  
256 GB SSD,  
Screen 15.6",  
NVIDIA GEFORCE GTX 1650



### Mid-Infrared spectra



**Figure 1.** Raw mid-infrared spectra of the biodiesel samples (left) and after sample removal (Am158), followed by the replacement of the peak region assigned to the ZnSe crystal by the average of 50 variables above and below the region (right).





### ***Predicting the CFPP of biodiesel samples using MIR Spectroscopy data***

Table III summarizes the results obtained for the predicted models using  $R^2$  and RMSEP as the figures of merit for the CFPP of biodiesel samples. In this table, it was only reported predict models with  $R^2 \geq 0.70$ .

**Table III.** Figures of merit from predicting models of CFPP of biodiesel samples using the MIR Spectroscopy data

| <b>Model</b>                   | <b><math>R^2</math></b> | <b>RMSEP, °C</b> | <b>Time Taken, s</b> |
|--------------------------------|-------------------------|------------------|----------------------|
| Bayesian Ridge                 | 0.95                    | 1.38             | 0.25                 |
| Lasso Lars CV                  | 0.95                    | 1.39             | 8.73                 |
| Ridge                          | 0.94                    | 1.45             | 0.09                 |
| Ridge CV                       | 0.94                    | 1.45             | 0.10                 |
| AdaBoosting Regressor          | 0.94                    | 1.46             | 4.17                 |
| Orthogonal Matching Pursuit CV | 0.94                    | 1.49             | 0.48                 |
| Extra Trees Regressor          | 0.93                    | 1.60             | 6.95                 |
| Huber Regression               | 0.93                    | 1.64             | 0.78                 |
| MLP Regressor                  | 0.92                    | 1.69             | 3.22                 |
| Linear SVR                     | 0.91                    | 1.76             | 1.66                 |
| Passive Aggressive Regressor   | 0.86                    | 2.25             | 0.23                 |
| Tweedie Regressor              | 0.86                    | 2.26             | 0.21                 |
| Transformed Target Regressor   | 0.85                    | 2.32             | 0.22                 |
| Linear Regressor               | 0.85                    | 2.32             | 0.15                 |
| Lasso Lars IC                  | 0.83                    | 2.46             | 0.35                 |
| Decision Tree Regressor        | 0.81                    | 2.60             | 0.43                 |
| K Neighbors Regression         | 0.77                    | 2.86             | 0.08                 |
| Random Forest Regressor        | 0.74                    | 3.06             | 21.93                |
| Kernel Ridge                   | 0.70                    | 3.31             | 0.08                 |
| Bagging Regressor              | 0.70                    | 3.31             | 2.13                 |



**Table IV.** Figures of merit from predicting models of kinematic viscosity at 40 °C of biodiesel samples using MIR Spectroscopy data after log transformation

| Model                            | R <sup>2</sup> | RMSEP, mm <sup>2</sup> s <sup>-1</sup> | Time Taken, s |
|----------------------------------|----------------|----------------------------------------|---------------|
| Extra Trees Regressor            | 0.94           | 0.01                                   | 11.60         |
| Bayesian Ridge                   | 0.93           | 0.02                                   | 0.28          |
| Ridge                            | 0.92           | 0.02                                   | 0.08          |
| Ridge CV                         | 0.92           | 0.02                                   | 0.09          |
| Nu SVR                           | 0.92           | 0.02                                   | 0.20          |
| Linear Regression                | 0.92           | 0.02                                   | 1.65          |
| Extra Tree Regressor             | 0.92           | 0.02                                   | 0.20          |
| Huber Regression                 | 0.91           | 0.02                                   | 1.02          |
| Lasso CV                         | 0.90           | 0.02                                   | 83.50         |
| Orthogonal Matching Pursuit      | 0.89           | 0.02                                   | 0.45          |
| Lasso Lars CV                    | 0.89           | 0.02                                   | 8.66          |
| Linear Regression                | 0.89           | 0.02                                   | 0.15          |
| Transformed Target Regressor     | 0.89           | 0.02                                   | 0.15          |
| Tweedie Regressor                | 0.88           | 0.02                                   | 0.12          |
| K Neighbors Regressor            | 0.88           | 0.02                                   | 0.10          |
| Gamma Regressor                  | 0.87           | 0.02                                   | 0.24          |
| LGBM Regressor                   | 0.87           | 0.02                                   | 1.59          |
| Poisson Regressor                | 0.86           | 0.02                                   | 0.13          |
| Hist Gradient Boosting Regressor | 0.86           | 0.02                                   | 14.84         |

# A summary of models and some of their characteristics

| Model                   | Allows $n < p$ | Preprocessing | Interpretable | Automatic feature selection | # Tuning parameters | Robust to predictor noise | Computation time |
|-------------------------|----------------|---------------|---------------|-----------------------------|---------------------|---------------------------|------------------|
| Linear regression,#     | Not            | CS, NZV, Corr | Yes           | Not                         | 0                   | Not                       | Yes              |
| Partial least squares   | Yes            | CS            | Yes           | May be                      | 1                   | Not                       | Yes              |
| Ridge regression        | Not            | CS, NZV       | Yes           | Not                         | 1                   | Not                       | Yes              |
| Elastic net & Lasso     | Yes            | CS, NZV       | Yes           | Yes                         | 1-2                 | Not                       | Yes              |
| Neural networks         | Yes            | CS, NZV, Corr | Not           | Not                         | 2                   | Not                       | Not              |
| Support vector machines | Yes            | CS            | Not           | Not                         | 1-3                 | Not                       | Not              |
| MARS/FDA                | Yes            |               | Yes           | Yes                         | 1-2                 | May be                    | May be           |
| K-Nearest Neighbours    | Yes            | CS, NZV       | Not           | Not                         | 1                   | May be                    | Yes              |
| Single trees            | Yes            |               | Yes           | Yes                         | 1                   | Yes                       | Yes              |

CS = centering and scaling; NZV = remove near-zero predictors; Corr = remove highly correlated predictors; # regression only

| Model                          | Allows $n < p$ | Preprocessing | Interpretable | Automatic feature selection | # Tuning parameters | Robust to predictor noise | Computation time |
|--------------------------------|----------------|---------------|---------------|-----------------------------|---------------------|---------------------------|------------------|
| Model Trees / Rules, #         | Yes            |               | May be        | Yes                         | 1-2                 | Yes                       | Yes              |
| Bagged Trees                   | Yes            |               | Not           | Yes                         | 0                   | Yes                       | May be           |
| Random Forest                  | Yes            |               | Not           | May be                      | 0-1                 | Yes                       | Not              |
| Boosted Trees                  | Yes            |               | Not           | Yes                         | 3                   | Yes                       | Not              |
| Cubist, #                      | Yes            |               | Not           | May be                      | 2                   | Yes                       | Not              |
| Logistic regression, **        | Not            | CS, NZV, Corr | Yes           | Not                         | 0                   | Not                       | Yes              |
| {LQRM} DA, **                  | Not            | NZV           | May be        | Not                         | 0-2                 | Not                       | Yes              |
| Nearest shrunken centroids, ** | Yes            | NZV           | May be        | Yes                         | 1                   | Not                       | Yes              |
| Naive Bayes, **                | Yes            | NZV           | Not           | Not                         | 0-1                 | May be                    | May be           |
| C5.0, **                       | Yes            |               | May be        | Yes                         | 0-3                 | Yes                       | Not              |

CS = centering and scaling; NZV = remove near-zero predictors; Corr = remove highly correlated predictors; # regression only; \*\* classification only